

上海交通大学硕士学位论文

基于 pythonOCC 的零件
报价系统研究与开发

硕 士 研 究 生：刘屹冬

学 号：117291910031

导 师：柳伟 副研究员

申 请 学 位：工程硕士

学 科：材料工程

所 在 单 位：材料科学与工程学院

答 辩 日 期：2020 年 1 月

授予学位单位：上海交通大学

Dissertation Submitted to Shanghai Jiao Tong University
for the Degree of Master

**Research and Development of Part
Quotation System Based on PythonOCC**

Candidate:	Liu Yidong
Student ID:	117291910031
Supervisor:	Associate Prof. Liu Wei
Academic Degree Applied for:	Master of Engineering
Speciality:	Materials Engineering
Affiliation:	School of Materials Science and Engineering
Date of Defence:	Jan, 2020
Degree-Confering-Institution:	Shanghai Jiao Tong University

基于 pythonOCC 的零件报价系统研究与开发

摘 要

随着制造业向数字化、智能化的转型升级，企业间的竞争日趋激烈。在这样的背景下，在产品生产的早期阶段进行快速准确的报价是必不可少的。在制造业转型升级的过程中，CAD 模型已经成为了贯穿整个生产周期的信息载体，这些蕴含了丰富信息的 CAD 模型成为了可以利用的珍贵资源。

针对这一背景，本文设计了一种以零件的中性格式 (.stp) B-rep 模型文件作为输入的报价系统，并利用 pythonOCC 作为几何内核完成了系统的开发。该系统采用分而治之的思想，将输入的零件模型分为标准件、存在对应报价算法的零件和一般零件。对于这三类零件，分别采用针对性的报价策略进行报价。本文的具体研究内容如下：

设计并实现一个通用的报价系统框架，该系统以中性格式的 B-rep 模型文件 (.stp 文件) 为输入。通过特征识别、模型检索等技术，判断零件的类别。对于不同类别的零件，系统针对性地选择适合的报价算法。系统开放了可以轻易拔插的接口，研究人员或者用户自己可以非常快捷方便地将报价算法及相关数据信息植入报价系统。

研究标准件、存在报价算法的零件和一般零件分别对应的报价策略。对于标准件零件，设计并实现一种能够判别输入零件所属标准件类别并得到其规格参数的算法；对于存在对应报价算法的零件，以轴类零件为例，设计并实现轴类零件的报价算法，包括轴类零件的判定、加工特征的识别和加工成本的计算；对于一般零件，给出一个报价算法以保证系统的完整性和可用性，应当充分利用现有的 CAD 模型资源，并考虑材料、特征等因素对价格的影响。

以 pythonOCC 作为系统的几何内核，实现模型信息的读取、显示交互、特征识别和模型检索。同时以 PyQt 为界面框架，MySQL 为数据库，实现系统客户端的开发和服务端的搭建。

关键词：报价系统、分治、特征识别、标准件、轴类零件、pythonOCC

RESEARCH AND DEVELOPMENT OF PART QUOTATION SYSTEM BASED ON PYTHONOCC

ABSTRACT

With the transformation and upgrading of manufacturing to digitalization and intelligence, competition among enterprises is becoming increasingly fierce. In this context, quick and accurate quotations are essential in the early stages of product production. In the process of manufacturing transformation and upgrading, CAD models have become information carriers throughout the entire production cycle. These CAD models, which contain rich information, have become valuable resources that can be used.

Under such background, this paper designs a quotation system that takes the neutral format B-rep model file (.stp) of the part as input, and uses pythonOCC as the geometric kernel to complete the system development. The system adopts the idea of divide and conquer. The input part model is divided into standard parts, parts with existing corresponding quoting algorithm and general parts. For these three types of parts, targeted quoting strategy is used for quoting. The specific research contents of this paper are as follows:

Design and implement a universal quote system framework that takes a neutral format B-rep model file (.stp file) as input. Through the technology of feature recognition and model retrieval, the category of parts is obtained. For different types of parts, the system selects the appropriate quoting algorithm. The system opens an interface that can be easily plugged in. Researchers or users can quickly and easily insert the quotation algorithm and related data information into the quotation system.

Investigate the corresponding quotation strategies for standard parts, parts with quoting algorithms, and general parts. For standard parts, design

and implement an algorithm that can determine the type of standard part to which the input part belongs and obtain its specifications; For parts with corresponding quoting algorithms, take shaft parts as an example, design and implement the quotation algorithm for shaft parts, including the determination of shaft parts, the identification of processing features, and the calculation of processing costs; For general parts, a quotation algorithm is given to ensure the integrity and availability of the system. Existing CAD model resources should be fully utilized, and the impact of materials, characteristics and other factors on prices should be considered.

Use pythonOCC as the geometric kernel of the system to realize reading of model information, display interaction, feature recognition and model retrieval. At the same time, PyQt is used as the interface framework, and MySQL is used as the database to implement the development of the system client and the construction of the server.

KEY WORDS: quotation system, divide and conquer, feature recognition, standard part, shaft part, pythonOCC

目 录

摘 要	I
ABSTRACT	III
目 录	V
第一章 绪论	1
1.1 引言	1
1.2 研究背景与意义	1
1.2.1 产品报价的意义	1
1.2.2 CAD 技术的发展	2
1.3 研究现状	3
1.4 关键技术	5
1.4.1 基于内容的实体模型检索技术	5
1.4.2 特征识别技术	6
1.4.3 pythonOCC 及其开发	9
1.5 研究内容	10
1.6 本章小结	12
第二章 基于 pythonOCC 的特征识别	13
2.1 引言	13
2.2 特征识别的流程	13
2.2.1 相关标准及类库介绍	13
2.2.2 预处理	16
2.2.3 提取种子环	20
2.2.4 基本特征的识别	22
2.2.5 指定特征的识别	24
2.3 螺纹特征的识别	27
2.3.1 螺纹特征的识别流程	27
2.3.2 螺纹特征参数及提取流程	29
2.4 特征树的结构设计与构建	31
2.4.1 特征树的结构	31
2.4.2 特征树的构建	32
2.5 本章小结	33
第三章 标准件的报价	35

3.1 引言	35
3.2 标准件的分类.....	35
3.2.1 分类规则与编码表示	35
3.2.2 编码的数据结构	36
3.2.3 分类流程	39
3.3 标准件的参数提取	41
3.4 标准件类别的扩展	41
3.5 验证与分析	43
3.6 本章小结	44
第四章 轴类零件和一般零件的报价	46
4.1 引言	46
4.2 轴类零件的报价	46
4.2.1 轴类零件的判定	46
4.2.2 轴类零件的特征识别	47
4.2.3 轴类零件的价格计算	58
4.3 一般零件的报价	60
4.3.1 基于特征树和形状描述算法的相似性检索	60
4.3.2 基于相似零件的价格计算	63
4.4 验证与分析	65
4.4.1 轴类零件报价验证	65
4.4.2 一般零件报价验证	67
4.5 本章小结	68
第五章 系统设计与实现	69
5.1 系统的架构	69
5.2 系统的服务端	69
5.2.1 数据库的选择	70
5.2.2 存储引擎的选择	71
5.3 系统的客户端	71
5.3.1 基于 PyQt 的界面开发	71
5.3.2 基于 pythonOCC 的内核开发	73
5.3.3 数据库的连接与访问	74
5.4 系统的扩展性分析	75
5.4.1 标准件类别的扩展性	75
5.4.2 报价算法的扩展性	76

5.4.3 成本信息的扩展性.....	76
5.5 系统使用示例.....	77
5.6 本章小结	79
第六章 总结与展望	80
6.1 全文总结	80
6.2 研究展望	81
参考文献	82
致谢	87
攻读学位期间的学术成果	89

第一章 绪论

1.1 引言

制造业是国民经济的支柱产业，为社会创造了大量的产品和就业机会，也是帮助我国保持国际竞争力、维持国际地位的重要产业，但是我国的制造业一直大而不强。为了推进我国从制造大国向制造强国转变，国务院于 2015 年出台了《中国制造 2025》白皮书，全力推动制造业向数字化、智能化的转型升级。

在产品的生产过程中，报价是必不可少的环节，传统的方式是报价人员根据产品信息，结合企业自身的生产条件，在与设计人员、研发人员和操作人员沟通的基础上得到产品的价格。这一过程不仅耗时耗力，而且容易出现差错。随着制造业向数字化、智能化的转型与升级，快速、准确且自动化的产品报价系统成为企业的迫切需要。

1.2 研究背景与意义

1.2.1 产品报价的意义

制造业不但是我国经济的主体支柱，也是我国经济增长的重要领域。2019 年上半年，我国制造业的生产总值达到了 13.29 万亿元，占工业 GDP 的 87%，占总 GDP 的 29%^[1]。除此之外，制造业创造了大量的就业机会，为解决我国的就业问题做出了巨大贡献。然而，我国的制造业一直处于大而不强的尴尬境地。随着人口红利的消失，制造业用人成本逐年提升，我国已逐渐失去人力成本方面的优势。在国际竞争日益激烈的今天，制造业的改革与转型升级势在必行。

2010 年，美国提出“再工业化”战略，旨在重振经济危机后的美国制造业，重塑美国工业辉煌；2013 年，德国推出“工业 4.0”项目，旨在利用信息化技术促进制造业变革，将生产中的供应、制造、销售等信息数据化，利用物联网等技术联通各个环节，并将技能、经验进行数据化、智能化，最终实现生产制造全周期的效率提升与性能优化；相对应的，国务院于 2015 年 5 月印发战略文件《中国制造 2025》，以创新驱动为核心，促进制造业数字化、网络化、智能化，推动传统产业向中高端迈进，逐步化解产能过剩，加快制造业结构升级，以期到 2025 年能够基本实现工业化，从制造业大国跻身制造业强国的行列。

随着生产力的不断提高，制造业企业之间的竞争愈发激烈。为了在竞争中获得优势，企业必须从产品生产的各个环节去进行优化，以缩短产品生产周期、减少产品生产成本和提高产品生产质量。在这一背景下，如何缩短产品的生产周期

成为了制造业企业能否从竞争中脱颖而出的关键因素之一。

报价的时机和准确性往往会直接影响企业的运营状况^[2]。报价过高，企业失去竞争优势；报价过低，企业的利润直接减少；报价过晚，企业可能错失订单。因此，快速、准确的报价对于制造业企业至关重要，是赢得订单、赢得潜在客户、保持企业竞争力的基本前提。

目前，绝大多数的企业在进行产品报价时，往往依靠繁琐的市场调研、复杂的数据分析计算并结合企业自身的人力、设备成本和工程师的相关经验，考虑诸如人力成本、设备成本、生产工艺等许多方面进行报价。从收到报价申请到提出最终报价，整个报价过程相当复杂和耗时耗力，通常需要财务、设计、研发、生产各个部门密切配合，

与此同时，现代社会发展日新月异，用户的需求也日渐多样化和个性化，以客户需求为导向的制造业已成为发展的必然趋势，相当一部分的订单可能包含独特的需求。由于产品在市场上还未出现，且工程师没有相关经验，使得报价的难度和复杂性进一步提升。

除此之外，随着信息时代的到来，人们的生活节奏大大加快。信息的急速膨胀无疑会促进某些好的产品在设计在短时间内需求量激增，五花八门的新产品也层出不穷，产品的种类越来越多，市场的需求越来越迫切。这就更加要求企业的生产必须能够快速响应市场，在低成本高质量的前提下，以最短的时间开发出用户和市场需要的产品，其中，能否快速估算产品的成本是最为关键的问题之一^[3]，它关系到销售部门能否合理报价，直接影响企业的利益。

基于上述分析，产品的快速报价系统对于企业快速响应客户需求、缩短产品生产周期、抢占市场并最大化自身利润有着重要的实际意义。

1.2.2 CAD 技术的发展

数字化制造是数字化技术和制造技术的融合，是智能制造的基础，包含以 CAD（Computer Aided Design，计算机辅助设计）、CAM（Computer Aided Manufacturing，计算机辅助制造）和 CAE（Computer Aided Engineering，计算机辅助工程）等产品设计和仿真为主体的上游骨干技术群，由 NC（Numerical Control，数字控制）、CNC（Computer number control，计算机数字控制）、无图纸制造、自动化机器人和物流系统组成的下游数字控制制造技术群，以 IGES（The Initial Graphics Exchange Specification，初始化图形交换规范）、STEP（Standard for the Exchange of Product Model Data，产品模型数据交互规范）、PDM（Product Data

Management, 产品数据管理) 和 ERP (Enterprise Resource Planning, 企业资源计划) 等为主体的制造信息支持技术群^[4]。

法国汽车工程师皮埃尔·贝塞尔于 1962 年提出贝塞尔曲线^[5,6], 通过改变控制点以实现曲线形状的改变, 并将其广泛用于汽车车身设计。法国达索飞机制造公司在此基础上, 推出了三维曲面造型系统 CATIA, 标志着 CAD 技术从模仿工程图纸、提高绘图效率这种模式中解放, 第一次用计算机中的数据完整地描述零件的形状。在这之后, 工业界又相继出现了 UG、SolidWorks、Pro/E (于 2011 年更名为 Creo)、Inventor 等各有千秋的 CAD 软件, 产品的生产效率得到了极大的提高。时至今日, 利用各种 CAD 软件建模而成的三维 CAD 模型已经成为了产品生产过程中必不可少的信息载体。这些三维的 CAD 模型文件, 包含了零件全部的形状信息, 同时关联了大量的实际生产中的相关数据, 蕴含着丰富的经验与知识。

随着计算机硬件性能的提升、数据科学的发展和智能制造的提出, 制造业企业所拥有的大量的以三维 CAD 模型为中心的生产信息成为了极其宝贵的数据资源。据不完全统计, 早在 2000 年, 存在的 CAD 模型就已经超过了 200 亿个^[7]。如何将这些海量的 CAD 模型, 用于提升制造业的产品质量、生产效率, 是制造业充分发挥数字化技术潜力、真正走向智能化转型的关键之一。

1.3 研究现状

针对产品的快速报价, 国内外相关研究者进行了大量的研究, 取得了相当大的进展。

南洋理工大学的 N.S.Ong^[8]提出了一种针对 PCB (Printed Circuit Board, 印刷电路板) 的基于活动的估价方案, 以帮助设计人员估算印刷电路板组件在设计的前期概念阶段的制造成本。该方案通过分析活动图表、工作表和成本构成表以确定一个 PCB 生产过程中的成本。

湖南大学的单汨源和北京大学的于永和^[9], 根据产品的 BOM (Bill of Material, 物料清单) 结构提取相关参数, 训练了一个 3 层 BP 神经网络以预测注塑件产品的价格, 经过对 6 个注塑件样本进行验证, 成本估算误差控制在 5.5% 以内。

浙江大学的苏少辉等^[10]对 ETO (Engineering To Order, 面向订单的设计) 产品建立了一种成本估算方法, 将客户需求与材料、工艺间的关系表示为成本模型, 在成本模型的控制下, 根据客户需求参数进行新零件的成本估算。

北京航空航天大学的魏秋红等^[11]通过对成本估算方法和产品报价现状的分析, 给出了基于 PLM (Product Lifecycle Management, 产品生命周期管理) 的快速报

价系统的报价模型以及提供和维护基础数据的方法。

山东大学的李杨等^[12]提出 PDM 技术下的报价策略, 在基于 Web 环境下结合产品功能和结构配置及其成本估算的算法, 开发了基于 B/S 的报价系统。

这些报价系统是以 BOM 表、工作表等形式的文本信息作为报价系统的输入, 并不是直接研究零件本体。

西华大学的邢宪才等^[13]针对覆盖件模具, 利用企业历史数据, 以模具的长、宽、高、形面的复杂程度和板料的厚度作为参数, 训练神经网络模型, 计算结果显示了该方法的可行性。

天津大学的熊立华等^[14]基于实例库推理的方法, 通过实例库的组织、实例索引、实例初始化、实例搜寻和检索、实例改编, 实现了产品价格的快速响应。

这些报价系统以零件本体为研究对象, 但其要求的输入信息需要人工归纳、整理, 不利于报价过程的自动化。并且这一过程费时费力, 可能会出现误差和错误。

CAD 文件不仅是现代制造业必不可少的信息载体, 而且可以准确、完整地描述一个零件最重要且复杂的信息——形状和尺寸, 因此以 CAD 模型作为报价系统的输入更加合适。

南昌大学的张磊等^[15]以 CAD 软件 Pro/E 为平台, 实现了实体特征的快速识别提取, 并基于此设计了注射模具估价系统。中国科学技术大学的练国富等^[16]同样以 Pro/E 为平台, 设计了锻压机床材料成本快速报价系统, 一定程度上为整机报价提供了更为准确的数据。合肥工业大学的董玉德等^[17]对 CAD 软件 Creo 进行了二次开发, 利用了 Creo 提供的 API (Application Programming Interface, 应用程序接口) 对建模命令进行识别, 在一定程度上实现了加工特征的识别, 并以此为基础对零件进行报价。

这些报价系统利用了 CAD 模型中蕴含的零件信息, 具有相当大的实用价值。但是, 其内部原理是利用商用 CAD 软件对外提供的 API 来读取建模历史, 以分析零件的特征, 从而进行报价。由于不同的 CAD 软件之间数据互不相通, 不同工程师的建模习惯也不尽相同, 并且基于商用 CAD 软件二次开发的报价系统需要解决版权问题, 企业不得不额外为此支付相当昂贵的费用。此外, 这些系统把研究重点放在如何实现具体某一类零件的精确报价上, 报价系统不具有通用性、适用范围较窄。

基于以上原因, 以通用的中性格式 B-rep 模型文件 (如.igs、.stp) 作为输入, 使用开源、免费的几何造型内核或 CAD 软件进行报价系统的开发, 是开发商用的

通用零件报价系统的基础，具有重要的实际意义。

1.4 关键技术

随着制造业数字化程度的不断提高，三维 CAD 模型已经成为了产品设计、研发、制造、分析的核心载体。三维模型由于包含了产品全部的形状信息、设计意图表达直观，并且支持仿真优化、工艺的规划等环节的集成^[18]，因此以三维模型为信息载体的数字化制造正在被越来越多的企业所应用。

1.4.1 基于内容的实体模型检索技术

三维模型的检索方法主要包括两种：基于语义的检索^[19]和基于内容的检索^[20]。前者是把三维模型低层的几何数据转换为高层的知识表示，用文本或编码对三维模型进行标注，通过文字或编码的匹配实现检索；后者是指完全从模型本身包含的数据信息出发，将形状、尺寸、材料等内容通过某些算法提取并进行比较以实现检索。

基于语义的检索具有形式简单、容易实现、效率高等优点，但是模型的语义描述需要人工添加，不够准确、全面；基于内容的检索完全从模型本身出发，检索结果更为准确，本课题所涉及的检索技术就是基于内容的检索技术。

实体模型是三维模型的一种，在机械 CAD 中最为常用，主要的表达方式包括 B-rep 和 CSG（Constructive Solid Geometry，构造实体表示法）。前者是由点、线、面及其之间的拓扑关系构成了实体模型边界，后者是由圆柱、长方体等基本几何体之间的一系列布尔操作的构成。三维模型的检索本质上是对两个模型之间进行相似性评估，针对 B-rep 实体模型的相似性评估方法，分为以下几种：

（1）基于形状分布的方法

Osada^[21]等提出形状分布算法，通过对模型上采样点的几何属性的概率分布曲线进行比较，来表示两个模型的相似程度。经过试验，以模型上随机两点之间的距离作为形状描述符（D2）时效果最好。这种方法能够大体描述模型的形状，适合于粗略聚类。但是这种方法只考虑了形状相似性，没有考虑到两种实体模型拓扑结构的关系，对形状大体相似但细节特征截然不同的两种模型不能很好地区分。为此 Ip^[22]、Rea^[23]、Zou^[24]等对形状分布算法进行改进，在一定程度上提升了形状分布算法对细节特征的描述能力。

尽管形状分布算法对实体模型的区分度随细节特征的增加而降低，但由于其计算简单、稳定性较好，并且对基本几何体区分度良好，因此被广泛使用。

（2）基于属性邻接图的方法

实体模型的边界表示中可以提取出面边图,然后通过图匹配的方法比较图结构的相似性来度量两个实体模型的相似性。由于子图匹配是一个 NP 完全问题^[25],在模型复杂、面边比较多时,该方法的复杂度极高。McWerther^[26,27]通过添加面类型、边凹凸性等属性提高了图结构对拓扑相似性的描述能力。El-Mehalawi 和 Allen Miller^[28,29]在图匹配过程中引入约束条件,简化了非精确图匹配的过程。马露杰等^[30]将面边图划分为层次结构,用数值计算代替图结构的匹配,降低了计算量。

(3) 基于拓扑图的方法

Iyer^[31,32]等通过体素化得到实体模型的骨架图,降低了图匹配算法的时间复杂度。Gao 等^[33]提出了多分辨率的蔓延骨架图,通过预设层次权重以划分分辨率粒度。白静等^[34]在此基础之上提出了一种粒度由粗到细的图匹配算法。

Reeb 图是 George Reeb 于 1946 年提出的一种描述三维模型拓扑结构的方法,其复杂度和骨架图相当,用曲率、高度、测地距离等连续函数来描述特征间的连通关系。Hilaga 等^[35]提出了一种基于 Reeb 图的网格模型匹配方法。Bespalov 等^[36]以此为基础对网格化的 CAD 模型进行了检索,发现基于 Reeb 图的模型检索对拓扑结构的微小变化过于敏感,需要进一步寻找更加合适的连续函数。

(4) 基于特征的方法

为了降低面边图的匹配复杂度,研究人员提出了以特征为基本单元的相似性比较方法。其基本思想就是利用特征识别技术将实体模型中的形状特征或加工特征进行提取,进而将由面-边构成的边界模型转化由特征构成的特征模型,再以图结构或树结构进行表示,用图匹配或多粒度的评估方式计算两个实体模型的相似性。在这一方面,研究人员也进行了大量的研究工作^[37-47]。

1.4.2 特征识别技术

特征识别是指从零件的实体模型中自动地提取出具有特定工程意义的一组几何形状,这也是实现 CAD/CAM/CAPP (Computer Aided Process Planning, 计算机辅助工艺过程设计) 集成的技术基础。特征识别技术最早的研究起源于 20 世纪 70 年代,剑桥大学 CAD 中心的 Grayer^[48]在 1975 年第一次尝试从一个零件的实体模型中自动地提取出如型腔等可以帮助计算数控加工轨迹的几何形状。剑桥大学 CAD 中心的 Kyprianou^[49]于 1980 年在其博士论文中第一次正式提出了现有的特征识别思想,从而确定了基于 B-rep (Boundary-representation, 边界表示) 的特征识别基础,本课题所研究的对象正是零件的 B-rep 模型。随着特征识别技术应用

越来越广泛，对特征识别技术的研究受到了高度的重视，越来越多的特征识别算法被不断提出。

截至目前，特征识别方法已经出现了很多。从整体上进行划分，主流的特征识别方法大致分为两类，一类是基于边界匹配的特征识别，另一类是基于体积分解的特征识别^[50]。下面对两种特征识别方法进行简单介绍。

（1）基于边界匹配的特征识别

每种特征都有一定的边界模式，基于边界匹配的特征识别就是采用几何或拓扑的方法去搜索 **B-rep** 模型，寻找其中与特征边界模式相符的区域，从而识别出零件中的特征。将这种以零件的 **B-rep** 模型为基础，通过匹配特征的边界模式进行特征识别的方法归为基于边界匹配的特征识别，这也是研究最早、应用最广泛的特征识别方法。其基本步骤如下：

首先对 **B-rep** 模型进行搜索，将模型中特定区域与每一种类型的特征的边界模式进行匹配，确定该区域是哪一类特征；然后通过分析该区域的几何形状、尺寸，确定已识别的特征的参数，构造出一个完整的带有特征的几何模型；最后对能够合并的特征进行组合。

基于边界匹配的特征识别方法具体包括以下几种：

a. 基于规则的特征识别：

该方法是通过定义一系列规则来定义一个特征的边界模式，这些规则是基于几何、拓扑等知识的积累。对于每一个特征，都需要在规则库里定义一个对应的规则，通过对零件 **B-rep** 模型使用定义好的特征规则进行匹配，从而识别出特征。

这种方法是最早提出的特征识别方法之一，其主要存在的问题有以下几点：一是没有一个能够完备、唯一表示某个特征的统一方式；二是大量的规则匹配使得算法的效率很低；三是该方法无法识别相交特征。

b. 基于图的特征识别：

该方法是用图这种数据结构表示一个特征的边界模型，以 **B-rep** 模型中的面作为结点，以面与面之间的邻接关系作为弧，构成一个面边图。还有研究人员对一般的面边图进行改进，将边的凹凸性添加到弧的属性中，构成属性邻接图^[51]，这一改进使得特征边界模式的图表示更加的完整。

该方法是研究最多、应用最广泛的特征识别方法之一，其主要优点有：一是特征的边界模式有了统一的表示方式，并且该表示方式具有完备性和唯一性；二是特征的图表示易于自动生成。该方法的主要问题在于难以识别相交特征。

c. 基于痕迹的特征识别：

为了解决相交特征识别困难的问题, Marefat^[52]和 Vandenbrande^[53]在 90 年代初第一次提出了基于痕迹的特征识别。痕迹是指一个特征在实体模型中留存下的信息, 相交后的特征虽然已经不存在完整的边界模式, 但仍然在模型中留有痕迹。基于痕迹的特征识别理论上可以识别出所有的特征, 这些痕迹通常是几何或拓扑信息。

Vandenbrande 方法中的具体算法依赖于特征类型, 用户难以添加新的特征类别。Gao S 等^[54]提出了一种基于痕迹的统一定义的特征识别方法, 以特征的最小条件子图作为痕迹, 并提出加工面邻接图的概念, 以提高特征识别的效率。

(2) 基于体积分解的特征识别

特征不仅具有特定的边界模式, 同样也具有特定的体积模式。90 年代, 由于基于边界的特征识别一时难以解决相交特征的识别问题, 以及计算机软硬件技术的发展使得体积分解方法中巨大的计算量不再成为瓶颈, 基于体积分解的特征识别受到了人们的重视。

其基本步骤是: 首先将实体分解为凸体的集合, 然后重新组合这些凸体得到与特征相对应的体元, 最后对表示特征的体元进行分类以确定特征的类型。根据体积分解策略的不同, 该方法主要分为两类: 一是基于立体交替的特征识别方法, 二是基于单元体分解的特征识别方法。

a. 基于立体交替的特征识别:

Woo^[55]于 1982 年提出基于立体交替的特征识别方法, 该方法将非凸实体表示为一棵分解树, 这棵树以布尔运算符作为中间结点, 以凸体元作为叶结点, 根结点表示零件实体本身。基于该分解树, 通过自叶结点向上进行布尔运算的组合, 进行形状特征的匹配。

基于立体交替的特征识别存在两个问题: 一是难以适用于带有曲面的物体, 二是计算效率不高。

b. 基于单元体分解的特征识别:

该方法由 Armstrong 首先在 80 年代提出^[56], 单元体是指物体内部、凸性的体积单元, 该方法提出主要是为了自动生成数控代码。General Dynamics 公司为实现加工特征的识别, 对零件的切削体部分(毛坯模型减去设计模型)进行分解, 该方法现在广泛应用于加工特征的识别。

从 90 年代起, Sakurai^[57,58]对基于单元体分解的特征识别展开深入研究, 通过延展零件所有面, 面面求交后分解切削体为一列凸单元体, 通过组合相邻单元体然后去匹配特征的边界模式, 实现特征的识别, 该方法取得了相当大的进展。Shah

和 Shen^[59]提出了一种基于半空间剖分的特征识别方法,有效避免了不必要的面面求交,进而提高了特征识别的效率。

1.4.3 pythonOCC 及其开发

主流的 CAD 软件都是以几何内核为基础而开发的,所谓几何内核是一个定义了大量的图形算法和图形数据存储格式类库。OCC (Open CASCADE)^[60]是一个由法国 Matra Datavision 公司开发的几何内核,与 ACIS (AutoCAD 的几何内核)和 Parasolid (UG、Pro/E、SolidWorks、CAXA 的几何内核)并列,成为三大几何内核。OCC 本身是一个免费的、完全开放源代码的 C++ 对象类库,其几何建模技术可以广泛应用于 CAD、CAM、CAE 等软件的开发中。正是由于其开源免费的特点和强大的实体建模能力,近几年 OCC 在几何造型领域逐渐占据了重要的地位。OCC 类库提供的功能包括:

(1) 二维和三维造型功能

创建持久的基本形状类,如圆柱、圆锥、长方体等。

创建持久的特征类,如倒角、拉伸、抽壳等。

计算实体模型的几何特征,如面积、体积、重心、曲率等。

计算并几何体的变换结果,如投影等。

计算布尔运算的结果,如求和、求差、求交等。

(2) 可视化功能

将持久化的实体模型显示,并可以实现三维缩放、旋转、变焦、遮光等功能。

(3) 应用框架功能

向开发者提供一个应用程序框架,简化基于 OCC 的应用程序开发过程。

(4) 数据交换功能

提供对 IGES、STEP、STL 等格式的数据格式的文件输入输出。既可以对一个中性格式的模型文件进行解析,得到对应的实体模型类对象;也能够将程序中表示一个实体模型的对象,输出为中性格式的模型文件。

python 是一种面向对象的脚本语言,具有语法简单、类库丰富、开发效率高等优点,可以轻易地将其他语言编写的各种模块联结在一起,pythonOCC^[61]是 Thomas Paviot 利用 python 这种“胶水语言”的特性,对 OCC 类库进行封装得到的开源 python 库。以 pythonOCC 作为报价系统的几何内核,开发及运行环境的搭建十分方便,既可以实现对 STEP 等中性格式的 B-rep 模型文件的信息读取,又可以利用 python 语言中丰富的数据分析类库,而且由于 OCC 本身是开源免费的,以

pythonOCC 为几何内核进行软件开发也同样不需要像使用 ACIS 或 Parasolid 一样支付昂贵的版权费用。

1.5 研究内容

针对现有的对于报价系统研究的不足，本课题提出了一种以中性格式（.stp）的三维 CAD 模型为主要输入，基于几何内核 OCC（OpenCascade）的 python 版本 pythonOCC 开发而成的零件报价系统。该系统采用分而治之的思想，利用特征识别等技术对零件进行特征分析，进而对其分类，不同的类别针对性地采用最合适的报价算法。该系统的报价流程如图 1-1 所示。

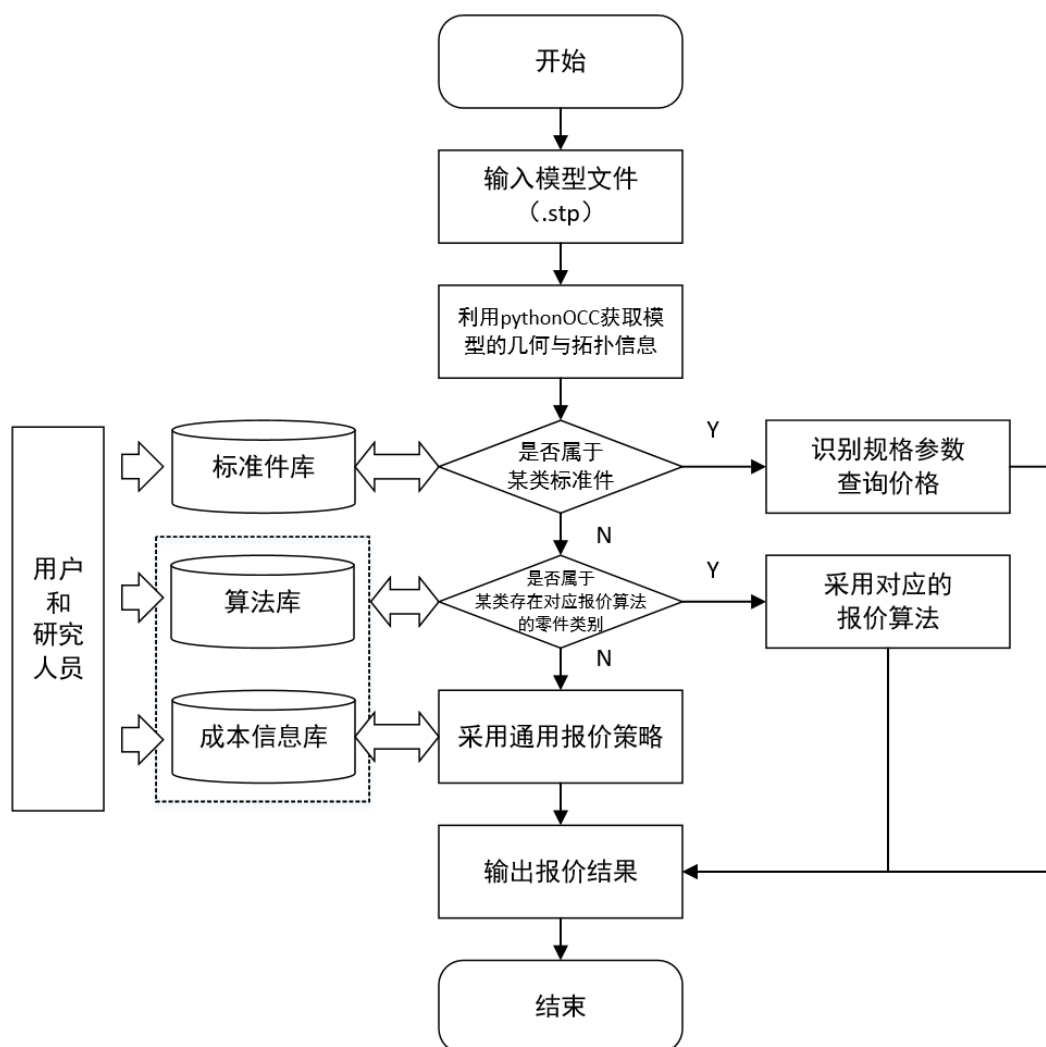


图 1-1 报价系统的流程图
Fig.1-1 Flow chart of quotation system

不同类别零件的加工方式、常用材料、参数类型及其与加工成本的关系都是

不尽相同的，想要用某一种或某几种报价算法对所有种类的零件价格做出准确的报价是不现实的。目前已经有相当多的针对具体某类零件的报价研究，对该类零件针对性地提出特定的报价算法，以期尽可能提高该类零件的报价准确率和报价效率。本课题不以精确地对各种类别的零件得出报价为研究目的，而是旨在设计并实现一个适用于大部分零件的通用报价系统，或者说是报价框架。该报价系统的通用性是依靠分而治之的思想和系统良好的扩展性实现的。该系统支持零件实例库的扩展、分类类别的扩展、分类算法的扩展和报价算法的扩展。新的零件类别及其对应的分类规则、报价算法可以十分方便地植入该系统。

在实现该框架的基础之上，本课题针对一些实际常用的零件类别和特征，设计了报价算法，并植入了该框架，以形成一个完整可用的报价系统。随着各种具体类别零件的报价算法的深入研究，新的更有效的报价算法可以随时植入进系统进行替换。本课题的具体研究内容如下：

（1）设计一个以 pythonOCC 作为几何内核的零件报价系统。该系统需要能够读取.stp 格式的 B-rep 模型文件，可以运行在主流 PC 端，能够显示零件的实体模型并与用户进行交互，并且系统应该是基于网络环境的，以实现资源的共享。

（2）研究基于 pythonOCC 的特征识别，使系统能够识别 B-rep 模型中指定拓扑结构的特征并提取其相应的参数。除此之外，特征类型应当是可以扩展的，用户通过输入简单的描述特征类型的信息，而不需要进行代码层的改动，就能实现特征类别的添加。

（3）设计并验证标准件检索算法，即解决如何根据零件的三维 CAD 模型确定零件是否属于系统数据库中存在的某类标准件以及如何得到该类标准件对应的规格参数的值的问题。标准件的检索在确保检索结果准确率和检索效率的基础之上，应当具有针对标准件类别扩展的自适应能力，即当用户添加新的标准件类别后即可自动检索到新类别，而不需要进行代码层的改动。

（4）以轴类零件为例，设计其对应的报价算法，通过推理轴类零件的加工过程以计算成本，并将该算法植入报价系统。创建加工成本相关数据，通过特征识别具体类型的提取其上具体类型的加工特征的识别与参数提取，进而推理出可行的加工流程，计算加工成本以报价。加工特征的识别应当在确保识别准确的基础之上，应当具有针对加工特征类别扩展的自适应能力，即用户添加新的加工特征类型后即可识别该加工特征，无须在代码层进行修改或扩展。

（5）对于一般零件，设计其报价算法以使整个报价系统完整。采用一种算法实现一般零件的准确报价是极其困难的，本系统提出的一般零件报价算法在准确

性上不做过高要求,主要目的是使整个报价系统的完整,但该算法应当考虑材料、表面精度等非形状因素对零件价格的影响。

(6) 完成该报价系统客户端、服务端的搭建与开发,使该系统可以运行于主流的 PC 设备之上,能够实现一个完整的报价过程。

1.6 本章小结

本章首先介绍了报价系统的研究背景、研究意义与研究现状。着重介绍了现有的报价系统的优缺点。指出目前关于以中性格式实体模型为输入的通用报价系统的空缺,并介绍了本课题提出的通用报价系统所需要用到的几种关键技术。在本章的最后,具体介绍了本课题的研究内容。

第二章 基于 pythonOCC 的特征识别

2.1 引言

特征的概念最早由 J.W.Seltes^[62]于 1978 年提出, 80 年代后期这一概念被广泛接受, 而后相继出现了面向不同领域的多种特征概念, 包括设计特征、制造特征和广义特征等。设计特征强调零件表面的轮廓形状, 一组形成某种特定形状的表面、棱边构成了一个设计特征, 用于修饰零件外观或实现特定的功能。制造特征是对应于某种加工操作而形成的几何形状, 如切削加工去除的体积形成的几何形状。

广义的特征是指因某种目的(如分析、生成或评价设计)而被看作是一个整体的、有着固定几何形状的实体模型的一部分, 它可以作为基本的分析与处理单元。本课题所述的特征是广义上的形状特征, 该特征既包括与零件价格直接相关的加工特征, 也包括了用于零件类别划分而需要识别的设计特征, 因其提取的思想和流程并无差别, 故不进行严格的区分。

特征识别是指从零件实体模型中提取出具有工程意义的几何形状, 是实现 CAD/CAM/CAPP 集成的基础。在以零件实体模型为输入的报价系统中, 首先要解决的问题就是如何从实体模型中提取出我们需要的信息, 这就需要特征识别技术。

本课题设计的报价系统以 pythonOCC 作为几何内核, 特征识别的实现依赖于 pythonOCC 提供的类库, 本章将详细介绍利用 pythonOCC 对零件的 B-rep 模型(.stp 格式文件)进行特征识别, 主要包括特征的边界识别、基本属性提取和指定特征的识别与对应的参数提取。

2.2 特征识别的流程

2.2.1 相关标准及类库介绍

表 2-1 OCC 模块及功能划分

Table 2-1 Module and function division of OCC

Foundation Classes	Modeling Data	Modeling Algorithm	Data Exchange	Visualization	Draw	Application Framework
核心类	几何类	几何工具类	STEP	2D 可视化	脚本测试类	开发框架类
基础数据类	形状属性类	拓扑工具类	IGES	3D 可视化		
数学算法类	拓扑类	建模操作类	STL	网格可视化		
		布尔运算类	VRML			
		特征类				
		网格类				

本课题设计的报价系统，以.stp 格式的零件模型文件作为输入。这种格式的文件是以 STEP（产品模型数据交互规范）为标准，按照 AP-203 或 AP-214 等应用协议用 EXPRESS 语言对实体三维模型进行描述并进行持久化存储。

pythonOCC 是对 OCC 的封装，实际模块划分和功能与 OCC 完全相同，OCC 类库包含 7 个模块，分别提供的主要功能如表 2-1 所示。

pythonOCC 支持对 STEP 数据模型进行读取以进行三维重建，对特征识别而言，实体模型中的拓扑和 pythonOCC 拓扑类的对应关系是必须明确的。表 2-2 给出了不同拓扑在 STEP 中的实体类型和 pythonOCC 拓扑实体类之间的对应关系，图 2-1 给出了这些拓扑实体类的继承关系。

表 2-2 拓扑在 STEP 中的类型和在 pythonOCC 中的类型

Table 2-2 Type of topologies in STEP and topologies in pythonOCC

Topology	STEP Entity	PythonOCC Type	备注
Vertices	vertex_point	TopoDS_Vertex	顶点
Edges	oriented_edge	TopoDS_Edge	边
Loops	edge_curve	TopoDS_Edge	环
	face_bound	TopoDS_Wire	
	face_outer_bound	TopoDS_Wire	
	edge_loop	TopoDS_Wire	
	poly_loop	TopoDS_Wire	
Faces	vertex_loop	TopoDS_Wire	面
	face_surface	TopoDS_Face	
	advanced_face	TopoDS_Face	
Shells	connected_closed_set	TopoDS_Shell	壳体
	oriented_closed_shell	TopoDS_Shell	
	closed_shell	TopoDS_Shell	
	open_shell	TopoDS_Shell	

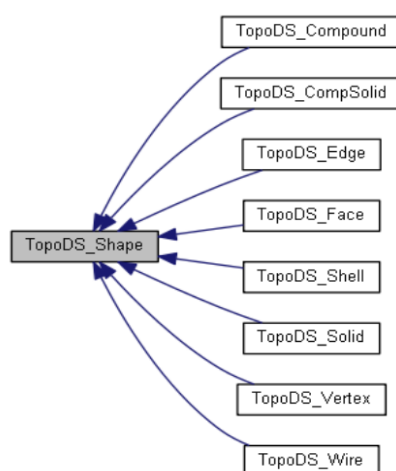


图 2-1 拓扑实体类继承关系^[31]

Fig.2-1 Topology entity class inheritance

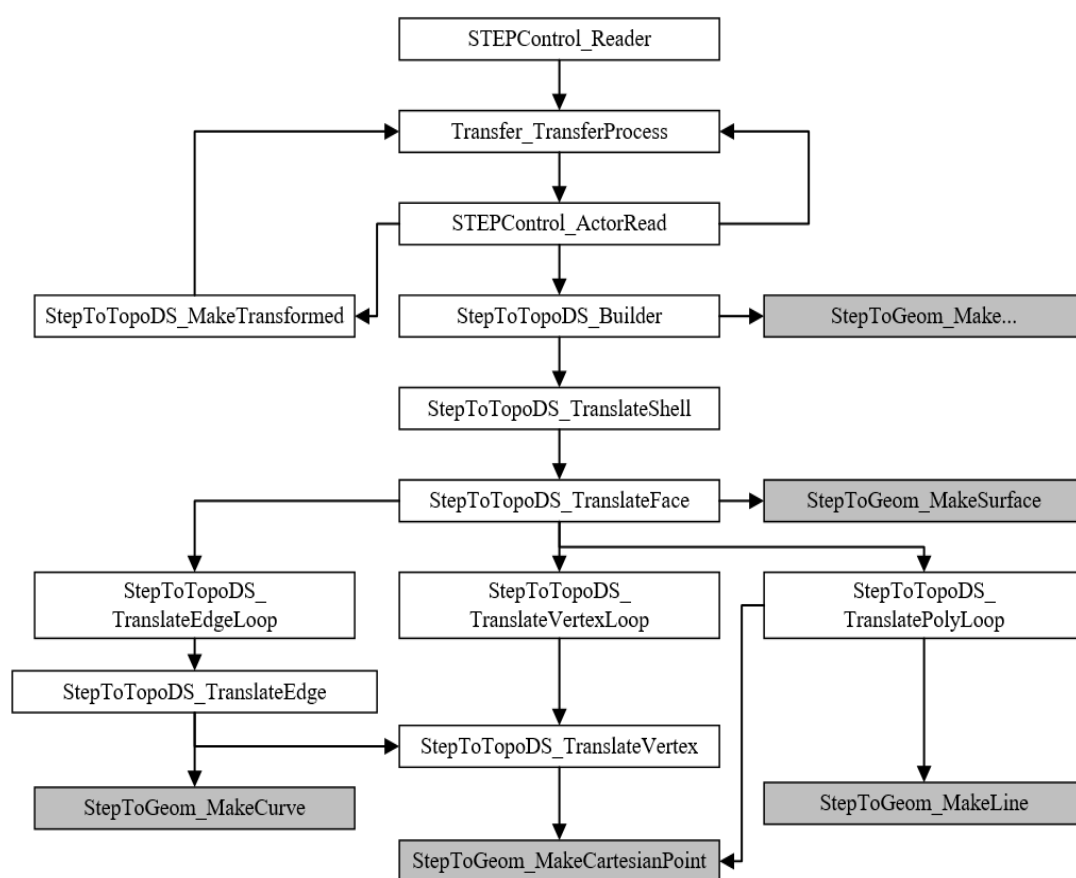


图 2-2 pythonOCC 读取 STEP 文件过程中的类调用结构^[31]
Fig.2-2 Structure of class call during reading STEP file in pythonOCC

特征识别中主要用到的包,除上述提到的读写 STEP 文件的 STEPControl 包、描述拓扑信息的 TopoDS 包、描述几何形状信息的 Geom 包外,还有 TopologyUtils 包和 BrepAdaptor 包。TopologyUtils 包提供了对 TopoDS 包中拓扑实体类的访问工具类; BrepAdaptor 包提供了 BrepAdaptor_Curve、BrepAdaptor_Surface 等类,用于将拓扑实体类进行转换,以得到几何属性参数,如边的长度、边上某点的切向、面的面积、面上某点的法向等信息。

此外, Thomas Paviot 对 pythonOCC 类库还进行了二次封装,封装了 DataExchange、ShapeFactory 和 TopologyUtils 三个包,分别用于文件数据读写、拓扑实体类的创建和拓扑实体类的访问。

利用 pythonOCC 进行特征识别的整体流程如图 2-3 所示,具体步骤在本章后续详细描述。

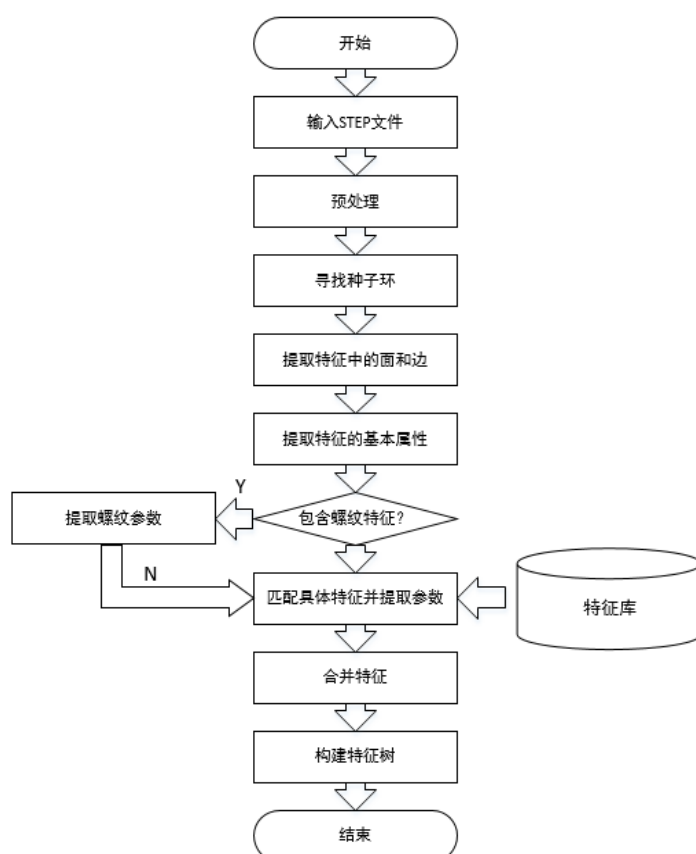


图 2-3 基于 pythonOCC 的特征识别流程图
Fig.2-3 Flow chart of feature recognition based on pythonOCC

2.2.2 预处理

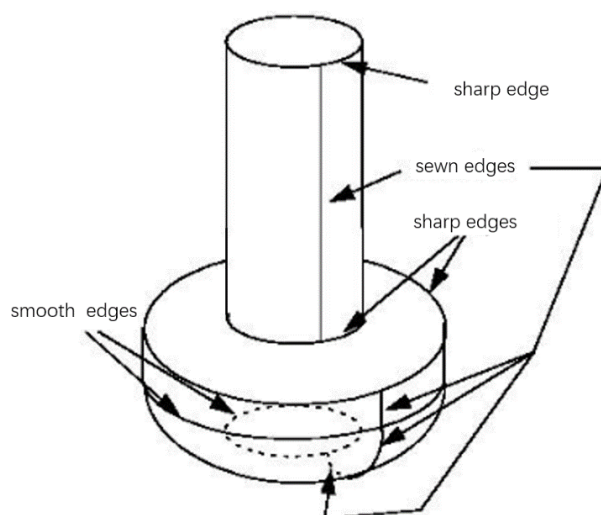


图 2-4 虚边示意图^[31]
Fig.2-4 Sewn edges

在 pythonOCC 自身提供了 `Topology_Explorer` 类用于访问一个 `TopoDS_Shape` 对象,具体而言,通过向 `Topology_Explorer` 对象中注入一个 `TopoDS_Shape` 对象,可以得到这个拓扑实体对象中包含的全部低层拓扑对象,如 `TopoDS_Vertex` (顶点)、`TopoDS_Edge` (边)、`TopoDS_Face` (面) 等。

几何内核在重建 B-rep 模型时会对某些面进行缝合,产生一些本不该出现的隐藏边,本文中称其为“虚边”(Virtual Edge),如图 2-4 中的 sewn edges,同理图 2-4 中 sewn edges 的端点即为“虚顶点”(Virtual Vertex)。这些边和顶点在实际零件中并不存在,会对基于图结构的特征识别算法造成干扰,因此通过 `Topology_Explorer` 得到的面、边、顶点不一定对应实际中真实的面、边、顶点,我们必须设计算法移除这些不存在的虚边和虚顶点,构造与实际一一对应的面、边、顶点,并重新建立他们之间的映射关系,这个过程称为预处理过程,具体流程如下:

为了区分拓扑实体对象中的面、边、顶点和现实中零件真实的面、边、顶点,本文中称前者为模型面 (Face)、模型边 (Edge) 和模型顶点 (Vertex),称后者为工程面 (Engineering Face)、工程边 (Engineering Edge) 和工程顶点 (Engineering Vertex)。

由于 python 语法简单、代码简短,本文中直接使用 python 代码对一些算法和类的结构进行说明,其中#字符后面的文字为注释。

首先定义工程面、工程边、工程顶点类:

```
class EngFace:
    def __init__(self):
        self.face_list = []
        self.eng_edge_index_list = []
        self.type = GeomAbs_OtherSurface
```

其中, `self.face_list` 是一个用于存放组成工程面的模型面对象的列表;
`self.eng_edge_index_list` 是一个用于存放工程面上的工程边在 `self.eng_edge_list` 中的位置的列表; `self.type` 表示工程面的类型。

```
class EngEdge:
    def __init__(self):
        self.edge_list = []
        self.eng_vertex_list = []
        self.eng_face_index_list = []
```

```
self.type = GeomAbs_OtherCurve
```

其中, *self.edge_list* 是一个用于存放组成工程边的模型边对象的列表;
eng_vertex_list 是一个用于存放工程边上的工程顶点对象的列表;
self.eng_face_index_list 是一个用于存放工程边所连接的工程面在 *eng_face_list* 中的位置;
self.type 表示工程边的类型。

```
class EngVertex:  
    def __init__(self):  
        self.vertex = None  
        self.eng_edge_index_list = []
```

其中, *self.vertex* 表示工程顶点对应的模型顶点对象; *self.eng_edge_index_list* 是一个用于存放工程顶点所连接的工程边在 *eng_edge_list* 中的位置。

步骤一 读取 STEP 文件

用 DataExchange 包中封装好的 *read_step_file* 方法, 读取 STEP 模型文件, 将实体模型的全部信息封装为一个 TopoDS_Shape 对象, 其代码如下:

```
shape = DataExchange.read_step_file(file_name)
```

步骤二 建立虚顶点集合

创建列表 *virtual_vertex_list*, 用于存放虚顶点。利用 TopologyExplorer 类中的 *vertices* 方法遍历实体模型所有的模型顶点, 每访问到一个模型顶点时, 利用 TopoglyUtils 工具包中的 *edges_from_vertex* 方法得到该模型顶点所连接的若干个模型边 $e_0, e_1, \dots, e_n$ 。设 $e_0, e_1, \dots, e_n$ 在该模型顶点坐标处的单位切向量分别 $v_0, v_1, \dots, v_n$, 若同时满足:

- ①存在 $v_i \times v_j = 0$ ($0 \leq i, j \leq n$);
- ②对于①中的 i 和 j , e_i 和 e_j 的类型相同;

则该模型顶点为虚顶点, 将该 TopoDS_Vertex 对象到列表 *virtual_vertex_list* 中。

步骤三 建立虚边集合

创建列表 *virtual_edge_list*, 用于存放虚边。利用 TopologyExplorer 类中的 *edges* 方法遍历模型所有的模型边, 每访问到一个模型边时, 利用 TopologyUtils 工具包中的 *faces_from_edge* 方法得到该模型边所连接的两个模型面 f_0 和 f_1 。将取该模型边的中点 p_0 , 通过该坐标值构建两个分别属于该模型边所连接的两个相邻的模型面的两个点 p_1 和 p_2 。设 p_1 处的法向单位向量为 v_1 , p_2 处的法向单位向量为 v_2 , 若

同时满足：

① $v_l \times v_2 = 0$;

② f_0 和 f_l 的类型相同;

则该模型边为虚边，添加到列表 *virtual_edge_list*。

步骤四 建立工程顶点集合

创建列表 *eng_vertex_list*，用于存放工程顶点。利用 TopologyExplorer 类中的 *vertices* 方法遍历模型所有的模型顶点，每访问到一个模型顶点时，判断该顶点是否存在于 *virtual_vertex_list* 中，若不存在，则创建一个 EngVertex 对象，并将其 *vertex* 成员变量赋值为当前模型顶点的引用，然后添加该 EngVertex 对象到 *eng_vertex_list* 中。

步骤五 建立工程边集合

创建列表 *eng_edge_list*，用于存放工程边。遍历所有的模型边，每访问到一个未访问过的模型边时，判断该模型边是否存在于 *virtual_edge_list* 中，若存在则跳过该模型边，否则创建一个 EngEdge 对象，以该模型边为起点，对所有通过虚顶点可达的模型边进行广度/深度优先搜索^[63]，将搜索到的模型边加入 EngEdge 对象的 *edge_list* 中，然后将这个 EngEdge 对象添加到 *eng_edge_list* 中。利用 BrepAdaptor_Curve 类中的 *GetType* 方法得到这个 EngEdge 对象中每个模型边的类型，若全都为同一类型，则该工程边的类型也赋为这个类型，否则赋为“其他”类型。

步骤六 建立工程面集合

创建列表 *eng_face_list*，用于存放工程面。遍历所有的模型面，每访问到一个未访问过的模型面，以该模型面为起点，对所有通过虚边可达的模型面进行广度/深度优先搜索，将搜索到的模型面放入 EngFace 对象的 *face_list* 中，然后将这个 EngFace 对象添加到 *eng_face_list* 中。利用 BrepAdaptor_Surface 类中的 *GetType* 方法得到这个 EngFace 对象中每个模型面的类型，若全都为同一类型，则该工程面的类型也赋为这个类型，否则赋为“其他”类型。

步骤七 建立工程面、工程边、工程顶点之间的对应关系

遍历 *eng_face_list*，每访问到一个工程面，进一步遍历该工程面的 *face_list*，每访问到一个模型面，遍历该模型面中的模型边，每访问到一个模型边，遍历 *eng_edge_list*，得到该模型边所属的工程边，将该工程边在 *eng_edge_list* 中的位置索引添加到当前模型面对象中的 *eng_edge_index_list* 中，同时将该工程面在 *eng_face_list* 中的位置索引添加到当前工程边对象中的 *eng_face_index_list* 中。

遍历 *eng_edge_list*，每访问到一个工程边，进一步遍历该工程边的 *edge_list*，每访问到一个模型边，遍历该模型边中的模型顶点，每访问到一个模型顶点，遍历 *eng_vertex_list*，得到该模型顶点对应的工程顶点，将该工程顶点在 *eng_vertex_list* 中的位置索引添加到当前模型边对象中的 *eng_vertex_index_list* 中，同时将该工程边在 *eng_edge_list* 中的位置索引添加到当前工程顶点对象中的 *eng_edge_index_list* 中。

2.2.3 提取种子环

一组首尾相连的边构成环 (loop)，将特征与实体模型分割的界限称为种子环 (seed loop)，特征可以看作是从种子环生长出来的一组面和边。种子环的类型包括单面环和多面环，前者指种子环中的所有工程边在同一个工程面上，如图 2-5 中 a) 所示，后者指种子环中的工程边不只属于一个工程面，如图 2-5 中 b) 所示。本课题所实现的报价系统在特征识别方面只针对最为常见的单面环生长出的特征进行识别，种子环所属的工程面定义为基面。

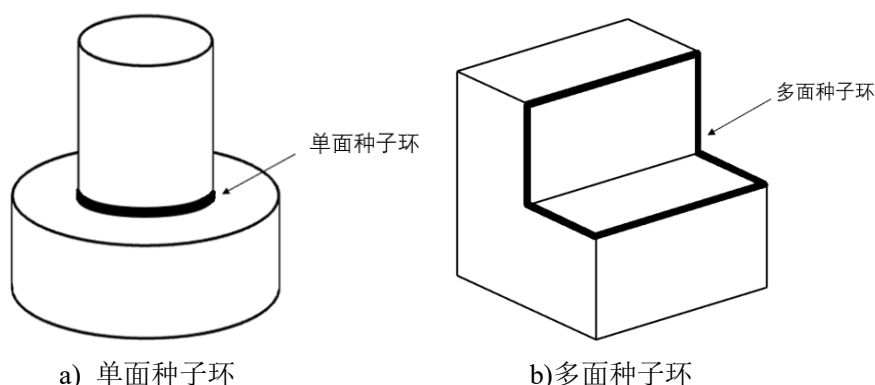


图 2-5 单面种子环和多面种子环

Fig.2-5 Seed loop based on single face and seed loop based on mutiple faces

经过预处理后，全部的工程面、工程边、工程顶点以及相互之间的映射关系已经得到，然后进行种子环的提取，其步骤如下：

首先定义环类：

```
class Loop:
    def __init__(self):
        self.eng_edge_index_list = []
        self.base_eng_face_index = -1
        self.side_eng_face_index_list = []
```

其中，*self.eng_edge_index_list* 是一个用于存放组成环的工程边在 *eng_edge_list*

中的位置的列表; $self.base_eng_face_index$ 表示环的基面在 eng_face_list 中的位置; $self.side_eng_face_index_list$ 是一个用于存放与环相邻的工程面在 eng_face_list 中的位置的列表。

创建列表 $seed_loop_list$, 用于存放种子环对象。遍历 eng_face_list , 每访问到一个工程面, 创建一个类型为列表的变量 $temp_loop_list$ 进一步遍历该工程面的所有工程边, 每访问到一个未访问过的工程边时, 创建一个 **Loop** 对象 l , 以该工程边为起点对所有通过工程顶点可达的同样属于当前工程面的工程边进行广度/深度优先搜索, 将搜索到的工程边添加到 l 的 eng_edge_list 中, 将 l 的 $base_eng_face_index$ 赋值为当前工程面在 eng_face_list 中的位置索引, 并将 l 添加到 $temp_loop_list$ 中。

判断该工程面的类型:

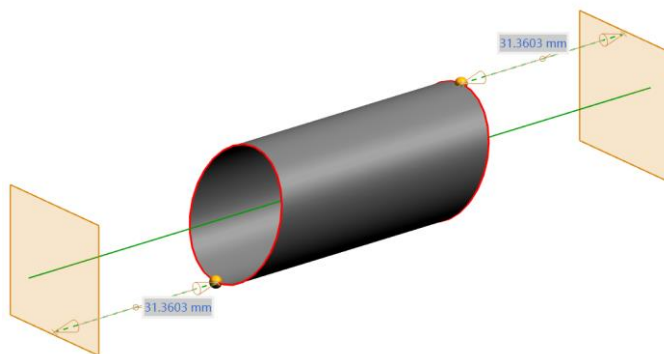


图 2-6 计算圆柱极值点示意图

Fig.2-6 Computing the extreme points on a cylinder face

若为圆柱面类型 (**GeomAbs_Cylinder**), 则计算该工程面在圆柱轴线方向上的两个极值点 P_1 和 P_2 , 具体方法如图 2-6 所示, 在圆柱轴线两个方向分别做一个垂直于轴线、边长大于圆柱直径且不与圆柱面接触的平面, 利用 **pythonOCC** 提供的计算两个实体最小距离的 **BRepExtrema_DistShapeShape** 类, 分别计算该圆柱面距离两个平面的最小距离的点, 即为两个方向的极值点。遍历 $temp_loop_list$, 每访问到一个 **Loop** 对象, 判断 P_1 或 P_2 是否在这个环上, 若 P_1 和 P_2 都不在该环上, 则该环为种子环, 添加到 $seed_loop_list$ 。

若为圆锥面类型 (**GeomAbs_Cone**), 则计算该工程面在圆锥轴线方向上的两个极值点 P_1 和 P_2 , 其中 P_1 为小端侧的点, P_2 为大端侧的点。遍历 $temp_loop_list$, 每访问到一个 **Loop** 对象, 判断 P_2 是否在该环上, 若 P_2 不在该环上, 则该环为种子环, 添加到 $seed_loop_list$ 。

若既不为圆柱面类型也不为圆锥面类型, 则利用 **ShapeFactory** 包中的

BRepBuilderAPI_MakeFace 类，以该环为边界、该工程面为几何形状，生成一个 TopoDS_Face 对象，计算并记录该面的面积。除生成的面中面积最大的外，其余的环均为种子环，添加到 *seed_loop_list*。

2.2.4 基本特征的识别

得到种子环之后，由种子环出发，进行基本特征识别与参数提取，基本特征类型是指特征的凹凸性和是否为圆柱类特征（孔、圆台），参数包括特征的高度/深度和截面积。首先定义特征类型枚举类（作为示例只定义了基本的 5 种）和特征类：

```
class FeatureType:
    OTHER = 1
    CIRCULAR_GROOVE = 2
    CIRCULAR_STEP = 3
    NON_CIRCULAR_GROOVE = 4
    NON_CIRCULAR_STEP = 5
```

其中，OTHER 表示未知的其他类型；CIRCULAR_GROOVE 表示圆孔/槽类型；CIRCULAR_STEP 表示圆台类型；NON_CIRCULAR_GROOVE 表示非圆形孔/槽；NON_CIRCULAR_STEP 表示非圆形的凸台。

```
class Feature:
    def __init__(self):
        self.type = FeatureType.OTHER
        self.para_list = []
        self.eng_face_index_list = []
        self.start_loop_index = -1
        self.end_loop_index = -1
        self.has_thread = False
```

其中，*type* 表示特征的类型，默认为其他类型；*self.para_list* 为存放特征的参数的列表；*self.eng_face_index_list* 是一个用于存放特征中所包含的工程面在 *eng_face_list* 中位置的列表；*self.start_loop_index* 表示特征起始位置的种子环在 *loop_list* 中的位置；*self.end_loop_index* 表示特征终止位置的种子环在 *loop_list* 中的位置；*self.has_thread* 表示特征是否包含螺纹。

创建列表 *feature_list*，遍历 *seed_loop_list*，每访问到一个未访问过的种子环，从该种子环中任取一工程边，在该工程边内的 *eng_face_index_list* 中任取一个不是

种子环基面的工程面，作为起点，对所有通过不属于某种子环的工程边可达的工程面进行深度/广度优先搜索，创建一个 Feature 对象 f ，将搜索到的工程面在 eng_face_list 中的位置索引添加到 f 的 $eng_face_index_list$ 中，并将 f 添加到 $feature_list$ 中。

接下来提取特征的基本属性与参数，包括特征的凹凸性、高度/深度和截面积。

特征的凹凸性：

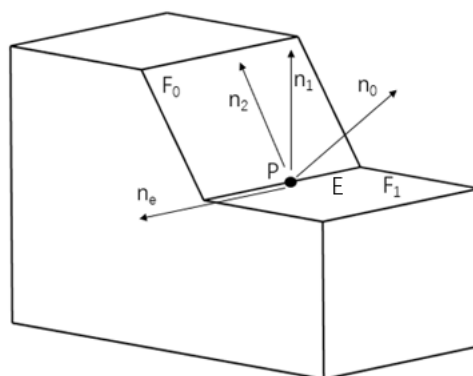


图 2-7 凹边判定

Fig.2-7 Judgement of concave edge

特征的凹凸性取决于种子环的凹凸性，即种子环中模型边的凹凸性。其中凹边是指，在该边的任意一点用垂直于该边的平面去截边所在的实体，在获得的截面上实体所占的角度大于 180° ，则该边为实体的一条凹边，反之为凸边。如图 2-7 所示，设进行判定的模型边为 E ， E 所连接的两个模型面分别记为 F_0 和 F_1 。取模型边中点 P ， P 在 E 上的单位切向量记为 n_e （依据右手螺旋定则，即右手四指方向为 F_1 转向 F_0 ，此时大拇指方向为 n_e 的方向）， P 在 F_0 上的单位法向量记为 n_0 ， P 在 F_1 上的单位法向量记为 n_1 。令 $n_2 = n_e \times n_0$ ，记 n_1 与 n_2 的夹角为 θ ，若 $0^\circ < \theta < 90^\circ$ ，则该模型边为凹边。若种子环中任一工程边中的任一模型边为凹边，则该特征为凸特征，否则该特征为凹特征。

特征是否为圆柱特征：

通过特征中的 $start_loop_index$ 获取该特征的种子环，遍历该 Loop 对象的 $side_eng_face_index_list$ ，若对应的所有工程面类型均为圆柱面并且同轴，则该特征为圆柱特征，否则为非圆柱特征。

特征的截面积和高度/深度：

遍历当前特征的 $eng_face_index_list$ ，每访问到一个工程面，计算其与当前特

征的种子环的距离，其中最大的距离值作为特征的深度/高度。

包围盒是指完全包含一个物体的（最小的）长方体，主要用于碰撞检测。本课题所设计的报价系统中对特征截面积和高度/深度的提取通过构造特征的包围盒实现，包围盒主要分为 AABB（Aligned Axis Bounding Box）包围盒和 OBB（Oriented Bounding Box）包围盒。前者是指包含物体并且长方体的边均平行于三个坐标轴的最小长方体，后者是指相对于坐标系任意方向中包含物体的最小长方体。相较于 AABB，OBB 计算更加复杂，但对物体的边界描述更为精确。

OCC 在 7.3.0 版本之前只提供创建 AABB 包围盒的功能，从 2018 年发布的 7.3.0 版本开始增加 OOB 包围盒类。

利用 pythonOCC 提供的包围盒类，将一个特征中的所有模型面都添加到包围盒的包围范围内，然后根据包围盒的顶点坐标计算其三个边长，其中最接近已经得到的特征深度/高度的边长记为 H ，其余两两边长记为 A 和 B ，则以矩形的内接椭圆面积近似表示特征的截面积，即特征截面积 $S = \frac{\pi AB}{4}$ 。

至此已将识别出的特征进行了一些基本属性参数的提取，包括特征的凹凸性、是否为圆柱特征、特征高度/深度和特征的截面积。

2.2.5 指定特征的识别

在工程实际中，仅仅知道特征的基本属性是远远不够的，特征的具体类型成百上千，不同类型的特征实现不同的功能，有着不一样的加工方式，对零件价格的影响规律也不相同，因此我们需要对不同类型的特征进行识别并提取它们各自独有的参数。

本课题实现的报价系统提供了对指定的特征类型进行识别的功能，采用基于图的特征识别方法，具体而言，是将特征库中的特征以属性邻接图的形式存储，在提取出实体模型上的特征后，转换为属性邻接图的形式，然后和特征库中的属性邻接图进行一一匹配，以确定该特征的类型。数据结构的形式和特征识别的具体流程如下。

特征的属性邻接图用两个数据结构表示：一是存放了所有工程面的一维数组，在 python 中对应一个列表，该列表中的元素为工程面在 *eng_face_list* 中的位置索引，该列表包含一个特征中的所有工程面和特征中 *start_loop* 和 *end_loop* 分别对应的基面（若存在）。二是标识这些工程面之间连接关系的邻接矩阵，是一个二维数组，在 python 中对应一个元素为列表的列表，其中粒度最小的元素为该工程面与其他工程面之间的连接关系，即是否相邻和连接它们的边的凹凸性、类型等属

性。

如图 2-8，以退刀槽特征为例进行说明。属性邻接图右侧的 3×2 表格表示存放所有工程面的类型要求的一维数组， 4×4 的表格是表示工程面之间连接关系二维数组（邻接矩阵）。属性邻接图中的结点用圆形表示，结点的属性用字母标识，具体含义如表 2-3 所示。属性邻接图中的弧用线段表示，弧的属性用两位数字标识，具体含义如表 2-4 和表 2-5 所示。其中，若某个结点和弧的属性可以为多种类型，则在后面继续添加字母或数字，例如一条边是凹边，类型既可以是直线也可以是圆弧，则其属性记为 212。此外，属性邻接图中深色结点代表特征对应的种子环的基面，浅色结点代表特征对象中的工程面。

表 2-3 属性邻接图中结点属性的含义

Table 2-3 Meaning of vertex's attribute in adjacency graph

A	B	C	D	E	F	G
任意类型	平面类型	圆柱面类型	圆锥面类型	圆环面类型	球面类型	其他类型

表 2-4 属性邻接图中弧的属性中第一位数字的含义

Table 2-4 Meaning of first number in edge's attribute in adjacency graph

1	2	3
任意凹凸性	凹边	凸边

表 2-5 属性邻接图中弧的属性中第二位数字的含义

Table 2-5 Meaning of second number in edge's attribute in attribute adjacency graph

0	1	2	3
任意类型	直线类型	圆弧类型	其他类型

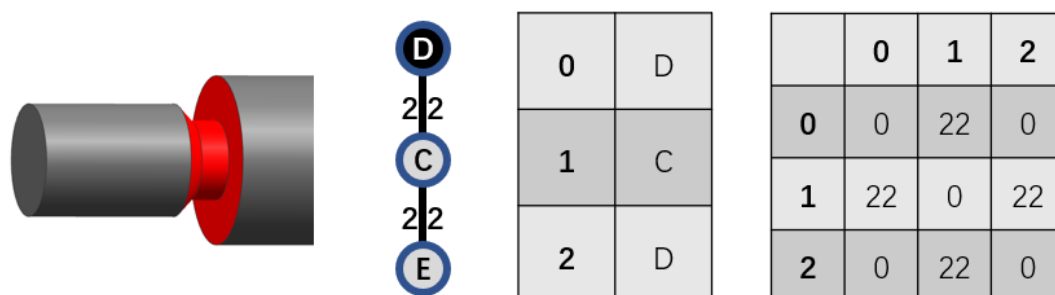


图 2-8 退刀槽特征及其属性邻接图表示

Fig.2-8 Relief groove feature and its attribute adjacency graph

其中属性邻接图右侧的 3×2 表格表示的是记录面的自身属性的一维数组（列表），规定基面必须为数组首元素。另外，由于退刀槽特征可能以圆锥面为基面被识别，也可能以圆环面为基面被识别，因此在特征库中实际要存储两个退刀槽特征，以保证能够准确匹配。

在属性邻接图构造好后，需要将该属性邻接图与特征库中的属性邻接图一一

匹配，图匹配也称为图同构，是指两个图的拓扑结构完全相同。图同构最原始的算法就是写出两个图的邻接矩阵，然后对其中一个邻接矩阵进行交换和列交换（交换 i, j 行的同时也要交换 i, j 列），若能使得两个矩阵相同，则两个图同构。显然这是一个排列组合的问题，最坏情况下需要进行 $n!$ 次的交换（ n 为图中结点的数量）。随着图中结点数量的增加，算法的效率急剧降低，由于其时间复杂度为非多项式，对大规模的图即面的数量非常多的特征而言，是不可行的。

研究人员提出了许多新的改进算法，其中基于图搜索的算法较为成功，主要包括 Ullmann 算法^[64]、SD 算法^[65]、Nauty 算法^[66]和 VF 算法^[67]，经测试 VF2^[68]算法（VF 算法的改进）适用于图较小且边较稀疏的情况，符合特征面边图的实际情况，故本课题采用 VF2 算法进行属性邻接图的匹配。

当一个特征的具体类型确定时，其参数往往可以由截面积、高度/深度、某类边的长度或某类面的面积等通用属性计算。特征库的添加采取如下方式：用户在添加一个新特征到特征库时，需要准备两个文件：一个是 B-rep 模型文件，另一个是脚本文件。

B-rep 模型文件应该只包含要添加的特征这一个特征，脚本文件的格式如下（以#开头的语句为注释）：

```

RULE START
# start_loop 对应的基准面的类型作为判定标准
start_base_face=true
# end_loop 对应的基准面的类型作为判定标准
end_base_face=true
RULE END
# 提取的参数名称及表达式
PARAMETER START
# 名为D1的参数等于任意圆锥面的大径
D1=Cone.OuterDiameter
# 名为D2的参数等于任意圆锥面的小径
D2=Cone.InnerDiameter
# 名为D3的参数等于任意圆环面的外径
D3=Torus.OuterDiameter
# 名为L1的参数等于任意圆锥面的轴向高度
L1=Cone.Height
# 名为L2的参数等于任意圆柱面的轴向高度

```

```

L2=Cylinder.Height
PARAMETER END
# 系统用于校验参数提取是否准确，若不准确，系统进行提示
CHECK START
D1==55
D2==35
D3==100
L1==10
L2==10
CHECK END

```

系统的处理流程如下：

对用户上传的特征模型，进行特征识别，提取出特征及其基本属性（截面积、高度/深度等），并将特征中的面边关系（包含基面）表示为属性邻接图，解析用户上传的文本，以确定基面类型是否作为属性邻接图中的判定标准，然后将属性邻接图持久化存储。进一步解析文本文件，以文件中所描述的规则进行参数的提取，并与校验结果进行比对，若结果不正确则弹出提示，否则进行持久化存储。需要注意的是，本系统目前支持的特征可能指包含一个 *start_loop*，也可能既包含一个 *start_loop*，又包含一个 *end_loop*。对于用户上传的模型中，特征既存在 *start_loop* 又存在 *end_loop* 的情况，需要分别以这两个 *loop* 作为种子环进行特征的表示与存储。

在自定义特征的添加中，新添加的特征的识别是能够保证的，但由于文本描述能力有限，只有少部分容易表达的参数提取规则可以以脚本的形式被系统理解并添加到特征库中，这也是系统需要进行改进的地方之一。

2.3 螺纹特征的识别

螺纹作为机械领域实际生产中最为常见的结构之一，由于结构复杂，其模型生成与计算会占用大量的系统资源，表现为 CAD 软件操作卡顿，甚至崩溃。在过去，技术人员在建模时往往不会创建螺纹特征，而是退而求其次地辅以文本信息记录模型中螺纹的参数。如今随着计算机性能的快速提升，越来越多的技术人员选择在建模时直接将螺纹特征的三维造型创建出来。并且由于螺纹特征中的边和面不是基本的几何曲线、曲面，图匹配的特征识别方法也难以用于螺纹特征的识别，为此本课题设计并在报价系统中实现了一种螺纹特征识别方法。

2.3.1 螺纹特征的识别流程

本系统的螺纹识别是以基本特征为对象，需要先进行基本特征的识别。然后在基本特征内进行螺纹线的识别，进而通过螺纹线信息求解出螺纹方程，得到螺纹的参数。由于螺纹线参数方程不相同，本系统目前仅支持圆柱螺纹的识别，暂不支持圆锥螺纹的识别。

步骤一 遍历 *feature_list*，每访问到一个特征，进一步遍历特征中的 *eng_face_index_list*，每访问到一个工程面索引，从 *eng_face_list* 中得到该工程面对象，判断该工程面类型是否为圆柱面，若为圆柱面，则遍历当前工程面的 *eng_edge_list*，每访问到一个未访问过的工程边，判断该工程边是否为螺纹线。

步骤二 首先判断该工程边的类型是否为直线或圆弧，若是则该工程边不可能是一条螺纹线，否则继续进行后续步骤。

步骤三 判断该工程边是否是一条平面曲线：在工程边上随机取 n 个点 ($n \geq 7$ 且 n 为素数) 组成集合 $P = \{p_0, \dots, p_{n-1}\}$ 。设该工程边在每个点处的法向量组成集合 $U = \{u_0, \dots, u_{n-1}\}$ 。同时设 p_0 指向 p_1 , p_0 指向 p_2 , ..., p_0 指向 p_{n-1} 的共 $n-1$ 个向量组成集合 $V = \{v_0, \dots, v_{n-1}\}$ 。 $\forall u_i \in U, \forall u_j \in U, \forall u_k \in U, \forall v_a \in V, \forall v_b \in V$, 当同时满足：

- ① $u_i \times u_j = u_k \times u_l$ 或 $u_i \times u_j = 0$ 或 $u_k \times u_l = 0$;
- ② $u_i \times u_j = u_k \times v_a$ 或 $u_i \times u_j = 0$ 或 $u_k \times v_a = 0$;
- ③ $u_i \times u_j = v_a \times v_b$ 或 $u_i \times u_j = 0$ 或 $v_a \times v_b = 0$;

则认为该工程边是一条平面 2D 曲线，不可能是一条螺纹线。否则继续执行后续步骤。

步骤四 判断该工程边是不是一条螺纹线：在工程边上从起始端点等距离地依次取 n 个点 ($n \geq 7$ 且 n 为素数，具体取值由所能接受的算法运行时间自行确定) 组成集合 $P = \{p_0, \dots, p_{n-1}\}$ 。设该工程边在每个点处的切向单位向量组成集合 $U = \{u_0, \dots, u_{n-1}\}$ 。设 p_0 指向 p_1 , p_0 指向 p_2 , ..., p_0 指向 p_{n-1} 的共 $n-1$ 个向量组成集合 $V = \{v_0, \dots, v_{n-1}\}$ 。 $\forall u_i \in U, \forall u_j \in U, \forall v_k \in V, \forall v_l \in V$, 当同时满足：

- ① $u_i \cdot u_{i+1} = u_j \cdot u_{j+1}$;
- ② $|v_k| = |v_l|$;
- ③ $v_k \cdot v_{k+1} = v_l \cdot v_{l+1}$;

则记录结果为真。对步骤四中的上述流程执行 m 次 ($m \geq 3$)，当 m 次结果都为真，则判断该工程边是一条螺纹线；当所有的工程边都不是螺纹线，则认为该特征不是螺纹特征，否则认为该特征是螺纹特征。

2.3.2 螺纹特征的参数及提取流程

步骤一 确定三维空间螺纹线参数方程

螺纹线的参数方程通常表示为:
$$\begin{cases} x = a \cdot \cos(t) \\ y = b \cdot \sin(t), \quad b = \pm a, \text{ 即以 } Z \text{ 轴为中心} \\ z = c \cdot t \end{cases}$$

轴, 过 $(a, 0, 0)$ 点的螺纹线。

为了利用上述形式的参数方程, 要建立一个局部坐标系 $C(O_c X_c Y_c Z_c)$ 并依次确定 Z_c 轴、原点 O_c 和 X_c 轴: 根据实际加工过程, 螺纹线一定具有相邻的圆柱面, 以该圆柱面的轴线方向作为 Z_c 轴, 并在螺纹线上任取一点 P , 如图 2-9 所示。 P 到圆柱面轴线的垂足为 Q , 以 Q 为坐标原点 O_c , Q 到 P 的射线为 X_c 轴, 由右手螺旋定则求得 Y_c 轴, 至此整个局部坐标系 $C(O_c X_c Y_c Z_c)$ 完全确定。

在螺纹线上任取一点 P , 设 P 在坐标系 C 中的坐标为 (x_p, y_p, z_p) 。

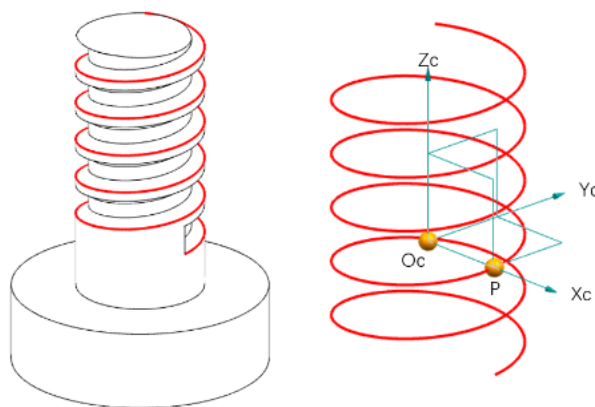


图 2-9 螺纹线提取
Fig.2-9 Thread extraction

a 和 b 的大小求解: $|a| = |b| = \sqrt{x_p^2 + y_p^2}$;

a 和 b 的正负关系求解: 记 Z_c 轴的方向向量为 u , 记向量 $v = (x_p, y_p, 0)$, 记 p 点切向向量为 w , 令 $f = u \times v$, 求得 u 与 w 夹角为 α , f 与 w 的夹角为 β 。若 α 与 β 同时为锐角或同时为钝角, 则 $a=b$, 否则 $a=-b$;

c 的求解: $c = |a| \cos \alpha$ 。

步骤二 提取螺纹参数

在访问到当前特征时, 创建一个列表类型的变量 `thread_curve_list`, 用于存放当前特征中的所有螺纹线。如图 2-10 所示, 螺纹特征主要的独立参数包括大径 D , 小径 d , 线数 n , 螺距 P 和旋向五个, 提取过程具体为:

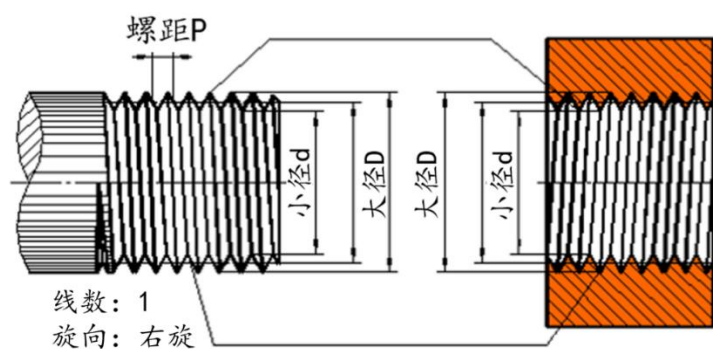


图 2-10 螺纹参数

Fig.2-10 Thread parameter

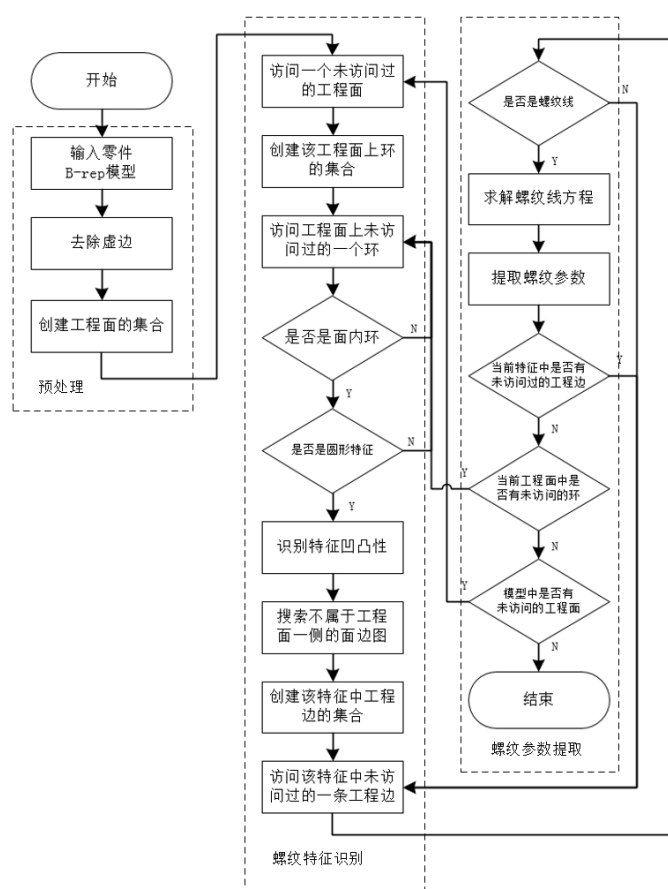


图 2-11 螺纹识别流程图

Fig.2-11 Flow chart of thread recognition

i) 大径 D 和小径 d : 对于一个螺纹线, 其参数方程中的 a 的 2 倍即代表了这这个螺纹线的直径。对于一个螺纹特征而言, 存在两种直径的螺纹线, 分别对应螺纹的大径和小径。遍历 *thread_curve_list*, 每访问到一条螺纹线, 求解其参数方程, 得到直径。直径的取值有两个, 较大的为该螺纹特征的大径 D , 较小的为该螺纹

特征的小径 d 。

ii) 线数 n : 一个线数对应两条大径螺纹线和两条小径螺纹线。因此线数 n 等于 $thread_curve_list$ 中的元素数量除以 4, 即 $n = \frac{\text{len}(thread_curve_list)}{4}$ 。

iii) 导程 S : 导程是指一条螺纹线上相邻两对应点的轴向距离。一个螺纹特征中所有螺纹线的导程都相等。取 $thread_curve_list$ 中的第一个螺纹线, 求解其参数方程, 导程 $S = 2\pi bc$ 。

iv) 螺距 P : 螺距是指相邻两个螺牙在中经线上对应两点的轴向距离。螺距等于导程除以线数, 即 $P = \frac{S}{n}$ 。

v) 旋向: 取 $thread_curve_list$ 中的第一个螺纹线, 求解其参数方程, 若 $ab > 0$, 则该螺纹特征为左旋螺纹, 否则为右旋螺纹。

螺纹特征的识别可以作为一个独立功能提供给用户, 其整体流程如图 2-11 所示。

2.4 特征树的结构设计与构建

实体模型中的特征不是无序的组织在一起的, 它们之间存在许多的依赖关系, 有着非常鲜明的层次结构, 因此用树结构来组织这些特征更有利于表达特征之间的依赖关系, 帮助进行零件成本的分析, 实现更为准确的报价。

2.4.1 特征树的结构

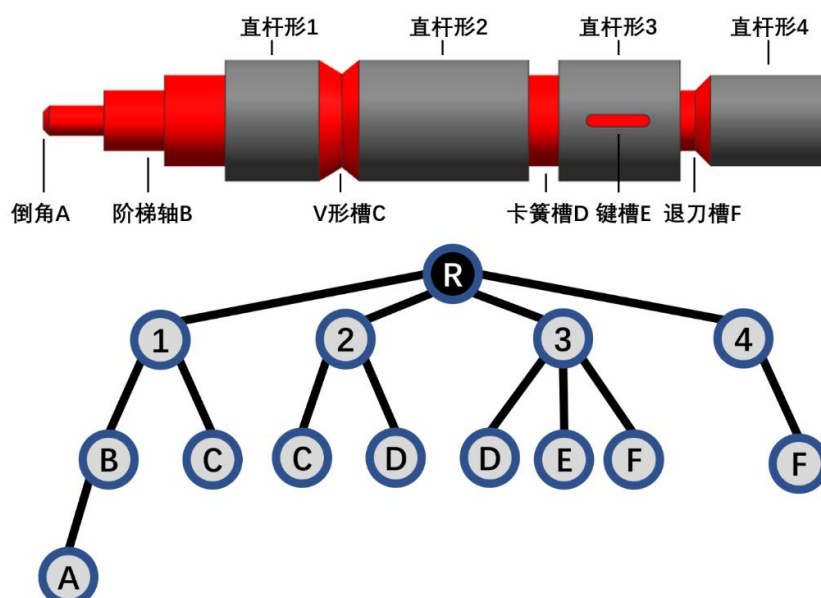


图 2-12 轴类零件与对应的特征树
Fig.2-12 A shaft part and its feature tree

特征树定义如下：根节点代表整个实体模型，是一个虚拟的结点，不包含任何具体的面和边。非根结点的孩子结点表示从该结点表示特征中的某个工程面上生长出的一个特征，或者说该孩子结点表示的特征的 *start_loop* 的基面属于父结点表示的特征。根结点的孩子结点表示实体模型中非特征的部分，或者说是主体/基体（Base Body）部分。

图 2-12 是一个轴类零件与其对应的特征树结构。

其中 *R* 表示虚拟的根结点，可以看出 V 形槽、卡簧槽和退刀槽等特征，由于既存在 *start_loop*，也存在 *end_loop*，故生长于两个基体或特征之上，树中出现了重复的结点，为了避免这种情况的影响，在结点中引入权重系数这一属性，采用均分权重的方式，将特征进行拆解。这是考虑到例如图 2-12 中的零件，可以看成在 V 形槽处断开的两个零件合并在一起，断开的零件各自拥有一个倒角/锥形特征。

2.4.2 特征树的构建

首先定义基体类型枚举类、基体类和特征树的结点类：

```
class BaseBodyType:
    OTHER = 1
    CYLINDER = 2
```

其中，OTHER 表示未知的其他类型或不确定类型；CYLINDER 表示基体为直杆形。

```
class BaseBody:
    def __init__(self):
        self.eng_face_index_list = []
        self.type = BaseBodyType.OTHER
```

其中，*self.eng_face_index_list* 是一个用于存放基体中包含的工程面在 *eng_face_list* 中位置的列表；*self.type* 表示基体的类型，默认为其他类型。

```
class FeatureTreeNode:
    def __init__(self):
        self.is_feature = False
        self.index = -1
        self.child_list = []
        self.level = 1
```

self.weight = 1

其中, *self.is_feature* 表示该结点是特征还是基体; *self.index* 表示该结点代表的特征或基体在 *feature_list* 或 *base_body_list* 中的位置; *self.child_list* 是一个用于存放孩子结点的列表; *self.level* 代表结点在特征树中的层数; *self.weight* 代表该结点的权重。

首先创建一个 *FeatureTreeNode* 对象作为根结点, 以 *index* 值为-1 作为根结点的标识。

根结点的孩子结点为基体, 需要在实体模型中进行提取, 流程如下:

创建列表 *base_body_list*, 用于存放基体对象。

若 *feature_list* 为空, 即该实体模型不存在特征, 则整个实体模型为一个基体, 创建一个 *BaseBody* 对象, 将所有的工程面添加到该对象的 *eng_face_index_list*, 将这个 *BaseBody* 对象添加到 *base_body_list*。

若 *feature_list* 不为空, 则遍历 *seed_loop_list*, 每访问到一个未访问过的种子环, 创建一个 *BaseBody* 对象, 则以该种子环中任一工程边的属于其基面的相邻工程面为起点, 对所有通过不属于某种子环的工程边可达的工程面进行深度/广度优先搜索, 将搜索到的工程面添加到这个 *BaseBody* 对象的 *eng_face_index_list* 中, 将这个 *BaseBody* 对象添加到 *base_body_list*。

遍历 *base_body_list*, 每访问到一个 *BaseBody* 对象, 创建一个 *FeatureTreeNode* 对象, 将其 *index* 赋为该 *BaseBody* 对象在 *base_body_list* 中的位置索引, *level* 赋为 2, 并将该对象添加到根结点的 *child_list* 中。

然后遍历根结点的 *child_list*, 每访问到一个 *FeatureTreeNode* 对象, 根据其 *is_feature* 和 *index* 的值从 *feature_list* 或 *base_body_list* 中得到特征对象或基体对象记为 *cur_obj*, 然后遍历 *feature_list*, 每访问到一个未访问过的特征, 判断该特征的 *start_loop* 对应的基面索引是否属于 *cur_obj* 的 *eng_face_index_list*, 若是则创建一个新的 *FeatureTreeNode* 对象, 其 *is_feature* 赋为 *True*, *index* 赋为当前特征在 *feature_list* 中的位置索引, *level* 赋为 *level+1*。然后以新的 *FeatureTreeNode* 对象为当前结点, 递归地创建整个特征树。在创建的过程中, 使用一个字典记录每个特征被结点“使用”了几次, 然后对特征树遍历一遍, 对每个结点赋值对应的权重。

2.5 本章小结

本章详细介绍了本课题如何利用 *pythonOCC* 对 *B-rep* 模型进行特征识别, 以单面内环作为种子环, 实现了种子环的提取, 并以种子环出发实现了特征的提取,

同时利用包围盒等方法实现了基本特征参数（如高度/深度、截面积）的提取。并且提出了一种从螺纹线出发识别螺纹特征并提取大小径、螺距和旋向等参数。除此之外，利用 VF2 算法进行属性邻接图的匹配，同时创建了一个简单的脚本语言规则，使得用户可以方便地添加自定义特征，系统可以在不修改源代码的基础上自动地去识别该特征并提取其特定的参数。

最后，根据特征彼此之间的依赖生长关系，设计了特征树的数据结构，以清晰完整地表达整个实体特征模型，从而使得后续的加工过程推理和相似性检索更为准确合理。

由于本课题的研究重点在通用报价系统的整体设计，而不是各个模块具体的先进算法设计，故本章所实现的特征识别和特征树结构都是以实际中简单常见的特征为基础设计的，即：

（1）特征的种子环都是单面环，工程实际中大多数特征都是生长在一个面上，系统可以准确的对其进行识别，但系统无法识别多面环生长出的特征。

（2）具体类型的特征识别采用图匹配的方法，因此无法避免图匹配存在的问题，即无法识别相交特征。

（3）系统以树结构对实体模型上特征的层次结构进行描述，由于树结构不存在环的特点，故这种特征树的表示形式不适合存在环形依赖关系的特征，例如图 2-13 所示的拓扑结构。

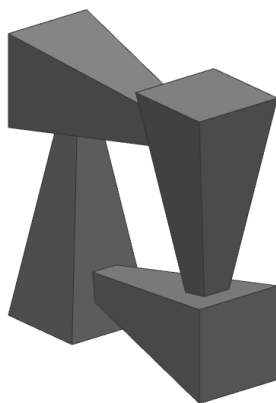


图 2-13 环形依赖的拓扑结构
Fig.2-13 Ring-dependent topology

（4）本章通过属性邻接图匹配结合定义的脚本语言，使用户可以方便地添加指定特征，而不需要进行代码层面的改动。但是由于脚本描述能力的局限性，只有部分简单特征的参数提取可以根据脚本定义规则。

第三章 标准件的报价

3.1 引言

随着制造业的不断发展，企业之间的合作越来越频繁，许多基础功能零件都走向了标准化，标准件的报价成为了零件报价中越来越重要的一部分。由于标准件是市场上已有的一组有着固定参数类型和规格的零件，其价格可以按照规格从价格表中查询得到。针对标准件的报价，系统要解决的主要问题就是如何根据 B-rep 模型识别出零件属于哪类标准件，如何提取该类别对应的规格参数，以及如何实现标准件数据库的可扩展性，即如何在不修改程序代码的前提下使用户能够简单方便地添加新的标准件类别。

本课题中提到的标准件具体是指系统中已存在其详细规格-价格信息的拓扑结构固定的一类零件。这类零件的报价实质上就是要解决如何对实体模型进行检索。对拓扑结构的相似性检索而言，基于图匹配的检索准确度最高，但是图匹配本身是一个 NP 完全问题，零件的面边数量又远远大于局部特征中的面边数量，其时间复杂度难以控制，因此需要设计复杂度可控的检索算法。为此，本系统设计并实现了一种自适应的标准件 B-rep 模型检索算法，利用 B-rep 特有的几何与拓扑信息以及标准件本身的特点，将 B-rep 模型本身蕴含的几何拓扑信息按照一定的方法进行编码。在保证了解析准确性和高效性的同时，能够自动生成新添加的标准件类别的检索规则，有效地解决了标准件报价中存在的问题。

3.2 标准件的分类

3.2.1 分类规则与编码表示

标准件的分类就是要解决一个 B-rep 模型属于哪一类标准件类别的问题。同一类标准件往往具有相同拓扑结构，但不同的规格有着不同的几何尺寸，利用这一特点，本课题提出以基本拓扑属性的组合作为一个标准件类别的判定标准，并以规则编码和结果编码的形式表示。

在本章中，以“编码规则”这一术语表示这一组用于判定标准件类别的拓扑信息；以“规则编码”这一术语表示以编码的形式标识具体使用编码规则中哪些规则，作为某个标准类别的判定规则；以“结果编码”这一术语表示以编码的形式标识该标准件类别的规则编码表示的判定规则所对应的结果。

编码规则是指通过一组 B-rep 中包括顶点、边、面在内的拓扑相关的规则组合，其作为区分标准件类别的依据，以规则编码的形式表示，本课题将编码长度

定为 32，其中前 16 位的含义为：

- 第 1 位——模型是否为实体模型；
- 第 2 位——模型中面的数量；
- 第 3 位——模型中边的数量；
- 第 4 位——模型中顶点的数量；
- 第 5 位——模型中是否存在平面；
- 第 6 位——模型中是否存在圆柱面；
- 第 7 位——模型中是否存在圆锥面；
- 第 8 位——模型中是否存在球面；
- 第 9 位——模型中是否存在直线边；
- 第 10 位——模型中是否存在圆弧边；
- 第 11 位——模型中平面的数量；
- 第 12 位——模型中圆柱面的数量；
- 第 13 位——模型中圆锥面的数量；
- 第 14 位——模型中球面的数量；
- 第 15 位——模型中直线边的数量；
- 第 16 位——模型中圆弧边的数量。

后 16 位为用于存放复杂拓扑规则的保留位。复杂拓扑规则是指如是否存在相连的圆弧和直线、是否存在相邻的平面和圆柱面等规则。

3.2.2 编码的数据结构

规则编码中的每一位表示是否使用当前位对应的编码规则作为该标准件类别的判断依据，因此是一个二值的布尔类元素。在 python 的设计中，为了实现原生的高精度运算等功能，其基本数据类型对应 c 语言中的复杂结构体。例如在 64 位环境下的 python3 中，整型变量和布尔变量对应如下的 c 语言中的结构体：

```
struct PyLongObject
{
    long ob_refcnt;
    struct _typeobject *ob_type;
    long ob_size;
    unsigned int ob_digit[1];
};
```

该结构体占 28 个字节的内存，因此在使用 python 语言编写的程序中如何节省

内存空间是必须考虑的问题。

32 位的规则编码最简单的表示方式是用一个长度为 32 的元素为布尔类型的列表表示。为了节省内存空间，二值的数组或列表可以用位图这种数据结构实现，位图是利用二进制表示，用每个 bit 位标识一个布尔变量，bit 位为 0 表示 False，bit 位为 1 表示 True。从语言层面来讲，位图结构就是一个整型变量或整型变量的数组/列表，通过读取和修改整型变量的值来得到或改变其在内存中指定位置的 bit 位的值。

python 中的 int 类型是 32 位有符号整型变量，数值部分只占 31 个 bit 位，因此需要用一个 32 位无符号整型变量作为位图的实现。NumPy^[69]是一个功能强大的 python 库，主要用于数值计算，它提供了一个 32 位无符号整型的实现类 uint32，本系统使用这一无符号整型变量作为位图结构的实现。基于面向对象的开发思想，将规则编码定义为 rule_code 类，内部封装一个 32 位无符号整型变量，以位图的数据结构存储规则的编码，并定义修改和获取指定规则的采用情况。由于 python 是动态类型的语言，类的实例通过一个名为 __dict__ 的字典存储字段名称及对应的值，字典本质是一个哈希表，存在空间的浪费，为了进一步节省内存空间，使用 __slots__ 去除对象的动态特性，具体实现如下：

```
class rule_code:
    __slots__ = ('bitmap')
    def __init__(self, val = 0):
        self.bitmap = numpy.uint32(val)
    def set_bit(self, position: int, val: bool):
        ori_val = self.get_bit(position)
        if ori_val < 0:
            return False
        pos = 32 - position
        if val is True and ori_val is 0:
            self.bitmap += 2 ** pos
        elif val is False and ori_val is 1:
            self.bitmap -= 2 ** pos
        return True
    def get_bit(self, position: int):
        if position < 1 or position > 32:
            return -1
```

```
pos = 32 - position
return (int)(self.bitmap // (2 ** pos) % 2)
```

在 `rule_code` 类中，以名为 `bitmap` 的 32 位无符号整型作为位图，对外提供 `get_bit` 和 `set_bit` 两个方法，分别用于获取和设置某位编码的值。

与规则编码不同的是，结果编码中不只包含了二值的布尔元素，还包含了诸如面的数量、边的数量等规则对应的多值结果，无法仅仅用一个 bit 位表示。为了节省内存空间，设计如下的数据结构以表示结果编码。将结果编码分为两个部分：一部分用来标识布尔类型的结果，考虑到已确定的 16 个规则中仅有 7 个规则为布尔型，因此用 NumPy 包中的 8 位无符号整型（`uint8`）作为位图结构来进行表示；另一部分用来标识整数类型的结果，由于整型变量占内存空间较大，并且其中只有不确定数量的一部分规则会被用作判据，并且考虑到规则的匹配都是按照规则编码的顺序从前向后进行检验的，因此采用链表的形式进行存储。

以图 3-1 所示的螺钉零件为例，给出一个具体的编码示例，其分类规则如图 3-3 所示，在内存中的数据结构如图 3-2 所示。

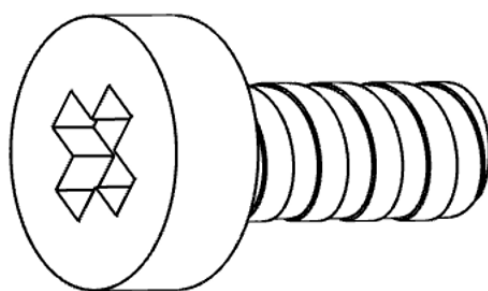


图 3-1 螺钉模型
Fig.3-1 Screw model

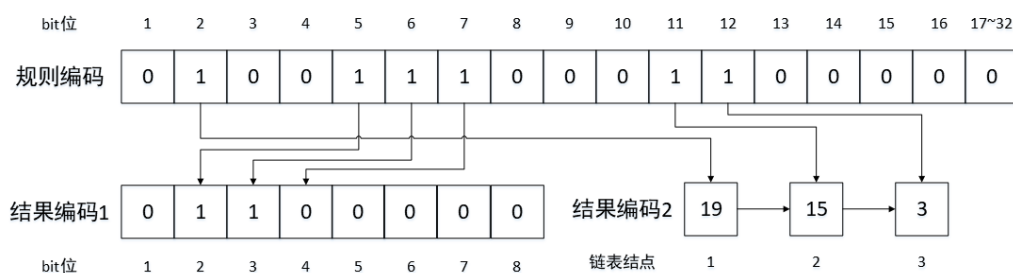


图 3-2 螺钉编码的数据结构
Fig.3-2 Data structure of screw classification code

规则编号	规则编码	结果编码	判别规则
1	0	/	是否采用 模型是否为实体 作为判别之一：否
2	1	19	是否采用 面的数量 作为判别之一：是；判别：该类别模型中面的数量为19
3	0	/	是否采用 边的数量 作为判别之一：否
4	0	/	是否采用 顶点的数量 作为判别之一：否
5	1	True	是否采用 平面的存在与否 作为判别之一：是；判别：该类别模型中存在平面
6	1	True	是否采用 圆柱面的存在与否 作为判别之一：是；判别：该类别模型中存在圆柱面
7	1	False	是否采用 圆锥面的存在与否 作为判别之一：是；判别：该类别中不存在圆锥面
8	0	/	是否采用 球面的存在与否 作为判别之一：否
9	0	/	是否采用 直线边的存在与否 作为判别之一：否
10	0	/	是否采用 圆弧边的存在与否 作为判别之一：否
11	1	15	是否采用 平面的数量 作为判别之一：是；判别：该类别模型中平面数量为15
12	1	3	是否采用 圆柱面的数量 作为判别之一：是；判别：该类别模型中圆柱面数量为3
13	0	/	是否采用 圆锥面的数量 作为判别之一：否
14	0	/	是否采用 球面的数量 作为判别之一：否
15	0	/	是否采用 直线边的数量 作为判别之一：否
16	0	/	是否采用 圆弧边的数量 作为判别之一：否
17~32	0	/	其他待扩充的判别

图 3-3 螺钉的一种判定
Fig.3-3 Judgement of screw

需要注意的是，在持久化存储时，需要将标准件对应于全部编码规则的结果进行存储。

3.2.3 分类流程

标准件分类流程如图 3-4 所示，其具体步骤如下。

首先，系统进行如第二章中所述的预处理过程，利用 pythonOCC 对用户输入的 B-rep 模型中的几何拓扑信息进行读取，并建立工程顶点、工程边、工程面的集合以及相互之间的映射关系。

设当前系统标准件库中已有 n 类标准件，记为 $S=\{s_0, s_1, \dots, s_{n-1}\}$ ，每类标准件对应的规则编码记为 $R=\{r_0, r_1, \dots, r_{n-1}\}$ ，每个标准件类别对应的结果编码记为 $T=\{t_0, t_1, \dots, t_{n-1}\}$ 。

步骤一 从数据库中读取每个标准件类别的规则编码和结果编码并封装为对应的 rule_code 和 result_code 对象，下述流程中对集合 R 和集合 T 中元素的操作实际都是对对应的 rule_code 对象和 result_code 对象的操作。创建集合 $B=R$ 。

步骤二 从规则编码第一位的规则开始依次向后判断，每一位的判断方式如下：设当前判断到了第 i 位的规则 P_i ，用户输入的零件 B-rep 模型对应于 P_i 的结果为 Q_i ，从 B 中的最后一个类别依次向前遍历，若当前类别以 P_i 为判定规则之一且当

前类别对应 P_i 的结果不为 Q_i ，则从 B 中移除该类别，直到：

- ① B 中仅剩一个类别 s ，则该零件属于类别 s ；
- ② 校验完全部规则， B 中剩下多个类别 $s_i, \dots, s_j$ ，则 n 属于类别 s_j （在 B 中排序最靠后的那个类别）。

由于该标准件分类算法本质上是实现了不同标准件类别的区分，所以可能会将一个不属于任何已有标准件类别的零件判断为属于某一类标准件。为了避免这一问题，设上述流程的分类结果为零件属于 s_k 类标准件，则最后需要判断零件是否符合 t_k 中的所有结果，若全都符合，则该零件属于 s_k 类，否则该零件不属于任何一种标准件类别。

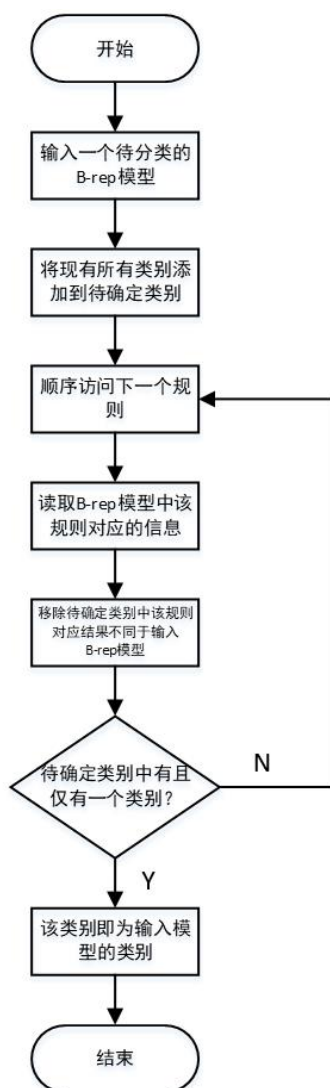


图 3-4 标准件分类流程图

Fig.3-4 Flow chart of standard part classification

3.3 标准件的参数提取

在确定了用户输入的零件模型所属的标准件类别后，需要进一步确定该零件属于这个标准件类别中的哪一个规格，对应的参数是多少。

由于不同类别的标准件参数的数量、类型都不相同，并且一个完整的零件要比局部的特征复杂得多，因此不适合采用第二章中提出的特征识别的思想实现标准件的参数提取。为了解决这一问题，本课题利用标准件是一系列具有不同规格的零件的特点，即标准件的参数是离散的，建立标准件的规格参数和通用的基本几何属性之间的双向映射，以实现不同标准件类别之间采用统一的描述，能够得到各自的规格参数。

本课题以实体模型的体积、表面积、棱边总长度作为规格参数的统一映射，通过计算实体模型的体积、表面积和棱边总长度三个几何属性，去查询规格参数表中这三个参数与之最为接近的规格，即为零件所属的标准件的规格，具体的参数通过查询规格参数表即可获得。

通过 pythonOCC 提供的工具类，获取零件的体积记为 $V(\text{mm}^3)$ ，表面积记为 $A(\text{mm}^2)$ ，棱边总长度记为 $L(\text{mm})$ 。设规格参数表中记录每个规格对应的体积、表面积和棱边总长的字段名分别为 $\text{volume}(\text{mm}^3)$ 、 $\text{area}(\text{mm}^2)$ 和 $\text{length}(\text{mm})$ ，则查询各个规格对应的 $|\sqrt[3]{V} - \sqrt[3]{\text{volume}}| + |\sqrt{A} - \sqrt{\text{area}}| + |L - \text{length}|$ 中值最小的规格，即为该零件的规格。

3.4 标准件类别的扩展

由于本系统只是提供了一个可用的报价框架，未内置足够多的标准件类别，并且在实际中标准件的种类成千上万，必然需要不断地向系统中添加新的标准件类别以丰富标准件库，提高实用价值。标准件类别的添加最重要的是要具有自适应的特性，即用户只需要添加必要的规格参数、示意图和 B-rep 模型等基本信息，就可以实现该类别零件的分类与参数提取，而不需要在代码层面再进行算法逻辑的添加或修改。其自适应的特性主要包含两个方面：一是分类的自适应，二是参数提取的自适应。

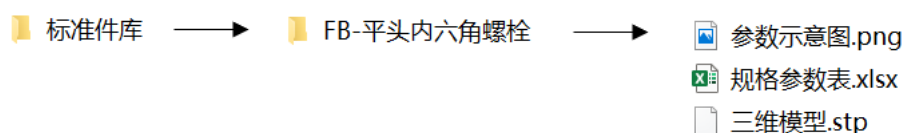


图 3-5 添加标准件类别需要的文件
Fig.3-5 Files for adding a standard part category

为了实现以上所述的自适应特性，系统要求用户在添加新的标准件类别时至少需要提供如下几个基本的文件：一个该类标准件的 B-rep 实体模型文件、一个规格参数表文件（Excel 表格的形式）。此外为了更好的显示标准件外形和参数与实体形状的对应关系，建议添加一个图片文件作为示意图。这些文件应当存放在一个文件夹内，文件夹名为标准件类别的名称。以平头六角螺栓标准件为例，其文件形式如 3-5 图所示，包含一个.stp 模型文件、Excel 表格和图片文件，具体内容分别如图 3-6、图 3-7 和图 3-8 所示。

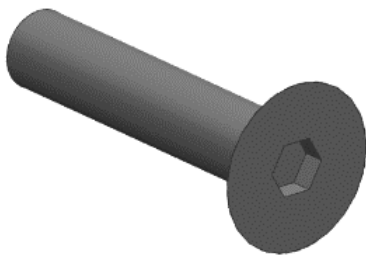


图 3-6 平头六角螺栓的 B-rep 模型文件内容
Fig.3-6 Content of B-rep model file for hexagon socket head bolt

1	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R
2	id	No	T	A	B	C	E	F	G	H	J	R	d1	d2	total_volume	totalArea	totalLength	price_1_to_9
3	2	10	8	37	20	21	35	20	10	9	29	9.5	6.2	10	5144.84	3334.73	550.72	60
4	3	13	6	43	26	23	38	20	13	11	32	12	6.2	10	8523.8	4491.06	597.93	60
5	4	13	9	43	26	23	38	20	13	11	32	12	6.2	10	12786.7	5315.96	621.93	60
6	5	16	5	44	24	26	40	20	13	12	34	14	6.2	12	7821.39	4615.31	634.7	60
7	6	16	7	44	24	26	40	20	13	12	34	14	6.2	12	10949.94	5210.01	660.7	60
8	7	16	9	44	24	26	40	20	13	12	34	14	6.2	12	14078.49	5804.71	666.7	60
9	8	20	6	48	28	27	42	20	16	14	36	17	6.2	12	11817.49	5822.82	675.89	69
10	9	20	8	48	28	27	42	20	16	14	36	17	6.2	12	15756.66	6450.71	691.89	69
11	10	20	10	48	28	27	42	20	16	14	36	17	6.2	12	19695.82	7078.59	707.89	69
12	11	25	7	50	30	30	46	20	18	17	39	19.5	6.2	12	16809.47	7118.75	717.73	70
13	12	25	9	50	30	30	46	20	18	17	39	19.5	6.2	12	21612.18	7780.48	733.73	70
14	13	32	6	54	30	33	48	20	21	20	42	23	6.2	14	17231.58	7915.12	771.75	73
15	14	32	8	54	30	33	48	20	21	20	42	23	6.2	14	22975.44	8638.87	787.75	73
16	15	32	10	54	30	33	48	20	21	20	42	23	6.2	14	28719.3	9362.62	803.75	73
17	16	38	6	58	30	35	52	20	23	23	46	26	10.2	14	20931.42	9350.7	839.18	75
18	17	38	8	58	30	35	52	20	23	23	46	26	10.2	14	27908.56	10141.88	855.18	75
19	18	38	10	58	30	35	52	20	23	23	46	26	10.2	14	34885.7	10933.07	871.18	75
20	19	45	7	64	34	38	56	20	26	27	50	29.5	10.2	14	30566.53	11676.69	896.97	92
21	20	45	9	64	34	38	56	20	26	27	50	29.5	10.2	14	39299.83	12517.65	912.97	92

图 3-7 平头六角螺栓的 Excel 文件内容
Fig.3-7 Content of Excel file for hexagon socket head bolt

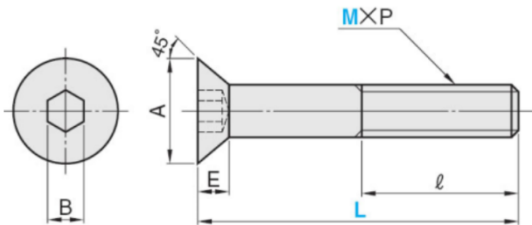


图 3-8 平头六角螺栓的示意图文件内容
Fig.3-8 Content of picutre file for hexagon socket head bolt

对于标准件的分类而言，需要在用户添加一个类别后自动地设计出该类别的

分类规则即规则编码，具体流程如下：

假设当前系统标准件库中已有 n 类标准件，记为 $S=\{s_0, s_1, \dots, s_{n-1}\}$ ，每类标准件对应的规则编码记为 $R=\{r_0, r_1, \dots, r_{n-1}\}$ ，每类标准件对应的结果编码记为 $T=\{t_0, t_1, \dots, t_{n-1}\}$ ，此时添加一个新的标准件类别 s ，具体输入包括 s 的 B-rep 模型和 s 的规格参数表。

根据编码规则，依次获取该模型的相关拓扑信息，得到结果编码 t_{n+1} ，用 32 个整型字段进行表示以持久化存储。

创建集合 $A=R$ 。按照规则编码中的顺序，依次对每个规则进行判断：如当前判断到第 i 个规则 P_i ，根据 t_{n+1} 得到该零件模型对应于 P_i 的结果为 Q_i ，然后遍历集合 A ，当 A 中所有对应于 P_i 的结果都为 Q_i 时，则不以 P_i 作为判别规则，对应第 i 位的规则编码置为 0；否则以 P_i 作为编码规则，对应第 i 位的规则编码置为 1，并将 A 中对应 P_i 的结果不为 Q_i 的类别移除。重复上述步骤来判断规则 P_{i+1} ，直至 A 为空集。此时将 r 中后面还未判断过的规则都置为 0，即为 s 的规则编码，将 r 添加到集合 R 的最后记为 r_{n+1} ，并将该规则编码用 1 个整型变量字段（位图）进行表示以持久化存储。

对于标准件的参数提取而言，只需要将用户上传的规格参数表写入数据库进行持久化存储即可，无需特殊处理。

3.5 验证与分析

本章设计了一种标准件检索算法，以实现标准件的报价。如图 3-9 所示，为了对该算法的性能进行测试，本系统添加了固定挡块、平头六角螺栓、内六角螺栓和定位销 4 种标准件类别，共 75 个零件。经测试，在对这 75 个零件的报价中，系统能够准确识别全部零件的类别与对应的规格参数，报价的准确率为 100%，报价的平均时间为 0.301s，其主要耗时为 B-rep 模型文件的解析、数据库读写和网络传输，算法本身的计算耗时基本可以忽略不计。

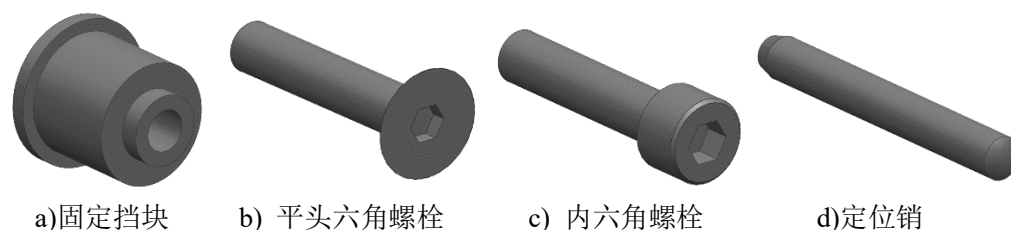


图 3-9 系统内置的四种标准件类别

Fig.3-9 Four standard part categories in quotation system

标准件报价的实际应用必须建立在大规模标准件库的基础之上，需要保证在

标准件类别足够多的情况下，仍能够准确地进行检索、报价，并且自适应添加类别的耗时不会随着零件类别数量的增多而激增。此外为了模拟算法在实际应用中标准件类别繁多的场景，通过随机生成规则编码和结果编码的形式，在原有的 4 种标准件基础之上，添加了 9996 种“标准件”，共 10000 种标准件类别。图 3-10 为添加标准件的算法耗时随标准件类别增多的变化情况，可以看出，算法的耗时并没有随标准件类别增多而激增，而是平稳地线性增长，在 10000 个类别的情况下，标准件类别的添加和报价用时均在 6s 左右，满足大规模标准件类别报价的要求。除此之外，在 10000 个标准件类别的条件下，对固定挡块、平头六角螺栓、内六角螺栓和定位销 4 种标准件类别共 75 个零件的检索、报价经验证，准确率仍为 100%。综上所述，该算法可以应用于工程实际中。

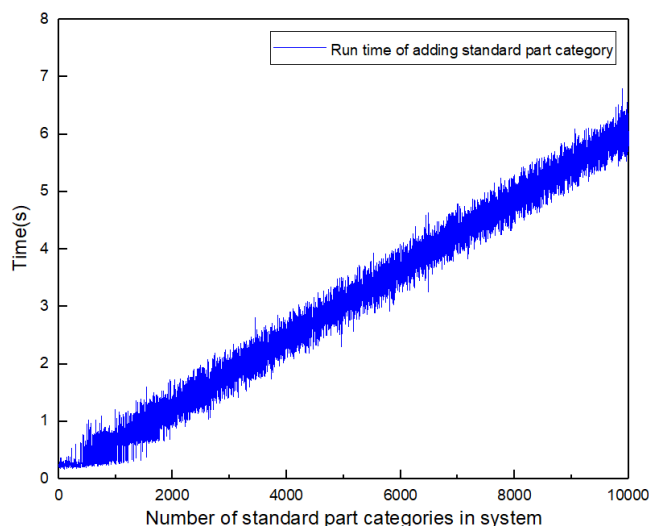


图 3-10 标准件类别增多对添加标准件类别用时的影响

Fig.3-10 Run time of adding standard part with increasing current standard part category number

但是该算法仍然具有如下缺点：一是该算法的本质是对不同标准件类别进行区分，并不是严格的去匹配某个标准件的拓扑结构，因此可能会出现某个不属于任何一个标准件类别的零件被判定为属于某个标准件类别；二是在规格参数的提取中，该算法将参数映射为基本几何属性，其前提是标准件的参数都是离散的，但在实际中有时标准件的参数取值是连续区间，该算法不适用于这种情况。

3.6 本章小结

本章设计了针对标准件的报价算法，利用标准件规格参数是离散的以及基本拓扑信息可以产生海量的组合的原理，设计了一种适用于标准件 B-rep 模型的检索算法，用于实现标准件的快速报价，同时该算法能够实现在添加标准件类别时

自适应地生成新类别的检索规则，满足了报价系统对良好的扩展性的要求，适合于标准件类别繁多的实际应用场景。

最后，添加 4 个具体的标准件类别和随机模拟生成的 9996 个标准件类别，在此基础上对算法的性能进行了测试。经测试，在内置了 10000 种标准件类别的情况下，算法能够准确地对标准件进行报价，并且标准件类别的添加耗时并不会随已有标准件类别数量增多而激增，而是平缓地线性上升，在 10000 种标准件类别的条件下，添加类别和检索报价的耗时均约为 6s，适合实际应用。

第四章 轴类零件和一般零件的报价

4.1 引言

标准件报价的本质是对数据库的检索，由于标准件是市场上已经存在固定规格型号的零件，其报价的准确率是可以保证的。非标准件由于没有固定的规格参数，很多时候市场上找不到现成的产品及对应的价格。其价格主要是通过对零件的加工过程分析得到的。

对于非标准件的报价，本系统将其分为两类分而治之地进行报价：一类是在拓扑结构上虽不完全相同，但具有某种共性，比如主要加工方式相同，如轴类零件（以车削为主）、箱体类零件（以铸造为主），这类零件可以视为一个整体设计出较为准确的报价算法并植入报价系统。其中，对于轴类零件，本系统实现了其基于推理加工过程的报价；另一类则是不具有某种显著的特征，难以分类，也就是一般零件。这类零件的报价依靠零件库中的相似零件信息的重用。据统计，大部分零件与生产中已有的零件完全相同或只有微小的改动，相似的零件往往具有相似的价格，正是因为这一点，采用基于相似零件的方式对一般零件进行报价。

4.2 轴类零件的报价

轴类零件是机械领域最常见的零件类型之一，主要起着传动、支撑和定位的作用，常常与其他零件进行联接和配合，其主要加工方式通常为车削加工。从几何形状上来讲，轴类被定义为长径比大于 1 的回转体。轴类零件包含了许多其他零件所不具有的特征，如键槽、退刀槽、同轴台阶等，这些特征根据位置的不同可以分为端面特征和外圆特征，根据是否为回转类且与主轴同轴可以分为主特征和辅特征，根据是否在实际中多个特征联合表示分为单一特征和组合特征。

4.2.1 轴类零件的判定

系统首先需要判断用户输入的零件是否为轴类零件，具体是判断依据以下两个条件：

- （1）零件是否为回转体；
- （2）若零件为回转体，其长径比是否大于 1。

对于条件（1）的判定，采用如下的算法实现：

步骤一 对用户输入的零件 B-rep 模型进行如第二章中所述的预处理，并提取基本特征。得到全局变量 *feature_list* 和 *eng_edge_list*。

步骤二 遍历 *feature_list*，每访问到一个基本特征，通过类内的 *eng_face_index_list* 遍历该特征中的所有工程面，每访问到一个工程面，进一步遍历该工程面中所有工程边，每访问到一条未访问过的工程边，将其标记为已访问。

步骤三 创建列表 *arc_list*，用于存放主体/基体部分的圆弧类型工程边。全局变量遍历 *eng_edge_list*，每访问到一条未访问过的工程边，则判断其类型是否为圆弧边：

若是圆弧边，则添加该工程边在 *eng_edge_list* 中的位置索引到 *arc_list* 中；

若不是圆弧边，则判定该零件模型不是轴类零件，算法结束。

步骤四 通过 *edge_list* 遍历 *arc_list*，每访问到一个工程边，利用 *BrepAdaptor_Curve* 类中的 *Circle* 方法将该工程边中的任一模型边转换为基本的圆弧几何类 *gp_Circ*。通过 *gp_Circ* 中的 *Axis* 方法得到该圆弧边的轴线。判断这些轴线是否都是共线的，若这些轴线不都是共线的则判定该零件不是轴类零件，算法结束；若这些轴线全都共线，则该零件为回转类零件，记其中心轴为 *A*。

步骤五 通过 *eng_edge_list* 遍历 *arc_list*，每访问到一个工程边，利用 *BrepAdaptor_Curve* 类中的 *Circle* 方法将该工程边中的任一模型边转换为基本的圆弧几何类 *gp_Circ*。通过 *gp_Circ* 中的 *Radius* 方法得到该圆弧的半径，最终得到轴线上全部圆弧边中半径最大者，其半径记为 *R*。

步骤六 以图 2-6 所示的方法得到轴线 *A* 两个方向上的极远点 *P₁* 和 *P₂*，分别以其为中心，创建一个边长为 $2R$ 的正方形平面实体对象，分别求取零件实体模型到这两个面的最小距离点记为 *Q₁* 和 *Q₂*。计算 *Q₁* 和 *Q₂* 在轴线 *A* 上的投影点的距离 *L*，*L* 即为零件的长度。若 $L \geq 2R$ ，则零件为轴类零件，否则零件不为轴类零件。

4.2.2 轴类零件的特征识别

本系统中内置了 12 种具体的轴上特征，包括：直杆形、轴端锥形（轴端倒角）、卡簧槽、V 形槽、轴端内六角孔、止动螺丝用槽、退刀槽、键槽、扳手槽、台阶、端面孔和侧面孔，以及 3 种组合特征：阶梯台阶、端面阶梯孔和侧面阶梯孔。特征识别的方法采用第二章所述的属性邻接图匹配的方法。具体的特征类型及对应的属性邻接图和邻接矩阵表示如下，其中深色结点代表特征对应的种子环的基面，字母和数字含义见表 2-3、表 2-4 和表 2-5。

（1）直杆形

直杆形属于基体类型，故不存在种子环与基面。其形状为一个圆柱体，包含三个面，由于倒角、台阶、端面孔等特征的存在，圆柱体所连接的面可能为平面、

圆锥面或圆环面等，因此将这两个面的类型属性规定为任意类型。

在本系统中的特征库中，直杆形的默认加工方式为车削。其特征模型、属性邻接图和对应的邻接矩阵如图 4-1 所示。

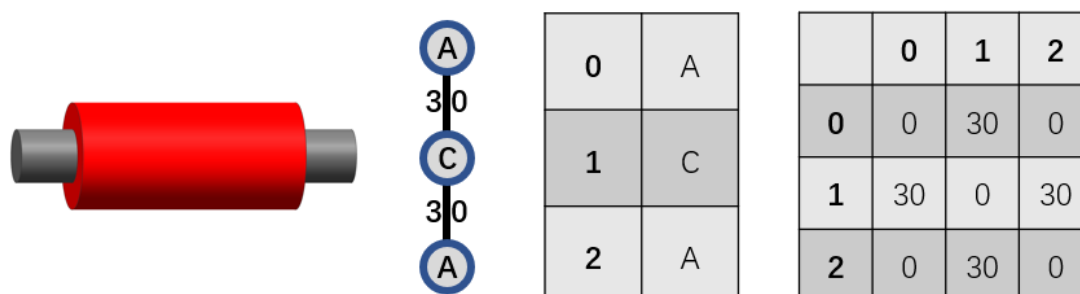


图 4-1 直杆形的模型、属性邻接图和邻接矩阵

Fig.4-1 Model, attribute adjacency graph and adjacency matrix of straight-rod shape

(2) 轴端锥形（轴端倒角）

轴端锥形属于特征类型，当其角度为 45° 时成为轴端倒角，其主要作用是去除零件上的毛刺和便于装配。该特征种子环的基面为圆锥面，特征中只包含一个平面，圆锥基面和特征中的平面以圆弧凸边连接。需要注意的是，由于端面上可能存在端面孔等特征，因此特征中包含的也可能是圆环面，在本系统中圆环面类型被认为属于平面类型。

在本系统中的特征库中，轴端锥形的默认加工方式为车削。其特征模型、属性邻接图和对应的邻接矩阵如图 4-2 所示。

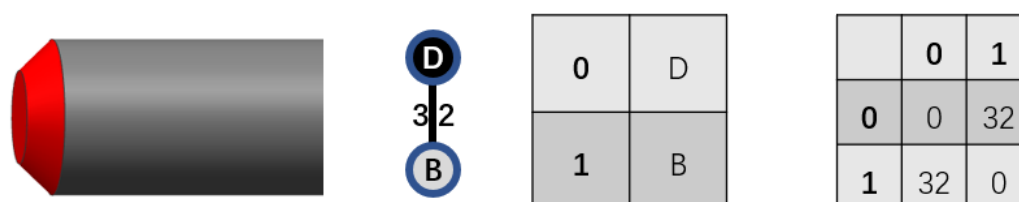


图 4-2 轴端锥形（轴端倒角）的模型、属性邻接图和邻接矩阵

Fig.4-2 Model, attribute adjacency graph, and adjacency matrix of shaft end cone (shaft end chamfer)

(3) 卡簧槽

卡簧槽属于特征类型，其作用为放置卡簧以进行紧固。该特征既包含 `start_loop` 也包含 `end_loop`，但以两个环分别作为种子环时，它们的属性邻接图是同构的，因此在特征库中只存放其中一个即可。

在本系统的特征库中，卡簧槽的默认加工方式为车削。其特征模型、属性邻

接图和对应的邻接矩阵如图 4-3 所示。

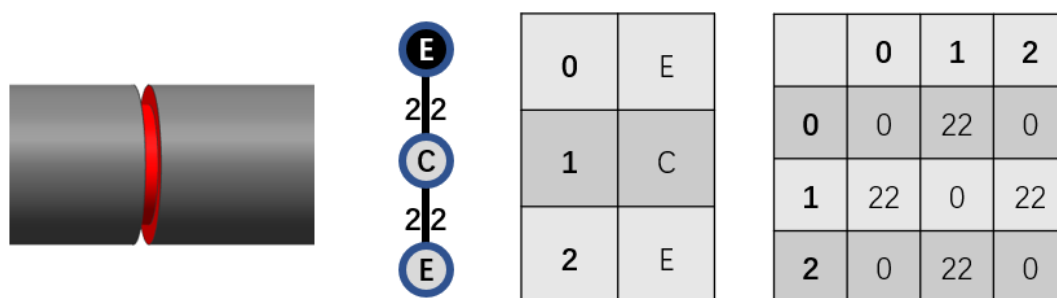


图 4-3 卡簧槽的模型、属性邻接图和邻接矩阵

Fig.4-3 Model, attribute adjacency graph and adjacency matrix of circlip

(4) V 形槽

V 形槽属于特征类型，该特征中不包含任何工程面，仅仅由两个重合的种子环分别对应的基面组成。与卡簧槽类似，在特征库中只存放一条记录即可。

在本系统中，V 形槽的默认加工方式为车削。其特征模型、属性邻接图和对应的邻接矩阵如图 4-4 所示。

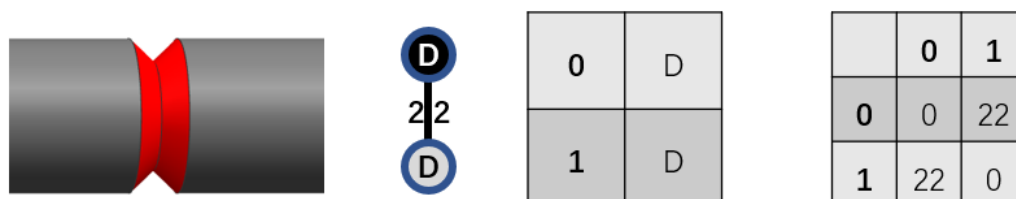


图 4-4 V 形槽的模型、属性邻接图和邻接矩阵

Fig.4-4 Model, attribute adjacency graph and adjacency matrix of V-groove

(5) 轴端内六角孔

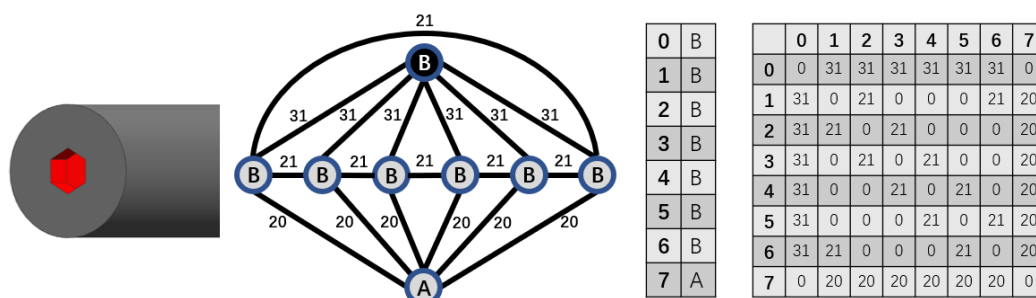


图 4-5 内六角孔的模型、属性邻接图和邻接矩阵

Fig.4-5 Model, attribute adjacency graph and adjacency matrix of hexagonal hole

轴端内六角孔属于特征类型，主要用于上紧螺栓等行为时扭矩的传递。该特

征中包含 7 个面，其中 6 个侧面的类型为平面，1 个底面的类型可能为平面可能为平面也可能为圆锥面或其他不规则的面，由于其类型从实际使用的角度看来并不影响该特征的功能，因此在属性邻接图中将其表示为任意类型。

由于内六角孔的加工方式较多，包括冲孔、拉孔或是采用特殊的刀具直接切削成型等方式在，因此在系统中不考虑其具体加工方式，而是直接记录一个由材料、尺寸确定的成本。其特征模型、属性邻接图和对应的邻接矩阵如图 4-5 所示。

(6) 止动螺丝用槽

止动螺丝用槽属于特征类型，用于与止动螺丝配合使用。与卡簧槽类似，在特征库中只存放一条记录即可。

在本系统中，V 形槽的默认加工方式为车削。其特征模型、属性邻接图和对应的邻接矩阵如图 4-6 所示。

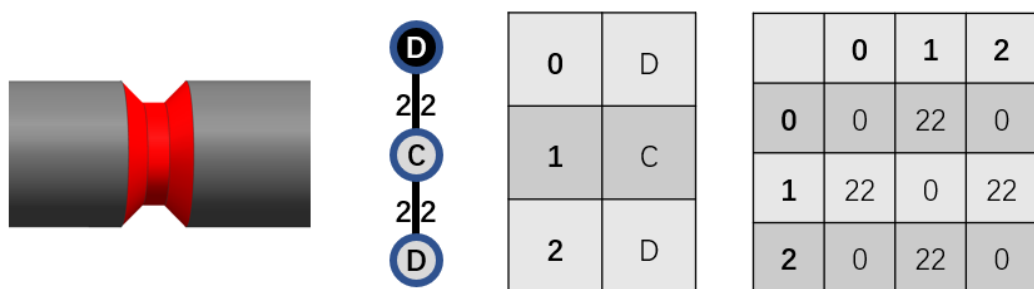


图 4-6 止动螺丝用槽的模型、属性邻接图和邻接矩阵

Fig.4-6 Model, attribute adjacency graph and adjacency matrix of groove for stop bolt

(7) 退刀槽

退刀槽属于特征类型，用于车内孔、螺纹时的退刀。如第二章所述，由于存在两个不同类型的基面，因此需要在特征库中存放两条对应的记录。

在本系统中，退刀槽的默认加工方式为车削。其特征模型、属性邻接图和对应的邻接矩阵如图 4-7 所示。

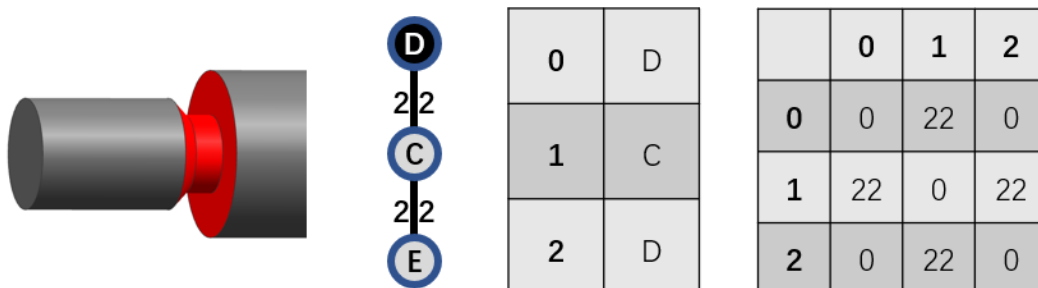


图 4-7 退刀槽的模型、属性邻接图和邻接矩阵

Fig.4-7 Model, attribute adjacency graph and adjacency matrix of relief groove

(8) 键槽

键槽属于特征类型，用于与键块配合使用以实现传动等功能。该特征位包含 5 个工程面，4 个侧面包括 2 个平面和 2 个圆柱面，底面为平面。键槽的功能决定了其种子环的基面是轴类的外圆，故将 0 号结点属性限定为圆柱面类型。

在本系统中，键槽的默认加工方式为铣削。其特征模型、属性邻接图和对应的邻接矩阵如图 4-8 所示。

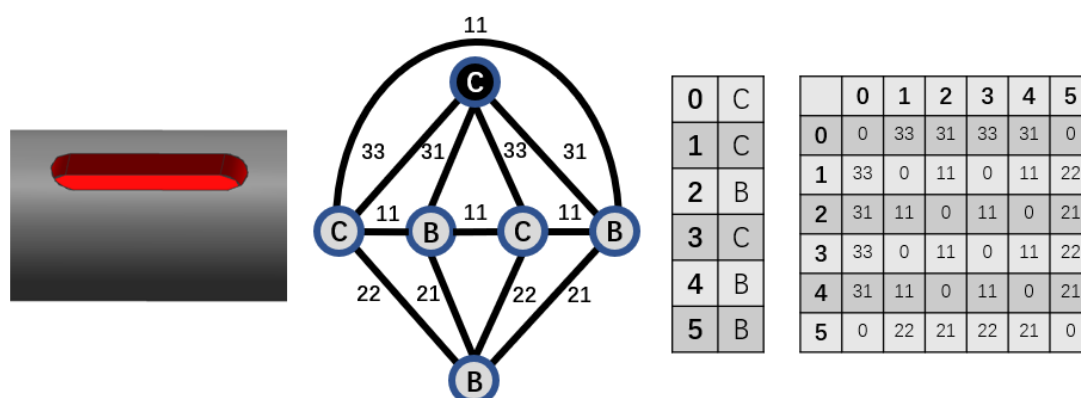


图 4-8 键槽的模型、属性邻接图和邻接矩阵

Fig.4-8 Model, attribute adjacency graph and adjacency matrix of keyway

(9) 扳手槽

扳手槽属于特征类型，用于配合扳手进行使用。该特征包含三个工程面，类型均为平面，其中与主轴平行的平面和特征种子环的基面由两条边连接，由于两条边的类型都是直线类型的凸边，因此将对应该连接关系的弧的属性表示为 31，即直线类型的凸边。当系统检测到特征中的两个工程面之间以多条工程边连接时，只有其中所有工程边均为直线类型的凸边，才认为匹配成功该特征的属性邻接图中这一条弧（及其相连的结点）。扳手槽的功能决定了在轴类零件中，该特征的种子环对应的基面应当为轴类的外圆，故将 0 号结点的属性限定为圆柱面类型。

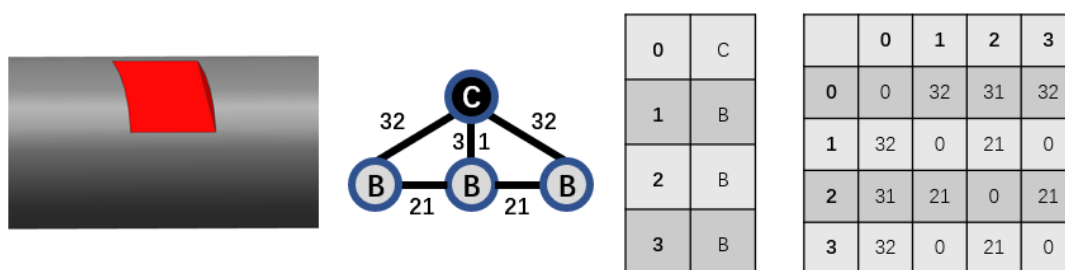


图 4-9 扳手槽的特征模型、属性邻接图和邻接矩阵

Fig.4-9 Model, attribute adjacency graph and adjacency matrix of wrench groove

在本系统中，扳手槽的默认加工方式为铣削。其特征模型、属性邻接图和对应的邻接矩阵如图 4-9 所示。

(10) 台阶

台阶属于特征类型，用于实现与不同内径的零件进行配合等功能。该特征包含两个工程面，其中最重要的是外侧面的圆柱面。由于倒角、锥形、端面孔、台阶等等类型的存在，该特征的顶面类型可能为平面类型、圆环类型或者圆锥面类型等，这些并不是决定一个特征是否为台阶特征的决定性因素，因此将顶面对应属性邻接图结点的属性表示为 A，即不限面的类型。

在本系统中，台阶的默认加工方式为车削。其特征模型、属性邻接图和对应的邻接矩阵如图 4-10 所示。

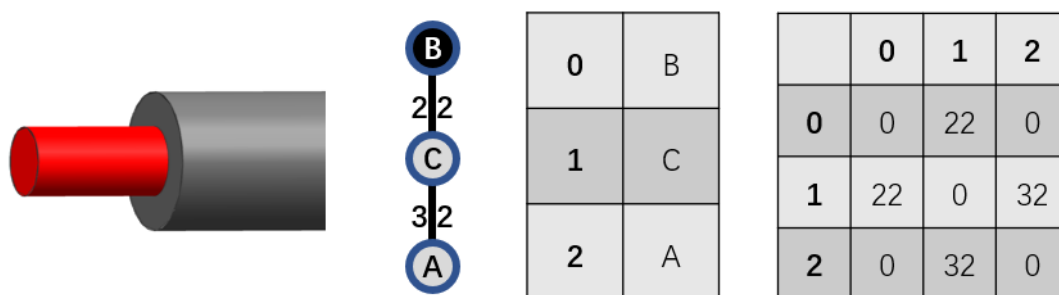


图 4-10 台阶的特征模型、属性邻接图和邻接矩阵

Fig.4-10 Model, attribute adjacency graph and adjacency matrix of step

(11) 端面孔

端面孔属于特征类型，用于实现与不同外径的零件进行配合等功能。该特征包含两个工程面，其中其决定性作用的是内侧面的圆柱面。由于端面孔特征的存在和孔加工方式的不同等因素，该特征的底面可能为平面类型、圆环类型或圆锥面类型等。这些并不是决定一个特征是否为端面孔特征的决定性因素，因此将底面对应属性邻接图结点的属性表示为 A，即不限面的类型。

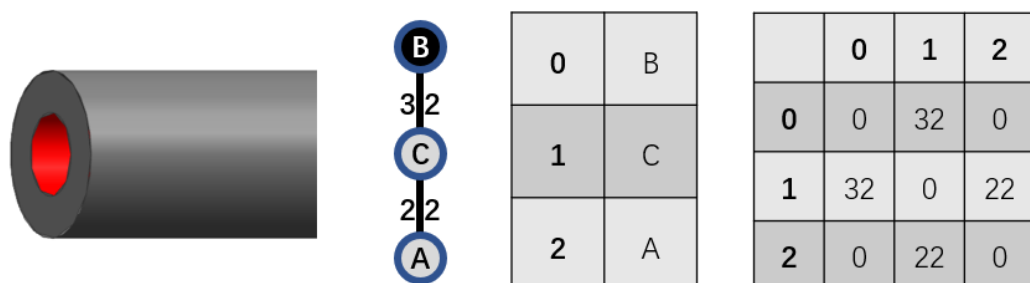


图 4-11 端面孔的特征模型、属性邻接图和邻接矩阵

Fig.4-11 Model, attribute adjacency graph and adjacency matrix of end face hole

在本系统中，端面孔的默认加工方式根据孔径的不同分为钻孔和钻孔+车削。其特征模型、属性邻接图和对应的邻接矩阵如图 4-11 所示。

(12) 侧面孔

侧面孔属于特征类型，用于实现某些特定功能。该特征包含两个工程面，其中其决定性作用的是内侧面的圆柱面。由于侧面阶梯孔特征的存在和孔加工方式的不同等因素，该特征的底面可能为平面类型、圆环类型或圆锥面类型等。这些并不是决定一个特征是否为侧面孔特征的决定性因素，因此将对应属性邻接图结点的属性表示为 A，即不限面的类型。

在本系统中，端面孔的默认加工方式为钻孔。其特征模型、属性邻接图和对应的邻接矩阵如图 4-12 所示。

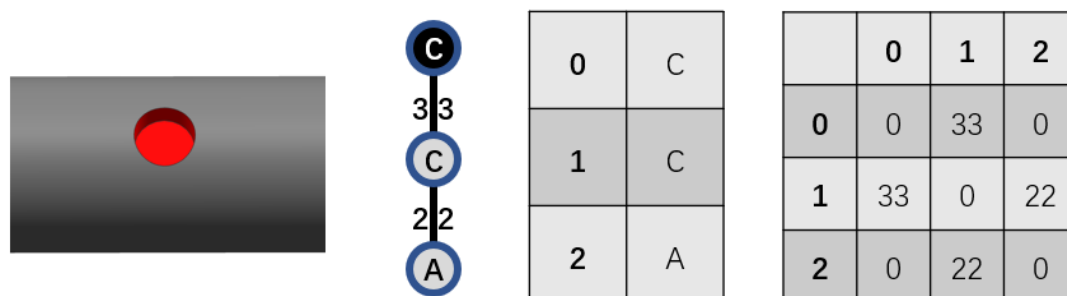


图 4-12 侧面孔的特征模型、属性邻接图和邻接矩阵

Fig.4-12 Model, attribute adjacency graph and adjacency matrix of side face

从实际的角度出发，特征除了要满足属性邻接图表示的拓扑规则外，还常常需要满足其他一些规则，如六边形内孔中侧面是否为正六边形、台阶或端面孔是否与轴类主体同轴等。因此在以上述的属性邻接图匹配的方式识别出特征后，仍需要以一些简单的规则对特征类型进行校验。

上述所有特征均为单一特征类型，除此之外，系统还内置了 3 种组合特征类型，包括端面阶梯台阶、端面阶梯孔和侧面阶梯孔。在提取出所有的单一特征并添加到 *feature_list* 中后，对这 3 种组合特征进行提取，即进行特征的合并。下面以阶梯孔这一组合特征的提取为例描述特征合并的过程。

首先定义组合特征类型枚举类和组合特征类：

```
class CombineFeatureType:
    OTHER = 1
    COMBINE_END_HOLE = 2
    COMBINE_END_STEP = 3
    COMBINE_SIDE_HOLE = 4
```

其中, OTHER 表示其他未知或不确定的类型; COMBINE_END_HOLE 表示端面阶梯孔; COMBINE_END_STEP 表示阶梯台阶; COMBINE_SIDE_HOLE 表示侧面阶梯孔。

```
class CombineFeature:
    def __init__(self):
        self.feature_index_list = []
        self.type = CombineFeatureType.OTHER
```

其中, *self.feature_index_list* 是一个用于存放组成组合特征的特征在 *feature_list* 中的位置; *self.type* 表示组合特征的类型, 默认为其他类型。

特征组合的基本思路是遍历所有特征, 每访问到一个特征 F_i 时, 进一步遍历其他所有特征, 判断哪一个特征 F_j 是否是 F_i 的子特征 (F_j 的种子环的基面属于 F_i 的工程面列表, 称 F_j 为 F_i 的子特征, F_i 为 F_j 的父特征), 然后进一步遍历所有未使用的特征, 判断其是否为 F_j 的子特征...这种算法的最坏时间复杂度为 $O(n!)$, 其中 n 为特征的数量, 在特征数量很多的情况下, 效率非常低。由于对于阶梯孔而言, 其包含的基本特征均为孔特征这种圆柱特征, 由直径和深度/高度两个参数标识其几何尺寸, 在一个阶梯孔中, 子特征与父特征存在如下约束关系:

- (1) 子特征和父特征均为孔特征 (凹陷的圆柱特征);
- (2) 子特征孔的直径小于父特征孔的直径;
- (3) 子特征孔的与父特征孔同轴。

利用这些约束, 对特征组合的算法进行改进, 以孔的直径作为比较值对孔特征的列表进行降序排序, 每次寻找当前孔特征子特征时, 只需要从当前孔特征起向后遍历列表即可, 一共只需要对列表进行一次两重循环遍历, 算法的最坏时间复杂度从 $O(n!)$ 降至 $O(n^2)$, 其中 n 为孔特征的数量。其具体步骤如下:

步骤一 创建一个列表 *hole_index_list*, 用于存放所有孔特征 (端面同轴孔、外圆孔和一般孔)。遍历 *feature_list*, 每访问到一个特征, 若该特征的类别为 FeatureType.END_HOLE (端面同轴孔)、FeatureType.SIDE_HOLE (外圆孔) 或 FeatureType.ORDINARY_HOLE (一般孔), 则添加该特征在 *feature_list* 中的位置索引到 *end_hole_index_list* 中。

步骤二 对 *end_hole_index_list* 以位置索引对应的孔特征的直径大小为比较值作降序排序。

步骤三 创建一个类型为列表的全局变量 *combine_feature_list*, 用于存放组合特征。通过 *feature_list* 遍历排序后的 *end_hole_index_list* 中的孔特征, 每访问到一

个未被使用的孔特征记为 H_i ，将 H_i 标记为已被使用，创建一个组合特征对象 $combine_feature$ ，将 H_i 在 $feature_list$ 中的位置索引添加到当前组合特征对象 $combine_feature$ 内部的 $feature_index_list$ 中，判断 H_i 的类型：

(1) $H_i.type == END_HOLE$ ，即 H_i 为与主轴同轴的端面孔，则将 $combine_feature$ 的 $type$ 赋值为 $COMBINE_SIDE_HOLE$ 类型；

(2) $H_i.type == SIDE_HOLE$ ，即 H_i 为外圆孔，则将 $combine_feature$ 的 $type$ 赋值为 $COMBINE_SIDE_HOLE$ ；

(3) $H_i.type == ORDINARY_HOLE$ ， H_i 为一般孔，则将 $combine_feature$ 的 $type$ 赋值为 $OTHER$ 。

进一步通过 $feature_list$ 遍历 $end_hole_index_list$ 中在当前孔特征后面的孔特征，每访问到一个未被使用的孔特征记为 H_j ，判断 H_j 的 $start_loop$ 对应的基面在 eng_face_list 中的位置索引是否存在于 H_i 的 $eng_face_index_list$ 中：

若是，则 H_j 在 $feature_list$ 中的位置索引添加到当前组合特征对象 $combine_feature$ 内部的 $feature_index_list$ 中，并将 H_j 标记为已被使用；

若不是，则跳过该孔特征，继续访问下一个。

步骤四 遍历 $combine_feature_list$ ，每访问到一个组合特征，判断其内部的 $feature_list$ 的长度是否为 1，即该组合特征是否仅仅包含一个单一特征：

若是，则将该组合特征对象从 $combine_feature_list$ 中移除；

若不是，则遍历该特征的跳过该组合特征对象，继续访问下一个。

然后用相同的方法对阶梯台阶进行提取。至此，阶梯孔和阶梯台阶两种组合特征都已经存放在 $combine_feature_list$ 中。

特征的属性除了自身的几何尺寸等参数外，还包括其在零件上的位置。由于轴类零件主体为车削而成，存在着一个主轴，天然就包含一个用于特征定位的一维坐标系，针对轴类零件的这一特点，利用 python 语言的动态特性，在 **Feature** 和 **BaseBody** 对象中动态添加 **position** 字段，分别表示特征/基体在轴线上的位置和沿轴线的方向，具体过程如下：

步骤一 轴类零件模型的摆正

设已经得到轴类零件的主轴为 A ，利用图 2-6 所述的方法获得实体模型在轴 A 方向上的两个极值点记为 O_1 和 O_2 ，并将 P_1 和 P_2 分别沿垂直方向在 A 上的投影点记为 P_1 和 P_2 。利用 pythonOCC 提供的获取实体模型属性的 **GProp_Gprops** 类中的 **CentreOfMass** 方法，得到该轴类零件的质心点记为 Q ，记 P_1 到 Q 的距离为 L_1 ， P_2 到 Q 的距离为 L_2 ，比较 L_1 和 L_2 ：

(1) $L_1 < L_2$, 说明 P_1 侧更“重”, 以 P_1 作为轴上计算特征位置的原点, 记 $O_3 = P_1$, $O_4 = P_2$;

(2) $L_1 > L_2$, 说明 P_2 侧更“重”, 以 P_2 作为轴上计算特征位置的原点, 记 $O_3 = P_2$, $O_4 = P_1$;

(3) $L_1 = L_2$, 说明零件质心恰好在轴线属于零件部分的中点的法平面上, 结合工程实际情况, 认为该零件在该法平面两侧形状相同, 任取 P_1 或 P_2 中某点为 O_3 , 另一点为 O_4 。

利用 pythonOCC 提供的 `BRepBuilderAPI_GTransform` 类对零件模型进行平移与旋转, 将 O_3 平移至世界坐标系的原点位置, 并将向量 O_3O_4 旋转为与世界坐标系中的 X 轴同向, 至此完成轴类零件模型的摆正。

步骤二 提取特征的位置

对于不同的特征而言, 其位置和方向的定义也不相同, 因此需要根据具体的类型来进行位置和方向的确定。特征的位置定义为特征上 x 坐标最小的一点在轴类零件中心轴 (X 轴) 上的垂直投影点, 方向是指特征是沿 X 轴正向还是反向, 例如对于轴端倒角特征, 规定若圆锥面大端指向小端的方向为 X 轴正向, 则该轴端倒角的方向为正向, 否则该轴端倒角的方向为反向。对于大部分特征而言, 其位置都可以用包围盒遍历 `base_body_list` 和 `feature_list`, 每访问到一个基体和特征, 判断其类型:

特征的位置是指特征在中心轴正方向 (X 轴正方向) 上的坐标, 又因为在特征识别的过程中特征的参数是以在中心轴正方向上的位置顺序存储的, 因此在确定了特征的位置后可以完全地重建出该特征。特征的位置采用 AABB 包围盒的方法, 得到在 X 轴方向上的极小值点, 该点在 X 轴上的垂直投影即为特征的位置。

由于某些包含圆锥面的特征, 在特征识别算法中不属于特征的圆锥面类型的基面在实际中被认为是特征的一部分, 同时也使得在特征识别中包含了圆锥面的特征在实际中不需要这一圆锥面。故包围盒方法不适用于某些特征, 提取这些特征的位置需要各自单独计算。即可能会涉及到圆锥面的特征需要单独计算位置, 具体包括:

(1) 直杆形

遍历该基体对象包含的所有工程面, 每访问到一个工程面, 判断是否为圆柱面, 若为圆柱面, 则遍历其工程边列表以获取两端的圆弧边, 进而得到两个圆心 $O_1(x_1, 0, 0)$ 和 $O_2(x_2, 0, 0)$, 若 $x_1 < x_2$, 则以 O_1 作为该直杆形基体的位置, 否则以 O_2 作为该直杆形基体的位置。

(2) 轴端锥形（包含轴端倒角）

通过该特征的 *start_loop_index* 获取其种子环，得到该种子环中的圆弧边的圆心 $O_1(x_1, 0, 0)$ 。通过该种子环获取其基面，遍历基面中的工程边得到其中直径最大的圆弧边的圆心 $O_2(x_2, 0, 0)$ 。若 $x_1 < x_2$ ，则以 O_1 作为该轴端锥形的位置，否则以 O_2 作为该轴端锥形的位置。

(3) V 形槽和止动螺丝用槽

通过该特征的 *start_loop_index* 和 *end_loop_index* 获取其起始种子环和终止种子环，进一步得到两个种子环的圆锥面类型的基面，记这两个圆锥面的大径对应的圆心分别为 $O_1(x_1, 0, 0)$ 和 $O_2(x_2, 0, 0)$ ，若 $x_1 < x_2$ ，则以 O_1 作为该特征的位置，否则以 O_2 作为该特征的位置。

(4) 端面台阶

由于阶梯台阶等组合特征的存在，端面台阶的位置并不一定就是在轴类零件的两端上。遍历该特征对象包含的所有工程面，每访问到一个工程面，判断是否为圆柱面，若为圆柱面，则遍历其工程边列表以获取两端的圆弧边，进而得到两个圆心 $O_1(x_1, 0, 0)$ 和 $O_2(x_2, 0, 0)$ ，若 $x_1 < x_2$ ，则以 O_1 作为该直杆形基体的位置，否则以 O_2 作为该直杆形基体的位置。

(5) 锥形台阶和退刀槽

通过该特征的 *start_loop_index* 获取其种子环，进一步得到该种子环对应的圆锥类型基面（对于退刀槽特征而言，若基面类型不为圆锥类型，则以 *end_loop_index* 代替 *start_loop_index*），记该圆锥面的大径对应的圆心为 $O_1(x_1, 0, 0)$ 。遍历该特征包含的工程面集合，得到其中的圆柱面，进一步遍历该圆柱面包含的工程边，若其为不属于种子环的圆弧边，则记该圆环的圆心为 $O_2(x_2, 0, 0)$ 。若 $x_1 < x_2$ ，则以 O_1 作为该锥形台阶的位置，否则以 O_2 作为该锥形台阶的位置。

在得到一个特征的位置后，向该特征对象中动态添加一个 *position* 字段，并将其赋值为特征位置点的 x 坐标。

对于组合特征而言，其包含的每个单一特征都有自己的位置，该组合特征的位置是指其中层级最高的特征的位置（以如前所述的父特征和子特征为例，父特征的层级高于子特征），具体流程如下：

对于阶梯台阶、端面阶梯孔和外圆阶梯孔而言，其在特征组合时都是将层级最高的特征放在其 *feature_index_list* 列表的首位，因此组合特征的位置就是其内部 *feature_index_index* 中第一个元素对应的特征的位置，因其很容易获取，不再向组合特征类对象中添加 *position* 字段。

在提取特征的位置后，最终需要构建轴类零件的特征树，构建方法如 2.4.2 中所述。对于轴类零件而言，另外以特征和基体的位置建立一个升序排列的列表 *shaft_feature_list*，以用于加工过程分析。

4.2.3 轴类零件的价格计算

轴类零件的加工方式以车削为主，属于机加工零件。在不考虑销售、物流、场地等固定成本的前提下，机加工零件的成本由坯料成本和加工成本两部分构成。加工成本看作各个特征（包含基体）的加工成本之和，每个特征的加工成本又可以分解为人力成本、刀具损耗成本、机床损耗成本和机床占用成本，可以得到：

$$C_{\text{总}} = C_{\text{坯料}} + C_{\text{加工}} = C_{\text{坯料}} + \sum_i (C_{i\text{刀损}} + C_{i\text{机损}} + C_{i\text{机占}} + C_{i\text{人力}}) \quad (4-1)$$

在式（4-1）中： $C_{\text{总}}$ 代表零件总成本； $C_{\text{坯料}}$ 代表坯料成本； $C_{\text{加工}}$ 代表零件加工成本； C_i 代表第 i 个特征的加工成本； $C_{i\text{刀损}}$ 代表加工第 i 个特征的刀具损耗成本； $C_{i\text{机损}}$ 代表加工第 i 个特征的机床损耗成本； $C_{i\text{机占}}$ 代表加工第 i 个特征的机床占用成本； $C_{i\text{人力}}$ 代表加工第 i 个特征的人力成本。

其中，材料的价格和密度可以查到，坯料的体积可以根据轴类零件的尺寸和棒材规格得到，因此零件的成本计算主要就是计算加工成本。由于轴类的特征已经识别，每种特征都有其特定的加工方式，而企业自身拥有人力、刀具和机床等相关成本数据，因此可以根据零件的加工过程计算出其成本，但是这种方式需要的信息过多，计算过于繁琐。事实上，由于操作人员通常与机床是绑定的，而且机床的损耗也与机床的占用时间成正比，企业往往将人力、机床损耗和机床占用三者合并在一起，折算为机床工时成本，本章也采用这种方式进行成本的计算。

本章内容旨在设计并实现轴类零件的报价方法，并不以建立准确、完善的加工方式、机床型号、刀具类型、人力成本等成本数据库为目的，因此只是在数据库中增加少量数据以实现计算流程，这些数据并不一定符合生产实际，实际应用中企业可以根据自身的人力、生产资料等情况添加适合自己的相关成本数据库，具体包括：

（1）特征库

特征库不仅记录了特征（包括基体）的属性邻接图用于特征识别，还记录了特征的对应加工方式，如直杆形对应车削、键槽对应铣削等。

（2）机床库

机床库记录了每种机床的适用范围和单位时间成本（将占用成本、损耗成本和操作人员的工资全部折算到一起）。

(3) 刀具库

刀具库记录了每种刀具类型的适用加工方式、适用材料、额定寿命和价格等信息，用于选择刀具和计算刀具的损耗成本。

(4) 工艺库

工艺库记录了采用某种工艺加工某种材料的加工参数，主要是切削速度、进给量和背吃刀量，用于计算工时。

(5) 材料库

材料库记录了材料的价格、密度、性能等信息。

(6) 性能处理库

性能处理库记录了热处理、表面处理等工艺的价格和所能达到的性能。

根据轴类零件轴上的最大直径和轴线方向的长度可以得到零件所需的最小圆棒坯料尺寸，根据长径比查表可得坯料棒材的直径，进而得到径向加工余量，轴向的加工余量采用相同值，最终得到棒材坯料的尺寸，坯料成本根据坯料体积和材料价格即可得到。

加工成本可以采用工时法进行计算，其核心思想就是将各种繁琐复杂的成本计算统一折算为工时成本，本章也是采用这种思想，主要分为机床成本、刀具成本和对应的机床工时和刀具工时。机床成本包含了机床占用成本、机床损耗成本和操作人员的人工成本等，刀具成本主要为刀具损耗成本机床。机床工时包括机加工时间、刀具更换时间和机床更换时间，刀具工时主要为机加工时间。最终可以得到：

$$C_{\text{总}} = \rho V_{\text{坯料}} P_{\text{材料}} \sum_i \sum_j (P_{ij\text{机床}} (t_{ij\text{机换}} + t_{ij\text{机加}} + t_{ij\text{刀换}}) + P_{ij\text{刀具}} t_{ij\text{机加}}) \quad (4-2)$$

式(4-2)中， ij 表示第 i 个特征的第 j 道工序， $P_{\text{机床}}$ 表示机床库中记录的机床工时成本，机换表示更换机床、装夹工件等过程，机加表示真正机加工的过程，刀换表示更换刀具的过程。为了更加准确地进行报价，本课题对轴类零件的加工成本计算是基于模拟加工过程，而不是给每种特征设定一个加工成本，简单地累加各个特征的成本。

为了简化计算，将一次更换机床、装夹工件等辅助时间定为3min，一次换刀时间定为30s，不需要换刀的两个工序间隔时间定为10s。

具体而言，在系统的轴类零件报价中，按加工方式对特征树进行遍历。整体上将加工过程分为粗车、(半)精车、铣削和后续其他加工。在模拟加工的过程中，为特征树中每一个结点添加一个浮点型变量用于记录当前结点表示的特征或基体的加工程度，如外圆的当前直径；同时用一个枚举类型记录当前进行的工序(本

文假定同一类型的加工方式都可以在同一台机床上进行); 用一个整型变量记录工件夹持的外圆面对应的特征或基体; 用一个布尔类型记录夹持的方向。

首先根据零件尺寸得到毛坯直径, 下料、运送至第一道粗车工序的过程设为占用 5 分钟的车床。然后找到零件中直径最大的基体部分(直杆形), 进行第一次粗车, 选取模型靠近坐标原点的一侧夹持, 根据毛坯的材料、尺寸和对应的机床、刀具、加工参数等, 计算出该工序占用车床的时间和使用对应刀具的时间。然后遍历 *shaft_feature_list*, 完成其他外圆的粗车加工。以同样的方式进行外圆的半精车加工和精车加工。需要注意的是, 退刀槽的加工应该在粗车之后、半精车之前。

下一步对轴上的键槽、扳手槽、侧面孔等特征采用类似的方式进行加工过程的模拟。基本思想依然是尽量减少机床和刀具更换的次数。最后根据数据库中的相关价格信息计算需要的热处理、表面处理等工艺的成本。

在实际生产中, 特征之间的加工顺序有着更复杂的经验规则, 与系统模拟的加工过程很可能不同。但是在不大量增加机床更换次数和刀具更换次数的情况下, 这种加工顺序的不同对成本影响并不大。

4.3 一般零件的报价

对于一般零件而言, 其既不属于标准件, 不能查询规格以得到价格, 也不属于某类加工方式确定的零件, 其报价是最为困难的。由于形状相似的零件往往功能相近, 加工方式也类似, 其价格往往也比较接近, 同时随着 CAD 技术在制造业的全面普及, 海量的 CAD 模型中很可能存在与目标零件相似的零件, 如何找到这些相似的零件并进行重用, 是解决一般零件报价的关键。

一般零件的报价影响因素十分复杂, 由于没有特定的类别, 其报价准确率也难以保证, 本文对于该类零件的报价尽可能地提出一些方法使得报价有所依据, 完善整个系统。

4.3.1 基于特征树和形状描述算法的相似性检索

目前三维模型的检索方法大都是针对三角面片表示的三维模型提出的, 而不是 CAD 领域常用的 B-rep 模型。这些研究的重点在形状的相似度上, 而不关心拓扑结构的相似程度。在 CAD 领域, 基于图匹配的特征识别和相似性评估是研究最多、效果最好的方法, 但是一方面对零件的复杂程度非常敏感, 随着零件中面的数量增加, 算法的复杂度激增, 另一方面该方法不能反映几何形状的相似性。

形状描述算法是 Osada^[35]于 2002 年提出的一种基于形状分布曲线的统计直方

图算法，此方法将三维模型的形状描述为从测量三维模型的几何属性的形状函数中采样的概率分布。

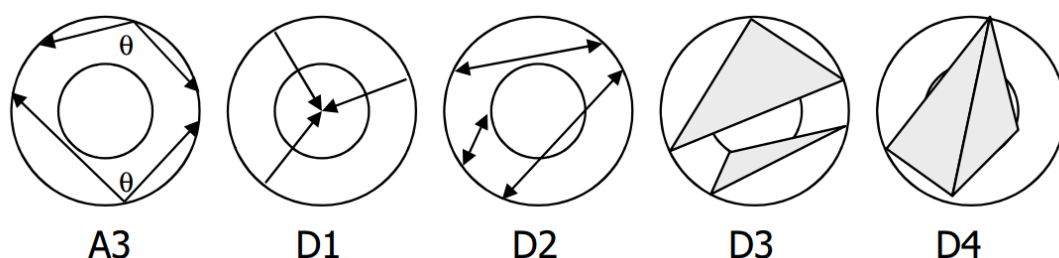


图 4-13 五种形状描述符^[35]

Fig.4-13 Five shape descriptors

如图 4-13 所示，Osada 提出了 5 种形状描述符：

A3：测量三维模型表面上三个随机点之间的角度；

D1：测量表面上固定点和一个随机点之间的距离，以模型边界的质心作为固定点；

D2：测量表面上两个随机点之间的距离；

D3：测量表面上三个随机点之间三角形区域的平方根；

D4：测量曲面上四个随机点之间四面体体积的立方根。

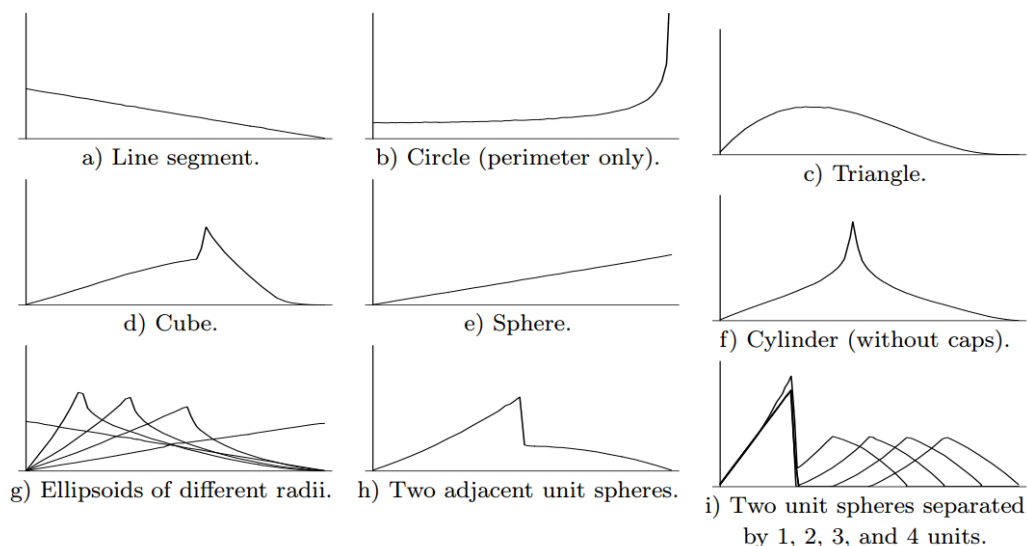


图 4-14 几种基本几何形状的 D2^[35]

Fig.4-14 D2 of several basic geometries

经验证，其中 D2 的效果最佳，图 4-14 是几种基本几何形状的 D2 描述符表示。Osada 还对包括巴氏距离等几种不同距离度量相似性地效果进行了研究。本文

使用巴氏距离衡量两个 D2 描述符的差异大小，并对其转换得到一个结果在 0 到 1 之间的相似度值。

对于两个离散概率分布 p 和 q 在同一个域 X ，定义巴氏距离为：

$$D_B(p, q) = -\ln(BC(p, q)) \quad (4-3)$$

其中：

$$BC(p, q) = \sum_{x \in X} \sqrt{p(x)q(x)} \quad (4-4)$$

本文对两个模型 D2 描述符的巴氏距离进行了如下转换，以得到一个范围在 0 到 1 之间的相似度值：

$$S = \frac{-1.118}{1 + e^{1.962 - 187.341 D_B}} + 1.118 \quad (4-5)$$

虽然 D2 描述符计算简单，对模型的旋转、平移、缩放具有不变性，能够很好地反映出模型的形状相似性，但是无法反映模型在拓扑结构上的相似情况。如图 4-15 所示，圆柱和长方体模型在无限细长的情况下将会退化为一根直线，两个在拓扑上完全不同的几何体此时的 D2 描述符却是非常相近的。

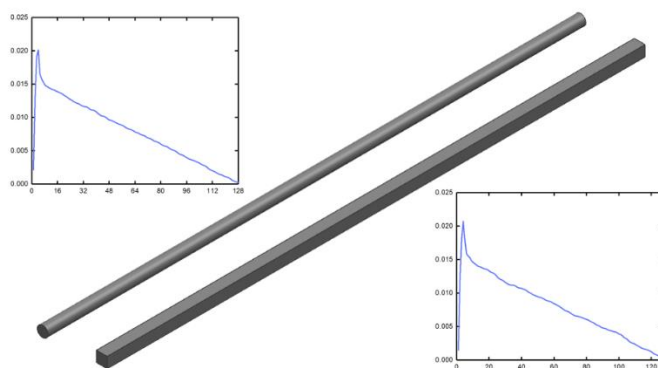


图 4-15 细长圆柱和细长长方体的 D2
Fig.4-15 D2 of slender cylinder and slender cuboid

李智^[70]等提出了基于层次结构的相似性评估方法，将拓扑结构图（面边图）与形状描述算法相结合，提出了一种以层次结构对拓扑细节进行描述的相似性比较方法，该方法保留了几何和拓扑相似性评估的优点，提高了效率和检索的准确度，取得了很好的效果。这种方法虽然通过对面边图结构进行分解以简化图匹配的复杂度，其整体上仍是基于递归和图匹配的思想，最坏情况下的时间复杂度仍然达到了 $O(n!n)$ ，其中 n 为层次结构中结点的数量。

本文用特征树的形式对实体模型的拓扑结构进行了分层表示，在此基础上提出了如下的方法进行相似度比较，既通过特征树的层次结构考虑了模型相似度分析中的拓扑结构和细节特征，又保证了算法的时间复杂度是线性的。具体方法如

下:

在经过特征识别构造出实体模型的特征树后, 根据该特征树按层进行 D2 相似度比较, 每层的权重为该层特征的总体积占有所有特征总体积的比例, 整体的相似度为两棵特征树每层相似度的按权重求和。特别的, 考虑到如上文所说, 将细长圆柱和细长长方体比较时, 虽然其拓扑结构不相同, 但是形状上仍然相似, 相似度不应该为 0。因此将特征树中虚拟的根结点对应的实体模型设为整个实体模型, 以描述整体形状的相似度。

对于两个模型 X 和 Y , 首先计算总的“特征体积” V_f :

$$V_f = V_x + V_y + \sum_i V_{xi} + \sum_j V_{yj} + \sum_k V_{xk} + \sum_l y_l \quad (4-6)$$

式 (4-6) 中, V_x 表示模型 X 的体积; V_y 表示模型 Y 的体积; V_{xi} 表示模型 X 中第 i 个基体的体积; V_{yj} 表示模型 Y 中第 j 个基体的体积; V_{xk} 表示模型 X 中第 k 个特征的体积; V_{yl} 表示模型 Y 中第 j 个基体的体积。

每一层的相似度的计算规则如下:

将第 n 层的特征分为 4 种类型, 即圆台类、圆孔类、非圆凸台和非圆孔槽, 分别对应 A、B、C、D, 则

$$S = \sum_n S_n = \sum_n \sum_{i=A}^{i=D} \frac{V_{ni}}{V_f} S_{ni} \quad (4-7)$$

式 (4-7) 中, S 表示模型 X 和 Y 的最终相似度; S_n 表示第 n 层的相似度加权后的值; S_{ni} 为两棵特征树中第 n 层中第 i 种类型的特征之间的相似度, 采用 D2 计算, 并转换为 0 到 1 之间的相似度; V_{ni} 表示两棵特征树中第 n 层中第 i 种类型的特征的总体积; V_f 为总的特征体积。

4.3.2 基于相似零件的价格计算

对于一个一般零件而言, 其价格除了取决于几何形状、尺寸等因素外, 也受到零件的体积、材料、性能要求等因素的影响。这些非形状因素对零件价格的影响是非常复杂的, 它们之间彼此耦合, 共同决定了一个零件的价格。

在众多报价方法当中, 最简单也最为直观的方法是重量法, 即根据零件的质量和材料的价格直接计算, 再用一定的系数进行修正, 得到零件的最终价格。虽然该报价方法结果并不十分准确, 但其思想具有普适性, 本章对于一般零件的报价也是基于这种思想。

一个最简单的、没有任何特征和性能要求的零件, 例如一个圆柱体, 由于其几乎不需要任何加工, 成本约等于材料的价格。不考虑物流、营销等生产管理成本, 一个零件的价格之所以会高于其等体积的材料价格, 主要有两个原因: 一是

零件加工的过程消耗了人力物力，增加了成本；二是在加工的过程中，由于机加工而导致零件体积减少，造成了材料的损失，使得最终的零件体积小于原始坯料。

对于一个不需要加工的零件而言，其成本可以认为是购买材料的费用，即 $C=\rho vp$ ，其中 C 为零件的成本， ρ 为材料密度， v 为坯料的体积， p 为材料单位质量的价格。随着特征的出现，零件需要进行加工和减材，成本增高。总体而言，特征数量越多、种类越多、占比越大，成本就越高，将特征对成本的影响记为特征系数 F ；同一形状但是材料不同的零件，它们之间的价格也不是简单地与材料价格成比例，这是因为不同材料的加工性能不同导致的，贵的材料可能加工性能很好，加工成本反而很低，由此对零件成本产生的影响记为材料系数 M ；表面精度、性能等要求，对零件成本的影响通常是独立的，例如热处理通常直接以零件体积或质量计费，而不考虑材料种类、加工特征等因素的影响，将性能要求对零件成本的影响记为性能系数 S ；体积成比例的零件，价格不一定成比例，一般而言，越小的零件加工难度越高，工序切换地越频繁，单位体积的成本可能更高，此外，对于体积十分巨大的零件，通常对坯料、运输、设备都有着特殊的要求，加工难度极高，成本也相应地增高，但这种零件实际中使用较少，本系统暂时不进行考虑，将体积对成本的影响记为体积系数 W 。

基于上述原因，并分析各个因素之间的耦合关系，提出以下估算零件成本的公式：

$$C = \rho v(pMFWS + S) \quad (4-8)$$

式 (4-8) 中： ρ 表示材料密度 ($\text{kg}\cdot\text{m}^{-3}$)， v 表示坯料的体积 (m^3)， p 表示材料的价格 ($\text{元}\cdot\text{kg}^{-1}$)， M 表示材料系数， F 表示特征系数， W 表示体积系数。

一般来说，对于形状复杂的小型件而言，不考虑表面处理等特殊要求，其加工成本与材料成本之比通常在 5~10:1 之间，该比值实际上取决于 F 、 W 和 M ，假定小型、复杂件的加工成本与材料成本之比为 8:1，进一步取 $F_{\max}=W_{\max}=M_{\max}=2$ ，即 $F, W, M \in [1, 2]$ 。

对于特征系数而言，将特征分为圆孔/槽、非圆孔/槽和凸台三种，分别记为 A, B, C。孔/槽对成本的影响，主要是通过直接去除特征部分的体积而产生的加工工时和减材；凸台对成本的影响，主要是通过去除凸台周围部分的体积而产生的加工工时和减材。给出 F 的表达式如下：

$$F = 1 + \frac{2n \sum_{i=A}^C Q_i R_i}{1 + 2n \sum_{i=A}^C Q_i R_i} \quad (4-9)$$

其中 n 为特征的总数量。

对于 $i=A$ ，即圆孔/槽特征：

$Q_A=0.7$, $R_A=\sum \frac{V_{Ai}}{V_{环}}$, V_{Ai} 表示第 i 个圆孔/槽特征的体积。

对于 $i=B$, 即非圆孔/槽特征:

$Q_B=0.9$, $R_B=\sum \frac{V_{Bi}}{V_{环}}$, V_{Bi} 表示第 i 个非圆孔/槽特征的体积。

对于 $i=C$, 即凸台:

$Q_C=0.8$, $R_C=\sum \frac{S_i H_i - \sum T_j V_{ij}}{V_{环}}$, S_i 表示第 i 个工程面的面积, H_i 表示第 i 个工程面

上最高的凸台的高度, V_{ij} 表示第 i 个工程面上第 j 个凸台体积, T_j 取决于凸台类型, 圆台的 $T_j=1$, 非圆凸台的 $T_j=1.2$ 。特别的, 当凸台特征既含有 `start_loop` 又含有 `end_loop` 时, 在逻辑上将该特征均分给两个环的基面。

对于材料系数 M , $M \in \{1, 1.25, 1.5, 1.75, 2\}$, 分别对应加工难度由易到难的不同材料。

对于体积系数 W , 令 $W = 1 + \frac{1}{1+V}$ 。其中, 规定 V 的单位为 cm^3 。

对于性能系数 S , S 与具体的处理类型, 通常按零件的质量或件数计费, 在数据库中以一条条记录的形式存储, 不具有统一的表达形式。

需要说明的是, 以上公式仅仅是为了反映各个因素对价格的定性影响而提出的, 是本文为使报价系统完整而提出的一个估算公式, 并不以准确地、定量地反映实际中特征和其他因素相互耦合对价格产生的影响为目标。

本系统对一般零件的报价算法具体是这样的: 设用户输入的需要报价的零件为 A 。利用 4.3.1 中所述的相似性比较算法, 得到零件库中与 P 的模型的相似度最高的零件 B , 记 A 与 B 的相似度为 Y , A 的价格为 P_A , B 的价格为 P_B 。根据式 (4-8) 计算得到的零件 A 的成本记为 C_A , 零件 A 的利润率记为 R_A 。记 A 根据自身信息得到的价格为 P_{AA} , A 根据 B 的相似性得到的价格为 P_{AB} , 可得

$$P_A = Y P_{AB} + (1 - Y) P_{AA} = Y P_{AB} + (1 - Y) C_A (1 + R_A) \quad (4-10)$$

这里做出 $R_A=20\%$ 的假设, 并将式 (4-3) 代入, 则有

$$P_A = \rho_A v_A p_A (M_A F_A W_A + S_A) \left(\frac{Y P_B}{\rho_B v_B p_B (M_B F_B W_B + S_B)} + 1.2 - 1.2Y \right) \quad (4-11)$$

最终采用式 (4-11) 估算零件 A 的价格。

4.4 验证与分析

4.4.1 轴类零件报价验证

驱动轴是最为常用的轴类零件之一, 起到传动、支撑和与其他零件配合等作用。本文以驱动轴为例, 对轴类零件的报价进行了验证。驱动轴材料为 45 钢, 不需要

任何热处理,其尺寸如图 4-16 所示。其中,键槽深 5mm,未标明的倒角为 C0.5mm,退刀槽宽 1.5mm 以下,深 0.3mm 以下即可,精度要求为:外圆和需要进行配合的两个轴肩端面为 $Ra1.6\mu m$,其余加工面精度要求为 $Ra6.3\mu m$,同轴度与垂直度不作要求。

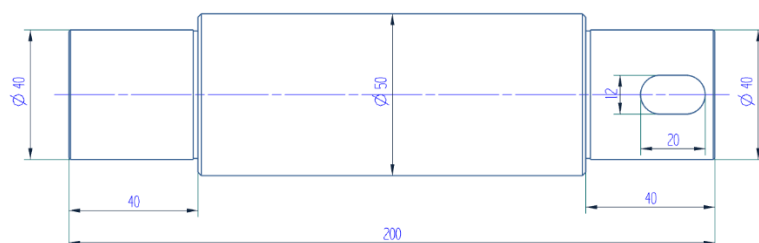


图 4-16 驱动轴尺寸
Fig.4-16 Dimension of the drive shaft

该轴上包含三个直杆形、两个退刀槽、四个倒角和一个键槽,故加工方式为车削和铣削。驱动轴的材料为 45 钢,具体尺寸和精度要求如图所示,对垂直度和同轴度等不作要求。加工过程中选取的相关成本和加工参数具体如表 4-1、表 4-2 和表 4-3 所示。

表 4-1 工时成本

Table 4-1 Cost per hour

	卧式车床	立式铣床	刀具
工时成本(元/小时)	20	20	10

表 4-2 车削参数

Table 4-2 Turning parameters

车削参数	粗车	半精车	精车
$a_p(\text{mm})$	一次全部切除	1	0.2
$v_c(\text{m/min})$	20	34	40
$f(\text{mm/r})$	0.6	0.12	0.12

表 4-2 中, a_p 代表背吃刀量; v_c 代表切削速度; f 代表进给量。

表 4-3 铣削参数

Table4-3 Milling parameters

铣削参数	粗铣	精铣
$a_p(\text{mm})$	2.5	0.1
$a_e(\text{mm})$	12	12
$a_f(\text{mm/z})$	0.025	0.02
$v(\text{m/min})$	25	35

表 4-3 中, a_p 代表铣削深度,即切削层沿铣刀轴线方向的尺寸; a_e 代表铣削宽度,即切削层垂直于铣刀轴线方向的尺寸; a_f 为每齿进给量,即铣刀每转一个齿,

铣刀和工件的相对位移，在键槽的铣削中铣刀齿数通常为 2； v 为铣削速度，代表铣刀旋转时，刀刃处相对工件的运动线速度。

通过对轴类特征的识别，结合数据库中的相关信息，计算出工时共计约 30 分钟，刀具实际工作时间约为 16 分钟，计算得到加工成本约为 12.7 元，材料成本（以直径 60mm，长 210mm 的棒材为坯料）约为 19 元，最终的零件成本约为 31.7 元。

零件的销售价格（不含税）除加工成本外还包括了质检、场地、物流、销售等其他成本。以企业利润占售价 10%，检验、物流、管理、销售、场地等费用占售价 10% 计算，最终报价 39.6 元。经过验证，该零件的市场价格为 40-45 元，报价误差在 1% 到 9.8% 之间，报价结果较为准确。

4.4.2 一般零件报价验证

图 4-17 中 a) 所示是一个 NC 加工用固定块组件，作为输入系统的待报价零件，记为 A 。其材料为 T10 碳素工具钢，无表面处理。系统对用户输入的这一零件采用 4.3.1 所述的相似性检索算法后，得到零件库中相似度最高的零件为图 4-17 中 b) 所示的高强钢高耐久型固定块组件，记为零件 B ，该零件的价格为 75 元，材料为 45 钢。



图 4-17 待报价零件与相似零件
Fig4-17 Target part and similar part

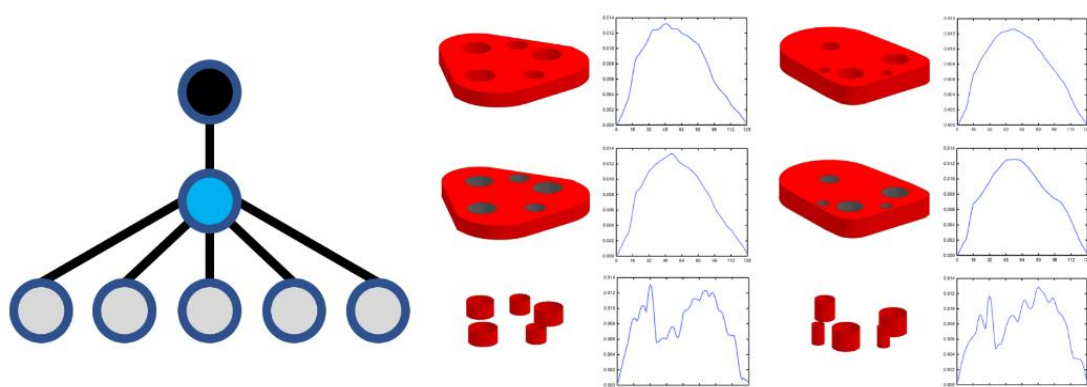


图 4-18 特征树结构与树中每层的相似度
Fig.4-18 Feature tree structure and similarity of each level

如图 4-18 所示,两个零件的特征树结构相同,计算得第一层的相似度为 94%,第二层的相似度为 92%,第三层的相似度为 79%,最终 A 和 B 的相似度为 91.9%。

对于零件 A , 计算得到: $v_A=7827\text{mm}^3$; $F_A=1.58$; $W_A=1.11$; 由于 T10 碳素钢属高碳钢, 机加工性能差, 取 $M_A=1.75$; 材料价格 $p_A=5.3$ 元/kg; 由于无表面处理, $S_A=0$ 。

对于零件 B , 计算得到: $v_B=34886\text{mm}^3$; $F_B=1.49$; $W_B=1.028$; 由于 45 钢为中碳钢, 机加工性能好, 取 $M_B=1.25$; 材料价格 $p_B=4.1$ 元/kg; 由于无表面处理, $S_B=0$ 。

代入式 (4-11) 计算得到零件 A 的报价为 32.1 元, 而零件 A 的实际价格为 31 元, 误差为 3.5%, 报价结果较为准确。

4.5 本章小结

本章介绍了报价系统对于系统标准件库中不存在的非标准件的报价策略。对于非标准件, 其报价分为两种方式。

(1) 零件虽然不是标准件, 没有规格可查, 但是零件属于某个明确的类别, 存在对应于该类别的报价算法。本系统针对轴类零件实现了这种报价策略。

轴类零件的加工方式固定, 容易进行加工过程的成本分析, 通过内置 12 种常见的轴类零件特征的拓扑定义和加工方式等信息, 实现了轴类零件指定加工特征的识别, 特征树的构建, 并通过建立加工特征加工信息库的方法, 实现了轴类零件的报价, 经验证结果较为准确。

(2) 对于不属于某个明确类别、没有针对性的报价算法的零件, 其报价最为困难, 这也是最一般的情况。这些零件没有固定的参数、确定的加工方式, 想要实现准确的报价几乎是不可能的, 本文尝试性地提出一种基于重量法的通用报价方式, 旨在在报价误差不太大的情况下使整个报价系统尽量完整, 可以覆盖一般零件。

本课题所提出的框架, 其核心思想是分而治之和可扩展性, 一方面企业根据自身的实际情况, 将工艺库、机床库和刀具库等进行更新, 另一方面当对某类零件的报价研究有了更好的算法时, 可以方便地植入该系统进行替换或扩充。因此课题本身提出的报价方法的细节和准确性并不是本课题的重点研究内容。

第五章 系统设计与实现

5.1 系统的架构

随着信息时代的到来，互联网技术飞速发展，越来越多的软件都由单机发展为联网，为了使用户能够实现数据的共享，并降低用户的使用成本，真正能够利用海量的 CAD 模型资源，本系统在设计之初就决定结合网络环境，采用 C/S 架构，将数据的存储与管理搭建于服务端，而将和用户交互的功能部署在客户端，整体的系统架构如图 5-1 所示。

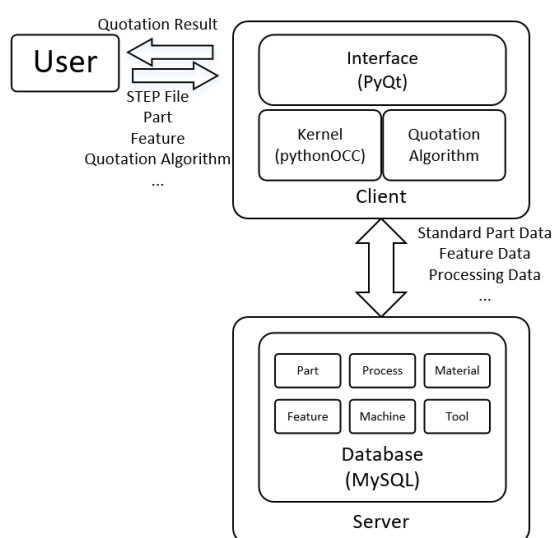


图 5-1 报价系统架构

Fig.5-1 Quotation system architecture

5.2 系统的服务端

报价系统的服务端主要负责数据的存储和管理，具体包括零件模型库直接以文件的形式存储在服务端硬盘中，材料库、工艺库、设备库、刀具库、标准件库等以数据库表的形式存储在服务端运行的数据库中。目前，本系统的服务端部署在云端，具体配置如下：

处理器核心数—单核；

内存—2GB；

硬盘—40GB；

网络带宽—1 兆；

操作系统—64 位 Windows Server 2012 操作系统。

5.2.1 数据库的选择

本系统中服务端的主要功能为数据的存储和管理，因此需要选择合适的数据库。数据库是报价系统的基础，包含了报价过程中几乎所有的信息，数据库的选型与设计是否合理，将在很大程度上影响报价系统的运行效率。

数据库分为关系型数据库和非关系型数据库。前者是以表结构对数据进行存储，其优点是格式一致、关系清晰，易于维护，使用 SQL^[71]（Structured Query Language，结构化查询语言）通用性好，支持复杂查询，但是表结构固定，灵活性差，高并发下读写效率低，主要用作持久存储；后者存储格式灵活，部署简单，读写效率高，但是不支持 SQL，不支持事务，主要用作缓存。

考虑到本系统的数据库以稳定地提供数据存储为主要目的，基本不会存在高并发的情况，故不使用非关系型数据库作为缓存，只采用关系型数据库作为持久化存储层。

目前市面上占有份额最大的三种关系型数据库分别为 Oracle、SQL Server 和 MySQL。根据报价系统的实际需求，对这三种数据库进行比较。

（1）Oracle 数据库

Oracle 数据库^[72]是美国甲骨文公司开发的关系型数据库，其功能强大，使用起来安全高速，并且十分稳定，并且由于其诞生早，金融、零售、能源、通信等各个行业的大型企业使用的数据库基本都是 Oracle。其主要缺点是价格十分昂贵，易用性不强。

（2）SQL Server 数据库

SQL Server^[73]是美国微软公司开发一款数据库管理软件，其具有使用简单方便、界面友好、与其他软件特别是微软自家的产品集成度高，可以非常容易地和 Access 或 Excel 中的数据进行相互转换。但是其不如 Oracle 稳定和功能强大。

（3）MySQL 数据库

MySQL^[74]是一款开源、免费、跨平台的关系型数据库，其性能出色，版本更新很快，并且简单易用。新版本的 MySQL 支持事务，支持 MyISAM、InnoDB 等多种存储引擎，目前越来越多地被使用在互联网等行业。

由于 MySQL 是一款免费（提供免费版本）、开源的关系型数据库，且其性能与易用性已经经受了市场的检验，本课题以 pythonOCC 作为几何造型内核的原因之一就是考虑到 OCC 是开源、免费的。因此，本系统选择同样是开源软件的 MySQL 作为数据库，具体版本为 MySQL5.7.28。

5.2.2 存储引擎的选择

MySQL 支持的最主要的三个存储引擎为 MyISAM、InnoDB 和 Memory，MyISAM 效率快但不支持行锁和事务，InnoDB 支持事务与行锁，Memory 是一个内存数据库引擎，综合数据库访问的效率、并发安全性来看，InnoDB 引擎支持事务，保障了数据库访问时的并发安全，同时其效率在报价的应用场景下完全足够，因此本系统的服务端数据库中所有的表均使用 InnoDB 作为存储引擎。

5.3 系统的客户端

报价系统的客户端主要使用 PyQt 框架实现系统界面，并以 pythonOCC 作为几何造型内核对实体模型文件进行解析与交互显示。目前报价系统的客户端暂时只支持 Windows 平台，本课题中的报价系统客户端运行环境等信息如下：

操作系统—64 位 Windows7 专业版；

处理器—英特尔 7 代 i3 四核处理器；

内存—16GB 物理内存；

python 版本—3.5.1；

pythonOCC 版本—0.18.2；

网络带宽—100 兆。

5.3.1 基于 PyQt 的界面开发

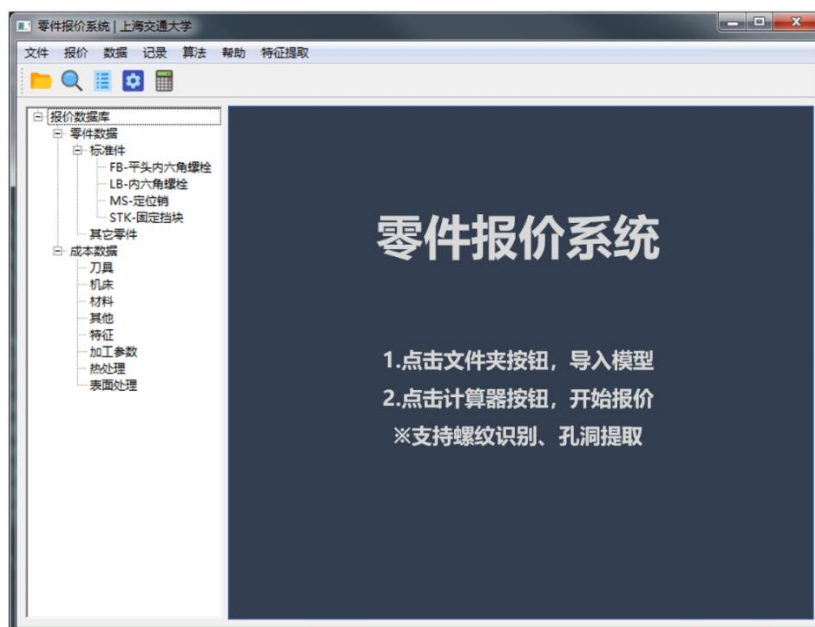


图 5-2 系统主界面

Fig.5-2 Main page of the system



图 5-3 标准件信息查看界面

Fig.5-3 Interface of viewing standard part information

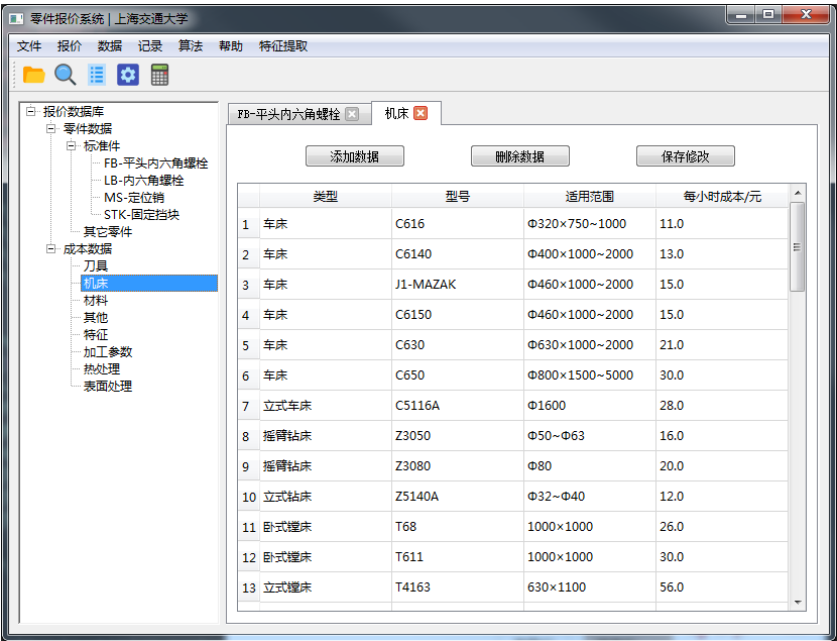


图 5-4 机床信息查看界面

Fig.5-4 Interface of viewing machine information

客户端负责与用户进行交互，必然少不了图形界面的开发，Qt^[75]是由 Qt Compony 于 1991 年开发而成的 C++跨平台图形界面开发框架。Qt 本身是开源免费的，并且遵循 LPGL 这一开源协议，因此使用 Qt 作为商用软件的界面是可行

的。

PyQt^[76]是由 Phil Thompson 开发的 python 库，是 python 语言和 Qt 框架的融合，使得开发人员能够使用 python 进行 Qt 界面的开发，本系统就是采用 PyQt 开发用户界面。

图 5-2 为系统主界面，右侧为主窗口用于显示模型、零件参数等信息。主界面对系统的基本使用进行了简要说明，点击左上角的文件夹按钮，导入.stp 模型文件，然后点击计算器按钮即开始进行报价。

图 5-3 和图 5-4 为系统查看平头内六角螺栓参数和机床信息的界面。系统中存储的报价相关的数据以表格的形式供用户查看。

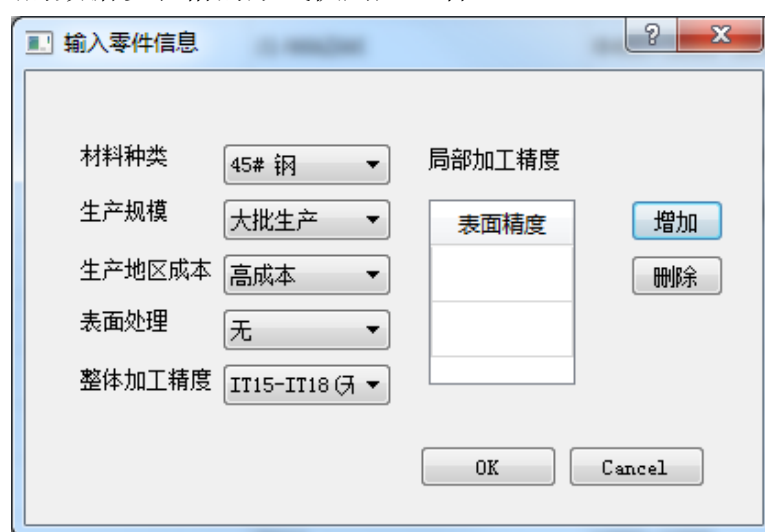


图 5-5 零件信息输入界面

Fig.5-5 Interface of inputting part information

图 5-5 为零件信息的输入界面，用户可以选择材料类型、零件数量规模、表面处理方式等零件信息，并结合模型显示以交互的方式，通过点选模型的表面，设置零件的加工精度。

5.3.2 基于 pythonOCC 的内核开发

pythonOCC 是 Thomas Paviot 开发的开源几何内核 OCC 的 python 封装版本，具有强大的三维造型能力，是实现实体模型显示与交互的不可缺少的组件之一。由于包之间存在依赖关系，手动搭建 pythonOCC 的运行环境较为麻烦，可以使用 Anaconda^[77]进行一键安装。Anaconda 是一个 python 包管理和环境管理的软件，利用内置的 conda 工具，可以快速安装各种包，具体命令如下：

```
conda create -n env_pythonocc -c pythonocc -c dlr-sc pythonocc-core==0.18.2
```

`python=3`

该命令在当前路径下创建一个名为 `env_pythonocc` 的虚拟环境并将 `pythonocc` 包及其所有依赖包安装到该环境中。

`pythonOCC` 中用到的主要类库已在第二章中进行介绍, 图为 `pythonOCC` 的模型显示画面内嵌入报价系统客户端后的效果。图 5-6 中为一个轴类零件模型。系统提供模型的旋转、缩放、平移、三视图、实体及线框表示等多种功能。

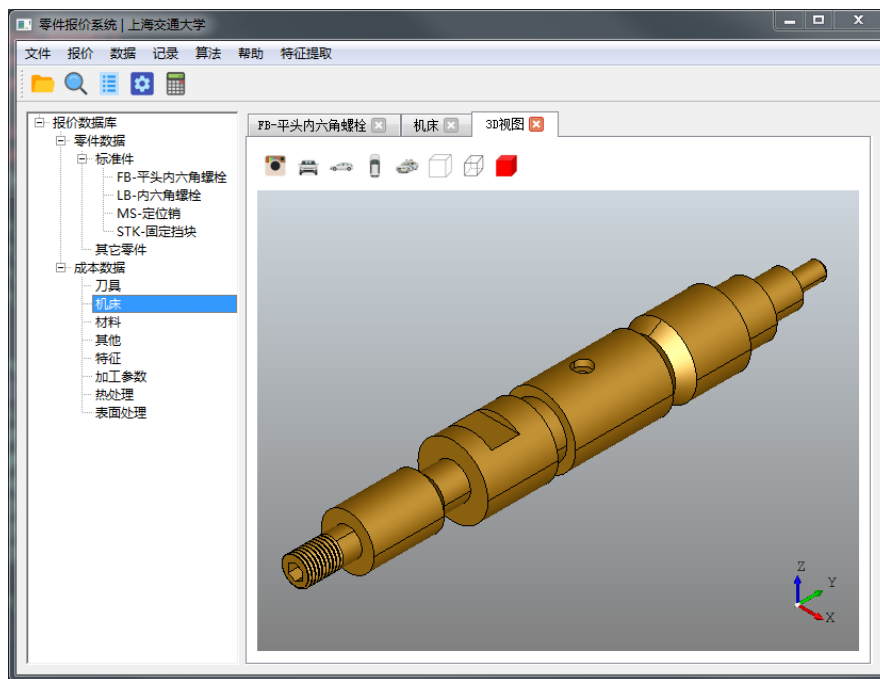


图 5-6 模型显示界面
Fig.5-6 Model display interface

5.3.3 数据库的连接与访问

客户端在进行报价的时候, 需要使用系统中已有的各种数据, 这些数据存储于服务端的数据库中, 若想获取这些数据, 必须进行数据库的连接与访问。

由于系统规模较小, 本系统中目前是直接使用 MySQL 的 `python` 驱动包 `pymysql` 进行数据库连接与访问, 而未使用 ORM (Object Relational Mapping, 对象关系映射) 框架。在对数据库进行访问时, 为了避免并发造成的不安全, 采用事务机制, 具体代码如下:

```
conn=pymysql.connect(...) # 连接数据库
cursor = c.cursor() # 获得游标对象
sql1="..." # SQL 语句1
```

```

sql2="..." # SQL 语句2
sql3="..." # SQL 语句3
...
try:
    cursor.execute(sql1) # 执行SQL 语句1
    cursor.execute(sql2) # 执行SQL 语句2
    cursor.execute(sql3) # 执行SQL 语句3
    ...
    conn.commit() # 如果执行成功就提交事务
except Exception as e:
    conn.rollback() # 如果执行失败就回滚事务
    raise e
finally:
    conn.close() # 关闭数据库连接

```

5.4 系统的扩展性分析

本课题设计的报价系统，并不以针对某类零件或是一般零件提出某种准确地报价方法为研究重点，而是以设计并实现一个可以通过良好地扩展性来实现通用性地报价的系统或者说是框架。因此系统的扩展性是否可用、易用，是该报价系统是否具有实际意义的决定因素。

具体而言，本系统的扩展性及其实现包括以下三个方面。

5.4.1 标准件类别的扩展性

标准件的报价本身并不困难，只要有足够的耐心和资料，人工地进行报价也能够十分准确。这就要求报价系统对标准件的报价除了准确外，还必须做到报价速度快和标准件类别丰富、齐全，并且随着类别的增加，检索的准确性不能降低。因此在添加标准件类别时，如何自动地生成新类别的准确的检索规则，同时又不影响现有类别的检索成为了系统设计的关键点之一。

如第三章所述，本文提出了一种利用 B-rep 基本属性组合以区分不同拓扑结构的思想，实现标准件的分类，进一步利用标准件参数规格化、离散化的条件，将规格参数映射为基本几何属性，实现了参数的提取。经过分析与试验，该算法不仅可以准确地进行标准件检索，同时实现了在添加新的标准件类别时能够自适应地计算出新类别的检索规则，且不对已有的标准件类别检索产生干扰。

5.4.2 报价算法的扩展性

Python 语言具有反射特性，即能够在运行时动态地通过类名、包名等字符串信息对类和包进行加载。利用这一特点，可以方便地实现报价算法地扩展性，具体是指用户可以自行地编写 python 文件放入指定路径的文件夹即可植入自己的报价算法，只需要遵循一定的规则，即该 python 文件中应该包含一个名为 quote 的方法，quote 方法的入参为 TopoDS_Shape 类型的实体模型，即待报价的零件模型，quote 方法的返回值为一个列表，列表首元素为布尔类型，表明该零件是否符合使用该报价算法的标准，若符合，则第二个元素为计算出的该零件的价格。该流程的代码如下：

```

if shape is not standard_part: # 若该零件经判断不属于标准件
    for algo_file in algo_folder: # 遍历报价算法目录
        algo = __import__(algo_file) # 动态导入算法包
        if algo.hasattr(quote): # 校验文件内是否含有报价方法
            res = algo.quote(shape) # 得到结果
            if res[0] is True: # 若零件属于该报价方法的适用范围
                price = res[1]
                return price # 返回零件的价格
# 与上面同理，对一般零件的报价算法用户也可以自行植入
for algo_for_general_part_file in algorithm_for_general_part_folder:
    algo_for_general_part = __import__(algo_for_general_part_file)
    if algo_for_general_part.hasattr(quote):
        res = algo_for_general_part.quote(shape)
        if res[0] is True:
            price = res[1]
            return price
return quote_for_general_part(shape)

```

这段代码同时也反映了整个报价系统的核心流程，分而治之的思想是建立在系统的可扩展性之上的，利用 python 语言的动态特性，系统实现了报价算法的可扩展性。

5.4.3 成本信息的扩展性

成本信息的扩展性是指如材料、机床、刀具等与零件成本相关的信息可以被添加和更新。这一点是许多报价系统都已经实现了的，也是极为重要的一点。报

价系统的开发者往往不是专业的报价人员或工程师，因此系统内置的材料、工艺参数等信息很可能不完善或者有错误，这就要求报价系统内的资料库必须是可以扩充和修改的。资料库的扩展比较容易实现，通过增加或修改数据库中的相应记录即可。

5.5 系统使用示例

报价系统的除报价功能外，还单独提供了孔洞识别、螺纹识别等功能。下面对几个主要功能进行演示。

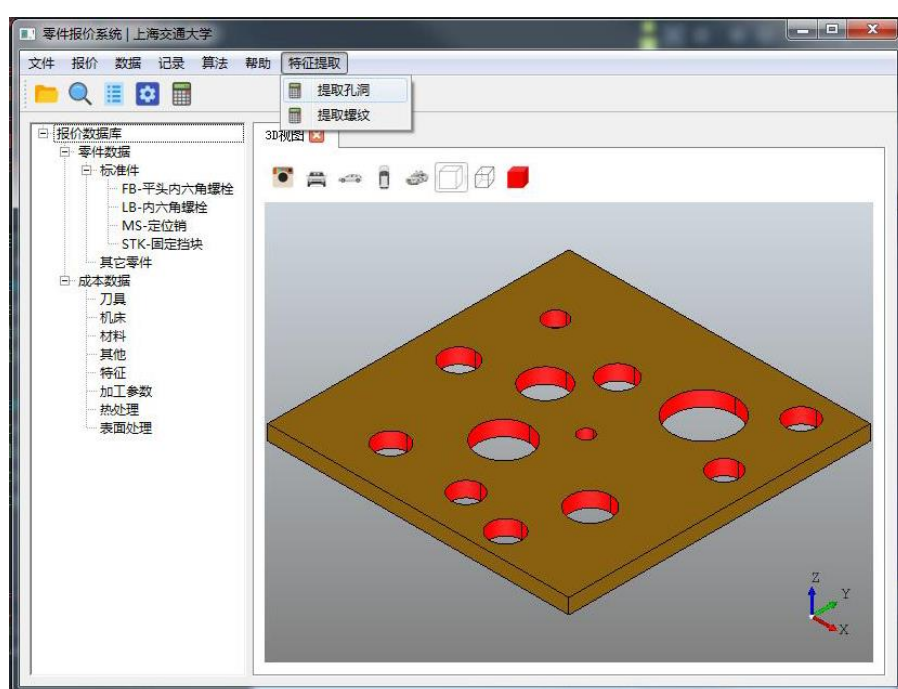


图 5-7 提取孔洞

Fig.5-7 Extraction of holes

(1) 孔洞提取

在导入零件模型后，点击菜单栏中特征提取下的提取孔洞，即可对零件中的通孔特征进行识别。在图 5-7 中，板上共有 13 个孔，全部识别并提取直径、深度等参数，以红色标识。

(2) 螺纹识别

螺纹识别是本文研究的重点之一，系统对外提供了螺纹特征识别与参数提取的功能。在导入带螺纹的模型后，点击菜单栏中特征提取下的提取螺纹，即可对模型中的螺纹特征进行识别。在图 5-8 中，平面上共有 4 个外螺纹特征，系统全

部识别，以红色标识，用点标记出螺纹线。如图 5-9 所示，系统将全部螺纹的参数提取成功。

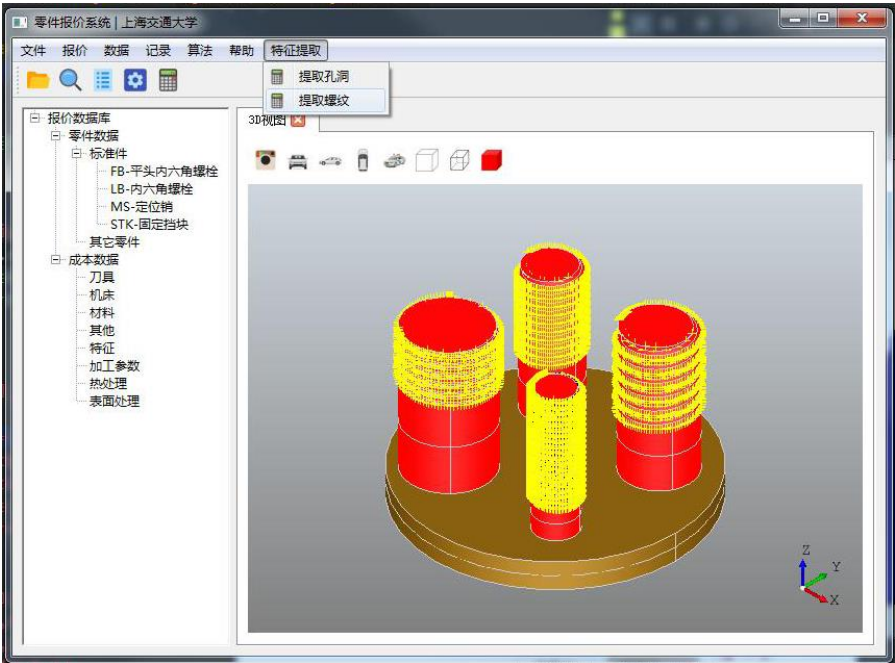


图 5-8 螺纹识别示例
Fig.5-8 Sample of thread recognition

```
This is a left-hand thread. Thread line number is 1. Outer diameter is 29.998932. Inner diameter is 28.000000. Pitch is 3.499843
This is a right-hand thread. Thread line number is 1. Outer diameter is 19.998256. Inner diameter is 15.000000. Pitch is 3.001378
This is a right-hand thread. Thread line number is 1. Outer diameter is 25.000000. Inner diameter is 20.000000. Pitch is 4.992838
This is a left-hand thread. Thread line number is 1. Outer diameter is 15.000000. Inner diameter is 13.000000. Pitch is 2.007690
```

图 5-9 螺纹参数提取结果
Fig.5-9 Result of thread parameter extraction

(3) 零件报价：

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P
1	报价id	是否为标准件	标准件类别	是否为轴类	相似零件id	相似度	报价方式	价格(1-99)	价格(100-200)	M	L	I	P	A	E	B
2	20191216110356	是	LB-内六角螺栓	/	/	/	标准件	1.73	1.57	8	20	20	1.25	13	8	6

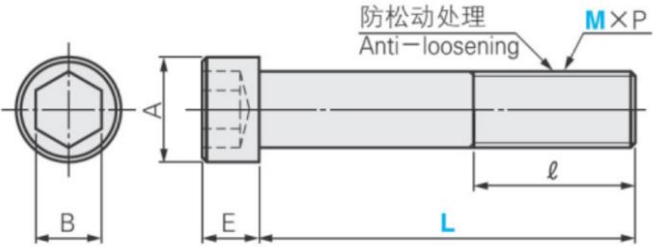
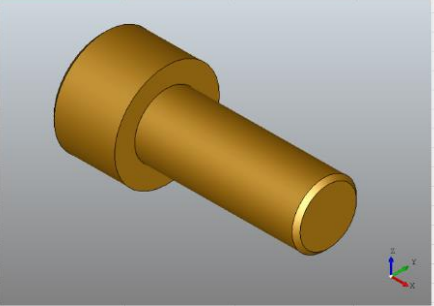


图 5-10 内六角螺栓的报价结果
Fig.5-10 Quotation result of hexagon socket bolt

以一个内六角螺栓的报价为例进行说明。导入内六角螺钉模型，如图 5-11 所

示,对材料及其他性能不作要求,可以直接点击报价按钮。系统以 Excel 表格的形式输出报价结果,如图 5-10 所示。

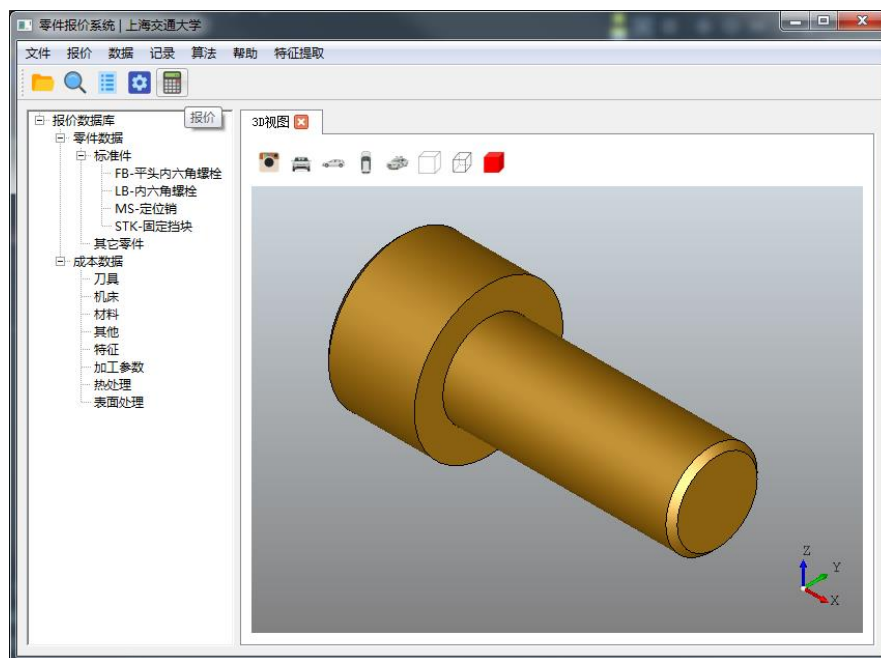


图 5-11 内六角螺栓的报价
Fig.5-11 Quotation for hexagon socket bolt

5.6 本章小结

本章介绍了本文提出的报价系统的具体实现,包括系统的架构、服务端与客户端的运行环境、数据库的选择、表结构的设计、客户端开发依赖的类库以及具体的界面操作流程。除此之外,对系统的扩展性进行了分析,该报价系统具有良好的扩展性,不仅可以通过写数据库实现资料库中材料、工艺等信息的添加与修改,同时能够实现标准件类别的自适应添加,而且除此之外,系统最为核心的报价算法的扩展性经验证可以利用 python 语言的动态特性十分方便地实现,即用户或研究人员可以通过在指定路径添加实现了某种报价算法 python 脚本,就可以将这个算法植入系统,系统在运行时会动态地检测是否有植入的新算法,并对其进行执行。

第六章 总结与展望

6.1 全文总结

本文围绕机械零件的报价展开了研究，设计并实现了一种通用的报价系统，该系统采用分而治之的思想，将零件分为标准件、存在对应报价算法的零件和一般零件，分别采用针对性的报价策略，以尽可能地得到合理的报价结果。本文主要的工作和研究成果如下：

(1) 设计了一个以 B-rep 模型为输入的通用报价系统的框架，采用分而治之的思想，将零件分为标准件、存在对应报价算法的零件和一般零件，分别采用合适的报价算法进行报价。该系统具有良好的扩展性，新的类别和对应的报价算法可以方便地植入到系统中。

(2) 利用开源几何造型内核 OCC 的 python 版本 pythonOCC，实现 B-rep 模型的特征识别。通过预处理、提取种子环等步骤识别特征，并利用包围盒和凹边判定等方法得到特征的基本属性。进一步地，采用属性邻接图匹配的方式对指定类型的特征进行特征识别，并设计了一种特征自适应添加的规则，用户通过上传指定形式的 B-rep 模型和系统规定的脚本文件，就能够方便地添加新的需要进行识别的特征类型。

(3) 设计并实现了一种标准件的检索算法。该算法利用 B-rep 基本拓扑的组合对不同类别的标准件进行区分，并利用标准件规格参数是离散的这一特点，将不同类型标准件的规格参数映射为统一的基本几何属性，从而实现了规格参数的识别。经验证，该方法具备应用于工程实际的能力。

(4) 对存在对应报价算法的零件，给出了一种具体实现——轴类零件。系统通过回转体的定义和长径比等信息对零件是否为轴类零件进行判断，并以属性邻接图匹配的方式，识别其上具体十几种加工特征，并根据轴数据库中的相关信息，对轴类加工过程进行分析，根据工时求得轴类零件的成本。经验证，该方法准确率较高。

(5) 对于一般零件，尝试性地提出一个包含材料、特征、体积、表面精度等因素的价格估算公式，并结合相似性检索得到一般零件的报价，使得整个报价系统完整。经验证，该方法具有一定的可行性。

(6) 利用 pythonOCC 和 PyQt 开发了系统的客户端，在云服务器上建立了系统的服务端用于存储数据，实现了 C/S 架构的报价系统的完整开发。

6.2 研究展望

产品报价系统的研究具有非常重要的研究价值与工程意义，可能涉及工程管理学、特征识别、相似性评估、机器学习和深度学习等诸多领域。本文虽然设计了并实现了一种以 B-rep 模型作为输入的通用报价系统框架，但并未对各种具体的报价算法做深入研究，其内置的报价算法都是建立在种种约束条件之上提出的较为简单的算法，报价系统的工程性也需要改进。具体而言，在以下几个方面需要进一步研究与探索：

(1) 本文所研究的特征识别的对象是一些比较简单的基本特征，以单个面的内环为种子环，不能识别生长在多个面的特征和相交特征。另外所研究的特征不包含自由曲线、曲面。下一步将优化算法以实现适用范围更广的特征识别。

(2) 本系统中标准件检索算法的分类过程是以拓扑信息的组合为规则的，具体选择哪些是这种算法能否对海量标准件类别中进行分类的关键。下一步将研究如何扩充和优化标准件检索算法中的编码规则，以提高分类能力。此外，标准件检索算法中的参数提取是以标准件规格参数是离散的为基础的，然而实际中部分标准件规格参数是连续的。下一步将研究利用机器学习等技术对包含连续参数的标准件的规格参数进行提取。

(3) 为了实现新添加的特征类型的参数提取，本文设计了一种简单的脚本，但该脚本所能表示的提取规则十分有限，只能描述一些简单的提取规则。下一步将研究如何优化该脚本，以表示更复杂的参数提取规则。

(4) 本文对存在对应报价算法的零件给出了一个具体实例——轴类零件。通过提取轴上的加工特征，利用数据库中的相关信息，进行加工过程的成本分析。下一步将完善数据库中的相关信息，使之更加准确，符合生产实际，同时去对存在对应报价算法的零件类别进行扩充，添加如箱体等新的类型。

(5) 对于一般零件的报价算法，本文提出了一个粗略的计算公式，该公式虽然考虑了材料、特征、表面精度等因素的耦合影响，但未考虑特征的方向、具体加工方式差异等诸多细节因素的影响，报价结果的准确性有待进一步验证。下一步将搜集更多模型数据，深入研究各个因素对价格影响的耦合作用，提出更加准确、具有实际意义一般零件报价算法。

(6) 本系统由于尚未被商业使用，其工程实用性考虑不够周全。下一步将完善系统的工程实用性，如增加用户身份校验机制、开发服务端程序以避免客户端直接与数据库交互、客户端程序打包安装，并将计算任务由客户端转移至服务端使客户端更轻量化等。

参考文献

- [1] 刘尚希, 石英华, 王志刚, 等. 2019 年上半年宏观形势分析报告[J]. 财政科学, 2019 (8): 3.
- [2] Niazi A, Dai J S, Balabani S, et al. Product cost estimation: Technique classification and methodology review[J]. 2005.
- [3] 陈国斌, 吴清烈. 大规模定制中延迟设计产品的一种快速成本估算方法[J]. 现代管理科学, 2005 (8): 89-90.
- [4] 张伯鹏. 数字化制造是先进制造技术的核心技术[J]. 制造业自动化, 2000, 22(2): 1-5.
- [5] Bézier P. Numerical control: mathematics and applications[J]. 1970.
- [6] Forrest A R. Interactive interpolation and approximation by Bézier polynomials[J]. The Computer Journal, 1972, 15(1): 71-79.
- [7] Regli W C, Cicirello V A. Managing digital libraries for computer-aided design[J]. Computer-Aided Design, 2000, 32(2): 119-132.
- [8] Ong N S. Manufacturing cost estimation for PCB assembly: An activity-based approach[J]. International Journal of Production Economics, 1995, 38(2-3): 159-172.
- [9] 单汨源, 於永和. 大规模定制产品多级神经网络成本估算方法研究[J]. 中国机械工程, 2004, 15(11): 1004-1007.
- [10] 苏少辉, 纪杨建, 祁国宁, 等. 大批量定制环境下面向订单设计产品的制造成本估算方法研究[D]., 2007.
- [11] 魏秋红, 夏榆滨. 基于 PLM 的快速报价系统的设计与实现[J]. 计算机与数字工程, 2005, 33(4): 1-3.
- [12] 李杨, 刘长安, 吴寿喜, 等. PDM 环境下基于 Web 的产品报价系统[J]. 现代制造工程, 2005 (8): 45-48.
- [13] 邢宪才, 刘渝, 王安兵. 基于 BP 神经网络的覆盖件模具机加工成本估计[J]. 机械与电子, 2008, 2008(5): 61-64.
- [14] 熊立华, 王云峰. 基于实例库推理的快速成本估算方法研究[J]. 计算机集成制造系统, 2004, 10(12): 1605-1609.
- [15] 张磊, 孙玲, 辛勇. 基于 Pro/TOOLKIT 异步模式特征识别方法[J]. 现代塑料加工应用, 2008, 19(5): 51-54.
- [16] 练国富, 吕盼粮, 竺长安, 等. 基于快速设计的锻压机床材料成本快速报价系

统研究[J]. 制造技术与机床, 2010 (4): 130-133.

[17] 董玉德, 米登斌, 陈明龙, 等. 基于三维几何特征的产品报价方法[J]. 中国机械工程, 2017, 28(22): 2738-2746.

[18] Zhou X, Qiu Y, Hua G, et al. A feasible approach to the integration of CAD and CAPP[J]. Computer-Aided Design, 2007, 39(4): 324-338.

[19] 潘翔, 张三元, 叶修梓. 三维模型语义检索研究进展[J]. 计算机学报, 2009, 32(6): 1069-1079.

[20] 杨育彬, 林琿, 朱庆. 基于内容的三维模型检索综述[J]. 计算机学报, 2004, 27(10): 1297-1310.

[21] Osada R, Funkhouser T, Chazelle B, et al. Shape distributions[J]. ACM Transactions on Graphics (TOG), 2002, 21(4): 807-832.

[22] Ip C Y, Lapadat D, Sieger L, et al. Using shape distributions to compare solid models[C]//Proceedings of the seventh ACM symposium on Solid modeling and applications. ACM, 2002: 273-280.

[23] Rea H J, Sung R, Corney J R, et al. Interpreting three-dimensional shape distributions[J]. Proceedings of the Institution of Mechanical Engineers, Part C: Journal of Mechanical Engineering Science, 2005, 219(6): 553-566.

[24] Zou K S, Ip W H, Wu C H, et al. A novel 3D model retrieval approach using combined shape distribution[J]. Multimedia tools and applications, 2014, 69(3): 799-818.

[25] Cordella L P, Foggia P, Sansone C, et al. A (sub) graph isomorphism algorithm for matching large graphs[J]. IEEE transactions on pattern analysis and machine intelligence, 2004, 26(10): 1367-1372.

[26] McWherter D, Peabody M, Regli W C, et al. Transformation invariant shape similarity comparison of solid models[C]//Proceedings of the DETC '01 ASME Design Engineering Technical Conferences, Pittsburgh, PA, Paper No. DETC. 2001.

[27] McWherter D, Peabody M, Shokoufandeh A C, et al. Database techniques for archival of solid models[C]//Proceedings of the sixth ACM symposium on Solid modeling and applications. ACM, 2001: 78-87.

[28] El-Mehalawi M, Miller R A. A database system of mechanical components based on geometric and topological similarity. Part I: representation[J]. Computer-Aided Design, 2003, 35(1): 83-94.

[29] El-Mehalawi M, Miller R A. A database system of mechanical components based on geometric and topological similarity. Part II: indexing, retrieval, matching, and similarity assessment[J]. Computer-Aided Design, 2003, 35(1): 95-105.

[30] 马露杰, 黄正东, 吴青松. 基于面形位编码的 CAD 模型检索[J]. 计算机辅

助设计与图形学学报, 2008, 20(1): 19-25.

[31] Iyer N, Kalyanaraman Y, Lou K, et al. A reconfigurable 3D engineering shape search system. Part I: shape representation[C]//ASME DETC. 2003, 3.

[32] Lou K, Jayanti S, Iyer N, et al. A reconfigurable 3D engineering shape search system. Part II: database indexing, retrieval, and clustering[C]//ASME 2003 International Design Engineering Technical Conferences and Computers and Information in Engineering Conference. American Society of Mechanical Engineers Digital Collection, 2003: 169-178.

[33] Gao W, Gao S M, Liu Y S, et al. Multiresolutional similarity assessment and retrieval of solid models based on DBMS[J]. Computer-Aided Design, 2006, 38(9): 985-1001.

[34] 白静, 唐韦华, 刘玉生, 等. 面向实体模型相似评价的层次图生成与高效匹配[J]. 计算机辅助设计与图形学学报, 2009 (7): 869-879.

[35] Hilaga M, Shinagawa Y, Kohmura T, et al. Topology matching for fully automatic similarity estimation of 3D shapes[C]//Proceedings of the 28th annual conference on Computer graphics and interactive techniques. ACM, 2001: 203-212.

[36] Bespalov D, Regli W C, Shokoufandeh A. Reeb graph based shape retrieval for CAD[C]//ASME 2003 international design engineering technical conferences and computers and information in engineering conference. American Society of Mechanical Engineers Digital Collection, 2003: 229-238.

[37] Elinson A, Nau D S, Regli W C. Feature-based similarity assessment of solid models[C]//Proceedings of the fourth ACM symposium on Solid modeling and applications. ACM, 1997: 297-310.

[38] Cicirello V, Regli W C. Machining feature-based comparisons of mechanical parts[C]//Proceedings International Conference on Shape Modeling and Applications. IEEE, 2001: 176-185.

[39] Bespalov D, Shokoufandeh A, Regli W C, et al. Scale-space representation of 3D models and topological matching[C]//Proceedings of the eighth ACM symposium on Solid modeling and applications. ACM, 2003: 208-215.

[40] Bespalov D, Regli W C, Shokoufandeh A. Local feature extraction and matching partial objects[J]. Computer-Aided Design, 2006, 38(9): 1020-1037.

[41] Hong T, Lee K, Kim S. Similarity comparison of mechanical parts to reuse existing designs[J]. Computer-Aided Design, 2006, 38(9): 973-984.

[42] Chu C H, Hsu Y C. Similarity assessment of 3D mechanical components for design reuse[J]. Robotics and Computer-Integrated Manufacturing, 2006, 22(4): 332-341.

[43] Cheng H C, Chu C H, Wang E, et al. 3D part similarity comparison based on levels of detail in negative feature decomposition using artificial neural network[J]. Computer-

Aided Design and Applications, 2007, 4(5): 619-628.

[44] Li M, Zhang Y F, Fuh J Y H, et al. Toward effective mechanical design reuse: CAD model retrieval based on general and partial shapes[J]. Journal of Mechanical Design, 2009, 131(12): 124501.

[45] Li M, Zhang Y F, Fuh J Y H. Retrieving reusable 3D CAD models using knowledge-driven dependency graph partitioning[J]. Computer-Aided Design and Applications, 2010, 7(3): 417-430.

[46] Bai J, Gao S, Tang W, et al. Design reuse oriented partial retrieval of CAD models[J]. Computer-Aided Design, 2010, 42(12): 1069-1084.

[47] Zhang R, Zhou X. Similarity assessment of mechanical parts based on integrated product information model[J]. Journal of Computing and Information Science in Engineering, 2011, 11(1): 011001.

[48] Grayer A R. The automatic production of machined components starting from a stored geometric description[J]. Advances in Computer Aided Manufacture, 1977, 137.

[49] Kyprianou L K. Shape classification in computer-aided design[D]. University of Cambridge, 1980.

[50] 高曙明. 自动特征识别技术综述[J]. 计算机学报, 1998, 21(3): 281-288.

[51] Joshi S, Chang T C. Graph-based heuristics for recognition of machined features from a 3D solid model[J]. Computer-Aided Design, 1988, 20(2): 58-66.

[52] Marefat M, Kashyap R L. Geometric reasoning for recognition of three-dimensional object features[J]. IEEE Transactions on Pattern Analysis and Machine Intelligence, 1990, 12(10): 949-965.

[53] Vandenbrande J H, Requicha A A G. Spatial reasoning for the automatic recognition of machinable features in solid models[J]. IEEE Transactions on Pattern Analysis and Machine Intelligence, 1993, 15(12): 1269-1285.

[54] Gao S, Shah J J. Automatic recognition of interacting machining features based on minimal condition subgraph[J]. Computer-Aided Design, 1998, 30(9): 727-739.

[55] Woo T C. Feature extraction by volume decomposition[C]//Proc. Conf. CAD/CAM Tech. Mech. Eng. 1982, 76.

[56] Armstrong G T. A STUDY OF AUTOMATIC GENERATION OF NONINVASIVE NC MACHINE PATHS FROM GEOMETRIC MODELS[J]. 1990.

[57] Sakurai H. Volume decomposition and feature recognition. Part I: polyhedral objects[J]. Computer-Aided Design, 1995, 27(11): 833-843.

[58] Sakurai H, Dave P. Volume decomposition and feature recognition. Part II: curved objects[J]. Computer-Aided Design, 1996, 28(6-7): 519-537.

[59] Shah J J, Shen Y, Shirur A. Determination of machining volumes from extensible

- sets of design features[M]//Manufacturing Research and Technology. Elsevier, 1994, 20: 129-157.
- [60] Opencascade[EB/OL]. <https://www.opencascade.com>. [Online; access 15-Dec-2019].
- [61] PythonOCC[EB/OL]. <http://www.pythonocc.org>. [Online; access 15-Dec-2019].
- [62] Seltes J W. A feature-based representation of parts for CAD[J]. America: ME Department, MIT, 1978.
- [63] Leiserson C E, Rivest R L, Cormen T H, et al. Introduction to algorithms[M]. Cambridge, MA: MIT press, 2001.
- [64] Ullmann J R. An algorithm for subgraph isomorphism[J]. Journal of the ACM (JACM), 1976, 23(1): 31-42.
- [65] Schmidt D C, Druffel L E. A fast backtracking algorithm to test directed graphs for isomorphism using distance matrices[J]. Journal of the ACM (JACM), 1976, 23(3): 433-445.
- [66] McKay B D. Practical graph isomorphism[M]. Tennessee, USA: Department of Computer Science, Vanderbilt University, 1981.
- [67] Cordella L P, Foggia P, Sansone C, et al. Performance evaluation of the VF graph matching algorithm[C]//Proceedings 10th International Conference on Image Analysis and Processing. IEEE, 1999: 1172-1177.
- [68] Cordella L P, Foggia P, Sansone C, et al. An improved algorithm for matching large graphs[C]//3rd IAPR-TC15 workshop on graph-based representations in pattern recognition. 2001: 149-159.
- [69] NumPy[EB/OL]. <https://numpy.org>. [Online; access 15-Dec-2019].
- [70] Li Z, Zhou X, Liu W. A geometric reasoning approach to hierarchical representation for B-rep model retrieval[J]. Computer-Aided Design, 2015, 62: 190-202.
- [71] Date C J, Darwen H. A Guide to the SQL Standard[M]. New York: Addison-Wesley, 1987.
- [72] Oracle[EB/OL]. <https://www.oracle.com>. [Online; access 15-Dec-2019].
- [73] SQL Server[EB/OL]. <https://www.microsoft.com/zh-cn/sql-server>. [Online; access 15-Dec-2019].
- [74] MySQL[EB/OL]. <https://www.mysql.com>. [Online; access 15-Dec-2019].
- [75] Qt[EB/OL]. <https://www.qt.io>. [Online; access 15-Dec-2019].
- [76] PyQt[EB/OL]. <https://wiki.python.org/moin/PyQt>. [Online; access 15-Dec-2019].
- [77] Anaconda[EB/OL]. <https://www.anaconda.com>. [Online; access 15-Dec-2019].

致谢

本文是在我的导师柳伟老师的悉心指导下完成的。从论文的选题、技术方案的设计、开发难点的攻克到论文的最终完成，每一步都饱含着柳老师的心血和汗水。柳老师严谨的治学态度、开阔的学术思维和踏实的工作作风使我受益良多。今后的工作生活中，我也将谨记柳老师的教诲，以柳老师为学习榜样，努力不负老师期望！

感谢周雄辉教授，周教授不仅为我们提供了非常好的科研环境和非常有价值的项目课题，并且在生活上、心理上都对我们关怀入微，让我们在研究生期间能够安心地学习知识技能，收获颇丰。在此向周老师表示最深的敬意和最诚挚的感谢！

感谢塑性院的赵震教授、陈军教授、崔振山教授、董湘怀教授、谢叻教授、彭雄奇教授、庄新村教授、韩先洪教授、于沪平副教授等诸位老师对我们学习上的关心与教导。感谢顾瑾老师、李琦老师、胡成亮班主任、刘伟老师、熊阿姨对我们生活上的关怀与照顾。感谢模具 CAD 国家工程研究中心主任、中国工程院院士阮雪榆教授，是您带领塑性院发展壮大，给我们提供了优越的科研条件，您永远活在每一个塑性学子的心中。

感谢课题组的所有人。感谢牛强老师、孔垂品博士，两位渊博的学识为我打开了计算机世界的大门。感谢李俊杰、冯青青、马恒源、朱振威博士在学术与生活方面为我答疑解惑。感谢卢一帆、曾定邦、邵康、李培建师兄，向我们无私传授找工作的经验。感谢徐涛涛、刘玲卓、张丽萍同学，两年多来大家相互学习、共同进步，希望在今后的工作生活中，能够保持联系、经常交流。感谢各位师弟师妹，你们对我的帮助同样重要。

感谢 238 的小伙伴们，能够与你们相遇实在是难得的缘分，在 238 不到两年的时间里，我们一起科研，一起玩耍，一起说走就走去吃海底捞。因为你们的存在，办公室的氛围总是轻松而愉快。非常幸运能和杨志云师兄、朱华佳同学、陈骥同学以及各位师兄师弟师妹们在 238 共同学习、生活，大家都是非常真挚而善良的人，希望我们都能永保初心。

感谢陈飞老师，无私地为我提供科研场所，让我在 238 进行了两年的学习与科研。在与陈老师不多的接触中，陈老师谦逊的为人、求实的治学态度、开阔的思维方式和对学生深切的关心都给我留下了深刻的印象。

感谢舍友李远、徐帅、杨逢春同学和前舍友常志东博士，从山大到交大，大家结下了深厚的友谊。

感谢塑性院的所有同学，能够和如此优秀的大家一起生活学习，是我一生的美好回忆。

感谢我的父母，这么多年始终支持我的每一个决定，从未给我额外的压力。随着年岁的增长，愈发了解父爱母爱之深沉，儿子无以为报。

感谢从小到大的各位老同学们，这么多年大家对我的关心始终不减，我的每一步成长都离不开大家的关怀与鼓励。

感谢我的女朋友于晴，这篇论文是在你的支持下完成的，你对我的爱，我会永远铭记于心。

衷心感谢所有关心、帮助过我的师长、同学和亲友们！

刘屹冬

2019年12月22日

于上海交通大学徐汇校区

攻读学位期间的学术成果

- [1] 刘屹冬, 柳伟. 自适应的标准件三维 CAD 模型检索系统: 中国, CN201911186374.6[P]. 2019-11-28.
- [2] 刘屹冬, 柳伟. 三维模型中螺纹特征的及参数的快速识别提取方法: 中国, CN201911186436.3[P]. 2019-11-28.

上海交通大学

学位论文原创性声明

本人郑重声明：所呈交的学位论文《基于 pythonOCC 的零件报价系统研究与开发》，是本人在导师的指导下，独立进行研究工作所取得的成果。除文中已经注明引用的内容外，本论文不包含任何其他个人或集体已经发表或撰写过的作品成果。对本文的研究做出重要贡献的个人和集体，均已在文中以明确方式标明。本人完全意识到本声明的法律结果由本人承担。

学位论文作者签名：

刘屹冬

日期： 2020 年 2 月 29 日

